

LAPORAN KERJA PRAKTEK

**RANCANG BANGUN SISTEM MANAJEMEN SIKLUS HIDUP
PENGEMBANGAN PERANGKAT LUNAK MENGGUNAKAN REACT
DAN LARAVEL PADA PT BANK NAGARI**

PT BANK NAGARI

Periode 1 Juli 2026 - 31 Agustus 2026

Oleh

MUHAMMAD GALID AVERO

2311532008

Dosen Pembimbing

DR. DERISMA S.T., M.T.

NIP. 198204192010122001



**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

ABSTRAK

Tata kelola pengembangan perangkat lunak di lingkungan perbankan memerlukan alur yang terdokumentasi, pembagian wewenang yang tegas, kontrol keamanan, serta jejak audit yang dapat ditelusuri. Kerja Praktek ini berfokus pada pengembangan NagariSDLC, yaitu sistem informasi internal berbasis web untuk mengelola proyek teknologi sejak pengajuan kebutuhan sampai rilis ke produksi. Sistem dibangun menggunakan Laravel 13 pada PHP 8.3 sebagai REST API, React 19 dengan Vite 8 dan Tailwind CSS 4 sebagai antarmuka *Single Page Application*, serta MySQL melalui Eloquent ORM sebagai lapisan data. Metode pekerjaan meliputi studi dokumentasi dan literatur, analisis kebutuhan, penelusuran proses bisnis dan *source code*, perancangan model peran dan mesin status, implementasi modul proyek dan *task*, pengembangan *wizard System Integration Test* dan *User Acceptance Test*, pengujian paralel *Quality Assurance* dan keamanan siber, pengelolaan dokumen, persetujuan individual, serta gerbang rilis. Landasan analisis menggunakan standar ISO/IEC/IEEE mengenai *lifecycle*, *testing*, dan kualitas produk; panduan *NIST* dan *OWASP* mengenai *secure development* serta *security testing*; artikel jurnal mengenai *agile*, *continuous software engineering*, dan DevOps; serta buku rekayasa perangkat lunak dan arsitektur. Hasil pengembangan membentuk alur SDLC terpusat dengan 12 peran pengguna, 27 status proyek, pengujian *SIT/UAT* bertahap, dua jalur *QA* dan keamanan siber yang independen, putaran pengembalian yang tidak menghapus histori, dan *quality gate* sebelum produksi. Analisis menunjukkan bahwa pemusatan *state transition*, *project scope*, *approval round*, transaksi, dan *audit trail* mendukung *functional suitability*, *security*, *reliability*, serta *maintainability*, sementara *performance*, *usability formal*, *CI/CD*, monitoring, *backup*, dan kesiapan operasi produksi masih memerlukan verifikasi serta keputusan lanjutan. *Snapshot* kualitas tanggal 26 Agustus 2026 mencatat 236 pengujian backend dengan 1.467 asersi yang lulus, pemeriksaan tanpa *error*, dan build produksi *Vite* yang berhasil.

Kata kunci: *audit trail, Laravel, React, RBAC, SDLC, SIT, UAT.*

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Allah Swt. karena atas rahmat dan karunia-Nya laporan Kerja Praktek berjudul "Pengembangan Sistem Informasi Tata Kelola Siklus Hidup Pengembangan Perangkat Lunak (SDLC) Berbasis Web pada PT Bank Pembangunan Daerah Sumatera Barat (Bank Nagari)" dapat diselesaikan.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban akademik atas pelaksanaan Kerja Praktek pada Divisi Teknologi dan Digitalisasi terkhusus di Grup Pengembangan Teknologi dan Digitalisasi. Selama lebih kurang 2 bulan mulai dari 1 Juli sampai dengan 31 Agustus 2026 kegiatan tersebut memberikan kesempatan kepada penulis untuk memahami penerapan pengembangan perangkat lunak, tata kelola proyek, pengujian, keamanan, dokumentasi, dan kolaborasi lintas peran pada lingkungan kerja nyata.

Penulis menyampaikan terima kasih kepada:

1. Bapak Dr. Ir. Wahyudi S.T., M.T., selaku Ketua Departemen Informatika Universitas Andalas, yang senantiasa memberikan arahan dan kebijakan terbaik demi kelancaran kegiatan akademik mahasiswa.
2. Ibu Rahmi Eka Putri, S.Kom., M.T., selaku Sekretaris Departemen Informatika Universitas Andalas, atas dukungan administratif serta perhatian yang diberikan kepada mahasiswa.
3. Ibu Dr. Derisma, S.T., M.T., selaku Dosen Pembimbing Kerja Praktek, yang dengan penuh kesabaran memberikan masukan serta bimbingan dalam penyusunan laporan ini.
4. Bapak Yunasrul, selaku Kepala Divisi Teknologi dan Digitalisasi PT Bank Nagari, yang telah memberikan izin dan kesempatan untuk melaksanakan kerja praktek.
5. Bapak Afrizon Indra, selaku Pimpinan Grup Pengembangan, yang senantiasa mendukung proses pembelajaran di lingkungan kerja.
6. Bapak Ferry Arjendi Putra, selaku Pembimbing Lapangan, yang dengan telaten membimbing dan mengarahkan penulis selama menjalani kerja praktek.
7. Abang-abang dan Kakak-kakak dari Grup Pengembangan Teknologi dan Digitalisasi, Bagian Ketahanan dan Keamanan Siber, serta Grup Perencanaan dan *Quality Assurance*.
8. Kedua orang tua, atas doa, kasih sayang, serta dukungan yang tak pernah putus.
9. Rekan magang Raditya Putra Farma dan Devani, yang menjadi partner belajar sekaligus teman diskusi dalam menyelesaikan tugas harian.

karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan pada masa mendatang. Harapan terbesar penulis, semoga laporan ini dapat memberikan manfaat, baik bagi pengembangan keilmuan di bidang

informatika, maupun bagi institusi yang berkepentingan di dunia industri perbankan. Padang,

Padang, 31 Agustus 2026
Penulis

Muhammad Galid Avero

DAFTAR ISI

ABSTRAK.....	i
KATA PENGANTAR.....	ii
DAFTAR ISI.....	iv
DAFTAR TABEL.....	v
DAFTAR GAMBAR.....	v
DAFTAR LAMPIRAN.....	v
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang.....	1
1.2 Rumusan Masalah.....	2
1.3 Tujuan.....	2
1.4 Manfaat Kerja Praktek.....	2
1.4.1 Manfaat bagi Mahasiswa.....	2
1.4.2 Manfaat bagi Instansi.....	2
1.4.3 Manfaat bagi Perguruan Tinggi.....	3
1.5 Batasan Masalah.....	3
1.6 Metode Pelaksanaan.....	4
1.7 Sistematika Penulisan.....	4
BAB II PROFIL INSTANSI DAN PELAKSANAAN KERJA PRAKTEK.....	4
2.1 Profil Instansi.....	4
2.1.1 Identitas Instansi.....	4
2.1.2 Sejarah Singkat.....	5
2.1.3 Visi dan Misi.....	5
2.1.4 Logo dan Lokasi.....	6
2.2 Struktur Organisasi.....	6
2.3 Unit Kerja dan Posisi Mahasiswa.....	7
2.3.1 Unit Kerja.....	7
2.3.2 Posisi sebagai Pengembang Fullstack.....	8
2.3.3 Uraian Tugas.....	8
2.4 Jadwal dan Kegiatan Kerja Praktek.....	8
2.4.1 Kegiatan Mingguan.....	8
2.4.2 Hasil Kegiatan.....	9
BAB III TINJAUAN PUSTAKA.....	10
3.1 Sistem Informasi dan Tata Kelola Teknologi Informasi.....	10
3.2 Software Development Life Cycle.....	10
3.3 Model Pengembangan Perangkat Lunak.....	11
3.4 Agile, Continuous Software Engineering, dan DevOps.....	11
3.5 Workflow, State Machine, dan Quality Gate.....	12
3.6 Arsitektur Perangkat Lunak dan Pemisahan Tanggung Jawab.....	12
3.7 REST, HTTP, dan Desain API.....	13
3.8 Laravel sebagai Kerangka Back End.....	13
3.9 React, Vite, Tailwind CSS, dan Usability.....	14

3.10 Role-Based Access Control, Least Privilege, dan Separation of Duties.....	14
3.11 Autentikasi, Sesi, Cookie, CSRF, dan CORS.....	15
3.12 Basis Data Relasional, ORM, dan Integritas Transaksi.....	15
3.13 Konsep dan Tingkatan Pengujian Perangkat Lunak.....	16
3.14 System Integration Test dan User Acceptance Test.....	16
3.15 Quality Assurance dan Model Kualitas Produk.....	17
3.16 Secure SDLC dan Pengujian Keamanan Siber.....	17
3.17 Audit Trail, Traceability, dan Tata Kelola Dokumen.....	18
3.18 Pemodelan UML dan ERD.....	18
3.19 Notifikasi, Realtime, Queue, dan Operasional.....	18
3.20 Sintesis Literatur dan Posisi NagariSDLC.....	19
3.21 Perbandingan Model Pengembangan.....	19
3.22 Pemetaan Karakteristik Kualitas.....	20
3.23 Ringkasan Landasan Teori.....	20
3.24 Matriks Literatur dan Relevansi Penelitian.....	21
BAB IV HASIL DAN PEMBAHASAN.....	23
4.1 Hasil.....	23
4.1.1 Analisis Proses Bisnis.....	23
4.1.2 Aktor dan Lingkup Sistem.....	24
4.1.3 Kebutuhan Fungsional.....	25
4.1.4 Kebutuhan Nonfungsional.....	26
4.1.5 Arsitektur Sistem.....	26
4.1.6 Model Peran dan Use Case.....	27
4.1.7 Mesin Status Proyek.....	30
4.1.8 Alur System Integration Test.....	31
4.1.9 Alur User Acceptance Test.....	32
4.1.10 Matriks Persetujuan UAT.....	34
4.1.11 Jalur QA dan Keamanan Siber.....	35
4.1.12 Gerbang Rilis.....	38
4.1.13 Model Data dan Relationship.....	38
4.1.14 Desain API dan Sequence.....	41
4.1.15 Perancangan Navigasi dan Antarmuka.....	44
4.1.16 Implementasi Back End.....	53
4.1.17 Implementasi Front End.....	54
4.1.18 Keamanan dan Audit Trail.....	54
4.1.19 Pengujian dan Verifikasi.....	54
4.1.20 Matriks Keterlacakan Kebutuhan.....	55
4.1.21 Pemetaan Kontrol Keamanan.....	56
4.1.22 Evaluasi Kualitas Produk.....	57
4.1.23 Keputusan Arsitektur dan Trade-off.....	58
4.2 Pembahasan.....	59
4.2.1 Kesesuaian terhadap Tujuan.....	59
4.2.2 Kesesuaian dengan SDLC dan Literatur.....	60

4.2.3 Analisis Workflow dan State Machine.....	60
4.2.4 Analisis Kontrol Akses dan Separation of Duties.....	60
4.2.5 Analisis SIT, UAT, QA, dan Keamanan Siber.....	61
4.2.6 Analisis Keamanan Aplikasi dan Audit Trail.....	61
4.2.7 Analisis Data, Transaksi, dan Kompatibilitas.....	62
4.2.8 Analisis Antarmuka dan Pengalaman Pengguna.....	62
4.2.9 Kelebihan Sistem.....	63
4.2.10 Kekurangan dan Batasan.....	63
4.2.11 Kendala dan Solusi.....	64
4.2.12 Potensi Pengembangan.....	64
4.2.13 Kontribusi Mahasiswa.....	65
BAB V PENUTUP.....	66
5.1 Kesimpulan.....	66
5.2 Saran.....	66
DAFTAR PUSTAKA.....	4
LAMPIRAN.....	4
Lampiran A Endpoint API.....	70
Lampiran B Kamus Data Ringkas.....	72
Lampiran C Tampilan Lengkap.....	73

DAFTAR TABEL

Tabel 2.1 Identitas pelaksanaan Kerja Praktek.....	4
Tabel 2.2 Jadwal kegiatan Kerja Praktek.....	8
Tabel 3.1 Perbandingan model pengembangan.....	19
Tabel 3.2 Pemetaan model kualitas ISO/IEC 25010:2023.....	20
Tabel 3.3 Sintesis landasan teori.....	20
Tabel 3.4 Matriks literatur dan relevansi NagariSDLC.....	21
Tabel 4.1 Aktor dan tanggung jawab sistem.....	24
Tabel 4.2 Kebutuhan fungsional.....	25
Tabel 4.3 Kebutuhan nonfungsional.....	26
Tabel 4.4 Daftar 12 peran pengguna.....	27
Tabel 4.5 Daftar 27 status proyek.....	30
Tabel 4.6 Matriks approver UAT.....	34
Tabel 4.7 Domain utama.....	38
Tabel 4.8 Endpoint API representatif.....	41
Tabel 4.9 Ringkasan hasil verifikasi.....	54
Tabel 4.10 Matriks keterlacakan kebutuhan.....	55
Tabel 4.11 Pemetaan kontrol dan risiko keamanan.....	57
Tabel 4.12 Evaluasi kualitatif berdasarkan ISO/IEC 25010:2023.....	57
Tabel 4.13 Ringkasan keputusan arsitektur.....	58
Tabel 4.14 Kendala dan solusi.....	64
Tabel 4.15 Roadmap pengembangan.....	65
Tabel A.1 Daftar endpoint API.....	70
Tabel B.1 Kamus data ringkas.....	72

DAFTAR GAMBAR

Gambar 2.1 Logo Bank Nagari.....	6
Gambar 2.2 Gedung/lokasi Kerja Praktek.....	6
Gambar 2.3 Struktur organisasi instansi.....	7
Gambar 2.4 Struktur unit Teknologi Informasi.....	7
Gambar 4.1 Proses bisnis sebelum sistem terpusat.....	23
Gambar 4.2 Proses bisnis yang diusulkan.....	24
Gambar 4.3 Arsitektur sistem.....	27
Gambar 4.4 Use case aktor utama.....	29
Gambar 4.5 Mesin status proyek.....	31
Gambar 4.6 Alur SIT.....	32
Gambar 4.7 Alur UAT.....	33
Gambar 4.8 Relasi approver UAT.....	35
Gambar 4.9 Jalur QA dan keamanan siber.....	36
Gambar 4.10 Putaran pengembalian pengujian.....	37
Gambar 4.11 ERD NagariSDLC.....	39
Gambar 4.12 Class relationship inti.....	40
Gambar 4.13 Sequence login.....	42
Gambar 4.14 Sequence persetujuan UAT eksternal.....	43
Gambar 4.15 Peta navigasi halaman.....	44
Gambar 4.16 Antarmuka login.....	45
Gambar 4.17 Antarmuka dashboard Admin.....	45
Gambar 4.18 Daftar proyek.....	46
Gambar 4.19 Detail proyek.....	46
Gambar 4.20 Papan Kanban.....	47
Gambar 4.21 Detail task.....	47
Gambar 4.22 Wizard SIT.....	48
Gambar 4.23 Wizard UAT.....	48
Gambar 4.24 Tugas QA.....	49
Gambar 4.25 Tugas keamanan siber.....	49
Gambar 4.26 Matriks persetujuan UAT.....	50
Gambar 4.27 Quality Gate.....	50
Gambar 4.28 Manajemen pengguna.....	51
Gambar 4.29 Manajemen grup.....	51
Gambar 4.30 Manajemen role.....	52
Gambar 4.31 Document Vault.....	53
Gambar 4.32 Activity Log.....	53

DAFTAR LAMPIRAN

Lampiran A - Endpoint API.....	70
Lampiran B - Kamus Data Ringkas.....	72
Lampiran C - Tampilan Lengkap.....	73

BAB I

PENDAHULUAN

1.1 Latar Belakang

Transformasi digital membuat perangkat lunak menjadi bagian penting dalam operasional organisasi, termasuk perbankan. Aplikasi tidak hanya dituntut berfungsi, tetapi juga harus dikembangkan melalui proses yang aman, terdokumentasi, dapat diaudit, dan memiliki pembagian tanggung jawab yang jelas. Kebutuhan tersebut mendorong penerapan Software Development Life Cycle (SDLC) sebagai kerangka untuk mengelola pekerjaan sejak kebutuhan dirumuskan sampai sistem dioperasikan [1], [2].

Proses SDLC yang dikelola melalui dokumen dan komunikasi terpisah menimbulkan risiko ketidaksamaan informasi, sulitnya memantau status, keterlambatan persetujuan, serta hilangnya bukti keputusan. Pada proyek dengan banyak pemangku kepentingan, masalah tersebut semakin besar karena setiap tahap memerlukan pelaksana, reviewer, dokumen, dan keputusan yang berbeda. Sistem tata kelola terpusat dibutuhkan agar alur kerja tidak hanya terlihat, tetapi juga ditegakkan oleh aturan aplikasi.

PT Bank Pembangunan Daerah Sumatera Barat (Bank Nagari) menjadi tempat pelaksanaan Kerja Praktek. Proyek yang dikerjakan adalah NagariSDLC, yaitu sistem informasi internal untuk mengelola siklus proyek teknologi dari pengajuan kebutuhan, review, analisis, pengembangan, SIT, UAT, QA, audit keamanan siber, pengajuan rilis, sampai live production. Profil instansi pada laporan ini disusun berdasarkan informasi perusahaan dan dokumen pelaksanaan Kerja Praktek yang tersedia.

NagariSDLC dibangun sebagai aplikasi web tiga lapis. Antarmuka menggunakan React 19, Vite 8, dan Tailwind CSS 4; layanan aplikasi menggunakan Laravel 13 pada PHP 8.3; data dikelola pada MySQL melalui Eloquent ORM. Komunikasi antarlapis menggunakan REST API berbasis JSON. Autentikasi SPA menggunakan token Sanctum di dalam cookie HttpOnly, sedangkan header Authorization Bearer dipertahankan untuk kompatibilitas klien dan pengujian. Mesin status terpusat menjadi inti yang membatasi transisi, mencatat riwayat, dan menghasilkan notifikasi.

Dari sisi tata kelola, sistem memadukan Role-Based Access Control (RBAC), cakupan akses proyek, persetujuan individual, Document Vault, activity log, dan histori status. Dua jalur pengujian setelah UAT internal—QA dan keamanan siber—berjalan paralel serta memiliki status masing-masing. Setelah keduanya lulus, Project Manager mengajukan migrasi dan rilis, kemudian Head of IT mengambil keputusan pada quality gate. Tidak terdapat UAT final setelah QA dan keamanan siber.

Berdasarkan ruang lingkup tersebut, laporan ini disusun untuk mendokumentasikan latar belakang, landasan teori, analisis, perancangan,

implementasi, pengujian, dan hasil proyek. Kedalaman penyajian mengikuti pola laporan Kerja Praktek yang menjadi referensi pengguna, tetapi seluruh substansi teknis ditulis khusus berdasarkan dokumentasi dan implementasi NagariSDLC.

1.2 Rumusan Masalah

1. Bagaimana merancang sistem informasi yang memusatkan proses tata kelola proyek teknologi dari pengajuan sampai produksi?
2. Bagaimana menerapkan mesin status dan kontrol akses agar setiap transisi dilakukan oleh peran yang berwenang dan tercatat sebagai jejak audit?
3. Bagaimana mengelola SIT dan UAT bertahap, termasuk persetujuan individual serta revisi minor dan mayor?
4. Bagaimana mengelola QA dan keamanan siber sebagai dua jalur paralel yang independen tanpa menimbulkan status yang ambigu?
5. Bagaimana menyediakan antarmuka web, layanan API, dokumen, notifikasi, dan informasi proyek yang konsisten antara front end dan back end?

1.3 Tujuan

1. Membangun sistem informasi tata kelola SDLC berbasis web untuk mendukung proses proyek teknologi di Bank Nagari.
2. Menerapkan state machine proyek, RBAC, cakupan akses, histori status, dan activity log sebagai kontrol tata kelola.
3. Mengimplementasikan pengelolaan task, SIT, UAT, persetujuan UAT per orang, QA, keamanan siber, dan quality gate rilis.
4. Menyediakan antarmuka responsif berbahasa Indonesia dengan satu lapisan layanan API terpusat.
5. Mendokumentasikan arsitektur, alur, model data, hubungan peran, endpoint, pengujian, dan keterbatasan sistem secara terstruktur.

1.4 Manfaat Kerja Praktek

1.4.1 Manfaat bagi Mahasiswa

1. Memperoleh pengalaman menerapkan pengembangan fullstack pada aplikasi dengan aturan bisnis dan audit trail yang kompleks.
2. Meningkatkan kemampuan menelusuri kebutuhan, merancang model data, mengembangkan REST API, dan membangun antarmuka React.
3. Memahami praktik pengujian, keamanan aplikasi, dokumentasi teknis, dan kolaborasi lintas peran.

1.4.2 Manfaat bagi Instansi

1. Mendukung digitalisasi proses tata kelola proyek perangkat lunak melalui satu sumber informasi terpusat.
2. Meningkatkan keterlacakan status, dokumen, task, pengujian, persetujuan, dan keputusan rilis.

3. Mengurangi risiko transisi tidak sah melalui state machine, kontrol peran, dan validasi prasyarat.

1.4.3 Manfaat bagi Perguruan Tinggi

1. Menjadi bukti penerapan pengetahuan akademik pada proyek perangkat lunak di lingkungan industri.
2. Memperkaya contoh studi kasus mengenai SDLC governance, RBAC, workflow, pengujian, dan audit trail.
3. Mendukung hubungan kerja sama dan pertukaran pengetahuan antara perguruan tinggi dan instansi.

1.5 Batasan Masalah

1. Laporan membahas aplikasi NagariSDLC dan tidak membahas sistem inti perbankan maupun transaksi keuangan.
2. Daftar peran otorisasi dibatasi pada 12 role tetap yang aktif pada implementasi.
3. Grup kerja dipakai sebagai pengelompokan tampilan dan tidak menjadi sumber otorisasi fase.
4. Data SIT/UAT disimpan pada projects.sit_uat_data dengan approval aktif UAT pada uat_approval_rounds dan uat_approvers.
5. Realtime produksi, database produksi, queue, object storage, CI/CD, backup, monitoring, SLA, dan kebijakan retensi final masih memerlukan keputusan operasional.
6. Tangkapan layar dan diagram dibatasi pada bagian yang relevan dengan pembahasan serta tidak menampilkan data produksi atau informasi sensitif.
7. Snapshot pengujian yang dicantumkan merupakan kondisi historis tanggal 26 Agustus 2026 dan bukan hasil menjalankan test pada penyusunan laporan ini.
8. Diagram proses, UML, ERD, sequence, dan navigasi merepresentasikan rancangan serta implementasi pada saat laporan disusun.

1.6 Metode Pelaksanaan

Pelaksanaan Kerja Praktek dilakukan melalui tahapan observasi proses kerja dan identifikasi kebutuhan, penelaahan dokumentasi proyek serta literatur, analisis dan perancangan sistem, implementasi secara iteratif, pengujian, dan penyusunan dokumentasi. Setiap hasil analisis dikonfirmasi terhadap alur kerja NagariSDLC dan implementasi aktif agar rancangan antarmuka, layanan API, model data, kontrol akses, serta transisi status tetap konsisten.

1.7 Sistematika Penulisan

Laporan ini disusun dalam lima bab. Bab I menjelaskan latar belakang, rumusan masalah, tujuan, manfaat, batasan, metode pelaksanaan, dan sistematika penulisan. Bab II memuat profil instansi, unit kerja, posisi mahasiswa, serta kegiatan Kerja Praktek. Bab III menguraikan landasan teori dan penelitian yang mendukung pengembangan NagariSDLC. Bab IV menyajikan hasil analisis, perancangan, implementasi, pengujian, dan pembahasan sistem. Bab V berisi kesimpulan dan saran, kemudian dilanjutkan dengan daftar pustaka serta lampiran teknis.

BAB II

PROFIL INSTANSI DAN PELAKSANAAN KERJA PRAKTEK

2.1 Profil Instansi

2.1.1 Identitas Instansi

Kerja Praktek dilaksanakan pada Bank Nagari, bank pembangunan daerah yang berakar di Sumatera Barat. Situs resmi perusahaan mencantumkan kantor pusat Divisi Teknologi dan Digitalisasi di Pagambiran Ampalu Nan XX, Kec. Lubuk Begalung, Kota Padang, Sumatera Barat.

Tabel 2.1 Identitas pelaksanaan Kerja Praktek

Keterangan	Isi
Nama instansi	PT Bank Pembangunan Daerah Sumatera Barat (Bank Nagari)
Unit/divisi	Divisi Teknologi dan Digitalisasi
Alamat kantor pusat	Pagambiran Ampalu Nan XX, Kec. Lubuk Begalung, Kota Padang, Sumatera Barat
Periode	1 Juli 2026 - 31 Agustus 2026
Pembimbing lapangan	Ferry Arjendi Putra, S.Kom., CITPM
Posisi mahasiswa	Pengembang fullstack pada proyek NagariSDLC

2.1.2 Sejarah Singkat

Berdasarkan profil resmi perusahaan, Bank Pembangunan Daerah Sumatera Barat didirikan pada 12 Maret 1962 dan disahkan melalui akta notaris Hasan Qalbi di Padang. Pembentukannya dipelopori oleh pemerintah daerah bersama tokoh masyarakat dan pelaku usaha sebagai lembaga keuangan yang mendukung pembangunan daerah. Operasi awal berlokasi di Jalan Batang Arau Nomor 54 Padang dengan modal awal Rp50.000.000 [36].

Perusahaan membuka kantor cabang pertama di Payakumbuh pada 1965. Bentuk badan hukum kemudian berubah menjadi Perusahaan Daerah pada 1973, dan sebutan Bank Nagari mulai digunakan pada 1996 untuk memperkuat pengenalan merek. Pada 2006 badan hukum kembali diubah menjadi Perseroan Terbatas. Profil resmi juga mencatat keputusan perubahan nama perseroan menjadi PT Bank Nagari pada 2021 [36]. Uraian ini digunakan sebatas konteks instansi dan perlu dicocokkan kembali dengan laporan tahunan yang berlaku pada tahun pengesahan laporan.

Perjalanan tersebut menunjukkan bahwa Bank Nagari berkembang bersama perubahan kebutuhan kelembagaan dan layanan. Dalam konteks Kerja Praktek, pengembangan NagariSDLC dapat dipahami sebagai bagian dari upaya memperkuat proses internal berbasis teknologi: bukan sebagai aplikasi transaksi

nasabah, melainkan sebagai sarana tata kelola pengembangan sistem agar keputusan, pengujian, persetujuan, dan bukti proyek lebih terstruktur.

2.1.3 Visi dan Misi

Visi resmi Bank Nagari adalah menjadi bank pembangunan daerah yang terkemuka dan terpercaya di Indonesia [35]. Makna terkemuka berkaitan dengan pengenalan dan posisi bank pada tingkat nasional, sedangkan terpercaya dikaitkan dengan penerapan manajemen perusahaan yang baik, layanan yang memuaskan, kejujuran, dan kepatuhan.

Misi Bank Nagari menekankan kontribusi terhadap pertumbuhan ekonomi dan kesejahteraan masyarakat serta pemenuhan kepentingan para pemangku kepentingan secara konsisten dan seimbang [35]. Misi tersebut dijabarkan melalui upaya menjaga pertumbuhan bank yang sehat, memberikan layanan prima, memberikan keuntungan yang memadai bagi pemegang saham, dan memberikan manfaat maksimal bagi masyarakat.

Hubungan visi dan misi dengan proyek NagariSDLC bersifat tidak langsung tetapi relevan. Sistem tata kelola pengembangan perangkat lunak yang dapat diaudit mendukung konsistensi proses internal, memperjelas tanggung jawab, dan membantu menjaga mutu perubahan teknologi. Laporan ini tidak menyatakan bahwa NagariSDLC merupakan satu-satunya instrumen pencapaian visi perusahaan; sistem hanya menjadi salah satu sarana pendukung pengelolaan proyek teknologi.

2.1.4 Logo dan Lokasi



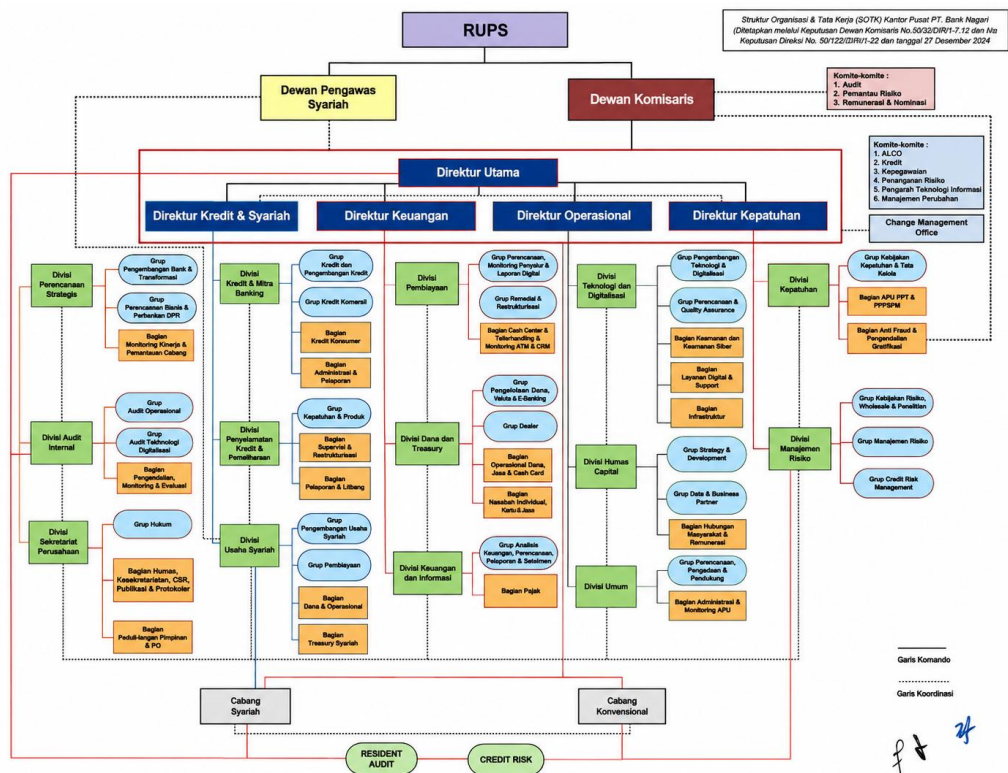
Gambar 2.1 Logo Bank Nagari



Gambar 2.2 Gedung/lokasi Kerja Praktek

2.2 Struktur Organisasi

Struktur Organisasi PT Bank Nagari disusun untuk mengatur pembagian tugas, wewenang dan tanggung jawab secara efektif, sekaligus memastikan kepatuhan terhadap prinsip Good Corporate Governance (GCG) serta regulasi perbankan yang berlaku. Penyusunan struktur ini mengacu pada ketentuan dari Otoritas Jasa Keuangan (OJK) dan Bank Indonesia (BI).



Gambar 2.3 Struktur organisasi instansi



Gambar 2.4 Struktur unit Teknologi Informasi

2.3 Unit Kerja dan Posisi Mahasiswa

2.3.1 Unit Kerja

Kegiatan dilaksanakan pada Divisi Teknologi dan Digitalisasi dan fokus di Grup Pengembangan Teknologi & Digitalisasi. Dalam konteks NagariSDLC, pekerjaan berhubungan dengan fungsi perencanaan, pengembangan, pengujian

mutu, keamanan siber, manajemen teknologi informasi, dan pemohon bisnis. Uraian ini menjelaskan domain aplikasi, bukan menetapkan struktur organisasi formal perusahaan.

2.3.2 Posisi sebagai Pengembang Fullstack

Penulis berperan sebagai pengembang fullstack yang mengerjakan sisi back end dan front end. Tanggung jawab back end mencakup model, migrasi, Form Request, controller, resource, service, state machine, autentikasi, dan pengujian otomatis. Tanggung jawab front end mencakup halaman, komponen, routing, role guard, context, layanan API, validasi antarmuka, dan integrasi dengan endpoint.

2.3.3 Uraian Tugas

1. Mempelajari kebutuhan produk, dokumentasi proyek, basis kode, dan model data yang aktif.
2. Mengembangkan dan menyempurnakan alur proyek, task, SIT, UAT, QA, keamanan siber, dan rilis.
3. Menjaga konsistensi kontrak field dan endpoint antara front end dan back end.
4. Menerapkan validasi, otorisasi, histori, notifikasi, dan perlindungan audit trail.
5. Melakukan pemeriksaan statis, pengujian terarah, dokumentasi, dan perbaikan berdasarkan temuan.

2.4 Jadwal dan Kegiatan Kerja Praktek

2.4.1 Kegiatan Mingguan

Tabel 2.2 Jadwal kegiatan Kerja Praktek

Minggu	Kegiatan	Luaran
1	Orientasi, pengenalan proses tata kelola SDLC bank, dan penyiapan lingkungan PHP 8.3, Laravel 13, React 19, serta MySQL.	Lingkungan kerja dan pemahaman domain awal.
2	Analisis kebutuhan, penelusuran basis kode, pemahaman model data, peran, dan alur proses.	Catatan analisis dan pemetaan modul.
3	Back end: model, migrasi, RBAC, dan autentikasi Sanctum.	Fondasi data, akses, dan sesi pengguna.
4	Back end: mesin status melalui ProjectWorkflowService, riwayat status, dan notifikasi.	Workflow terpusat dan audit trail status.
5	Back end: modul SIT/UAT, jalur QA dan keamanan siber, serta gerbang mutu rilis.	Alur pengujian dan quality gate.

Minggu	Kegiatan	Luaran
6	Front end: dashboard, papan Kanban, modul registrasi, dan manajemen pengguna.	Antarmuka kerja utama dan administrasi.
7	Front end: wizard SIT/UAT, halaman tugas QA/Siber, dan matriks persetujuan UAT.	Antarmuka pengujian dan approval.
8	Pengujian menyeluruh, perbaikan akhir, dokumentasi, dan finalisasi.	Snapshot kualitas dan dokumentasi proyek.

2.4.2 Hasil Kegiatan

Hasil kegiatan berupa implementasi aplikasi NagariSDLC, dokumentasi teknis, pemetaan workflow, perbaikan integrasi front end dan back end, serta bukti verifikasi yang tercatat pada dokumentasi proyek. Ringkasan kegiatan disajikan berdasarkan pekerjaan yang dapat diverifikasi melalui implementasi dan dokumentasi proyek.

BAB III

TINJAUAN PUSTAKA

3.1 Sistem Informasi dan Tata Kelola Teknologi Informasi

Sistem informasi merupakan kesatuan manusia, prosedur, data, perangkat lunak, perangkat keras, dan jaringan yang bekerja untuk menghasilkan informasi bagi operasi dan pengambilan keputusan. Karena itu, keberhasilan sistem tidak hanya ditentukan oleh kemampuan menyimpan data. Sistem harus membantu pengguna memahami pekerjaan, mengurangi ketidakpastian, menjaga kualitas informasi, dan menyediakan bukti yang cukup untuk mengevaluasi keputusan [1], [2].

Dalam konteks tata kelola teknologi informasi, proses pengembangan perangkat lunak perlu memiliki pemilik, pelaksana, reviewer, kriteria masuk dan keluar, serta bukti hasil. Tata kelola berbeda dari manajemen harian: tata kelola menetapkan arah, batas kewenangan, akuntabilitas, dan mekanisme pengawasan; manajemen melaksanakan pekerjaan di dalam batas tersebut. NagariSDLC menghubungkan keduanya melalui state machine, penugasan, persetujuan, histori, dan quality gate.

Proyek menjadi agregat utama pada NagariSDLC. Di sekelilingnya terdapat kebutuhan, task, anggota tim, dokumen, SIT, UAT, QA, pengujian keamanan siber, approval, release request, notifikasi, dan activity log. Struktur ini mengubah data yang semula berpotensi tersebar menjadi rangkaian informasi yang saling terkait. Nilai sistem terletak pada kemampuan merekonstruksi alasan sebuah proyek berada pada status tertentu dan tindakan apa yang masih diperlukan.

3.2 Software Development Life Cycle

Software Development Life Cycle (SDLC) adalah kerangka untuk mengelola perangkat lunak sejak gagasan dan kebutuhan, perancangan, implementasi, verifikasi, validasi, transisi, operasi, pemeliharaan, hingga penghentian. Sommerville serta Pressman dan Maxim menempatkan aktivitas spesifikasi, pengembangan, validasi, dan evolusi sebagai unsur penting rekayasa perangkat lunak [1], [2]. SDLC membuat pekerjaan kompleks dapat diuraikan menjadi keluaran, tanggung jawab, dan keputusan yang lebih terukur.

ISO/IEC/IEEE 12207:2026 menyediakan kerangka proses siklus hidup perangkat lunak yang dapat digunakan untuk pekerjaan internal maupun hubungan pemasok–pengakuisisi. Standar tersebut bersifat method-agnostic dan mengakui bahwa proses dapat dilakukan secara berulang, rekursif, serta paralel [15]. Artinya, disiplin lifecycle tidak identik dengan model air terjun; organisasi dapat menggunakan pendekatan iteratif selama tanggung jawab, hasil, dan keterlacakan tetap jelas.

NagariSDLC menerjemahkan lifecycle menjadi governance workflow. Tahap pengajuan, review, analisis, pengembangan, SIT, UAT, QA, keamanan siber, rilis, dan produksi memiliki aktor dan bukti masing-masing. Aplikasi tidak menentukan bagaimana developer mengorganisasi pekerjaan internal sehari-hari, tetapi menegaskan bahwa suatu quality gate hanya dapat dilewati ketika prasyarat yang disepakati telah terpenuhi.

3.3 Model Pengembangan Perangkat Lunak

Model sekuensial menekankan penyelesaian tahap secara relatif berurutan dan cocok ketika kebutuhan stabil serta bukti formal dibutuhkan. Model incremental membagi produk menjadi bagian yang dapat dikembangkan dan dievaluasi bertahap. Model spiral Boehm menempatkan identifikasi serta mitigasi risiko sebagai penggerak setiap putaran [29]. Model agile menekankan umpan balik cepat, kolaborasi, perangkat lunak yang bekerja, dan kemampuan merespons perubahan [25].

Tidak ada model yang selalu paling tepat. Penelitian sistematis *Dybå* dan *Dingsøyr* menunjukkan adanya manfaat sekaligus keterbatasan pendekatan *agile*, serta menekankan pentingnya konteks ketika menilai bukti empiris [26]. Proyek dengan tuntutan audit tinggi dapat membutuhkan dokumen dan approval formal, sementara ketidakpastian kebutuhan tetap menuntut iterasi. Oleh sebab itu, pendekatan hibrida sering lebih realistis daripada memaksakan satu label metode.

NagariSDLC dirancang untuk mendukung pola hibrida tersebut. Task dapat dikerjakan iteratif, revisi UAT dapat membuka pekerjaan perbaikan, dan jalur QA–Siber dapat berjalan paralel. Pada saat yang sama, sistem mempertahankan gerbang formal untuk *sign-off*, perubahan status, dan rilis. Dengan demikian fleksibilitas implementasi tidak menghilangkan akuntabilitas proses.

3.4 Agile, Continuous Software Engineering, dan DevOps

Agile Manifesto menghargai individu dan interaksi, perangkat lunak yang bekerja, kolaborasi dengan pengguna, serta respons terhadap perubahan tanpa meniadakan nilai proses dan dokumentasi [25]. Pada proyek yang diatur ketat, prinsip agile tidak boleh diterjemahkan sebagai penghapusan bukti. Dokumentasi perlu dibuat secukupnya, dekat dengan aktivitas yang dibuktikan, dan mudah diperbarui ketika keadaan berubah.

Fitzgerald dan Stol menjelaskan *continuous software engineering* sebagai upaya mengurangi keterputusan antara perencanaan, pengembangan, integrasi, pengujian, rilis, dan operasi [27]. Lwakatare dan rekan-rekan menunjukkan bahwa DevOps berkaitan dengan kemampuan memperbarui sistem operasional secara sering dan andal melalui praktik lintas fungsi [28]. Humble dan Farley menekankan pipeline rilis berulang dan berisiko rendah, sedangkan Forsgren, Humble, dan Kim menghubungkan praktik *delivery* dengan pengukuran kinerja teknologi serta organisasi [32], [33].

NagariSDLC belum merupakan platform CI/CD dan tidak mengotomatisasi deployment. Kontribusinya berada pada *continuity of governance*: hasil analisis mengalir ke pengembangan, task menjadi dasar SIT/UAT, hasil pengujian menjadi prasyarat rilis, dan keputusan produksi memiliki histori. Pengembangan lanjutan dapat mengintegrasikan pipeline otomatis, tetapi integrasi tersebut harus tetap tunduk pada quality gate dan otorisasi yang berlaku.

3.5 Workflow, State Machine, dan Quality Gate

Workflow menjelaskan aktivitas, urutan, pelaksana, input, keluaran, serta kondisi perpindahan. State machine memodelkan sistem sebagai himpunan keadaan dan transisi yang sah. Pemodelan ini bermanfaat ketika sebuah nilai status bukan sekadar label tampilan, tetapi menentukan aksi yang boleh dilakukan, aktor yang berwenang, dan bukti yang harus tersedia.

Quality gate adalah titik keputusan yang menahan perpindahan sampai kriteria tertentu dipenuhi. *Gate* yang baik harus memiliki kriteria objektif, pemilik keputusan, bukti, dan hasil yang dapat diaudit. Gate yang hanya ditampilkan pada antarmuka mudah dilewati melalui endpoint lain; karena itu aturan perlu ditegakkan pada lapisan domain atau service yang menjadi sumber kebenaran.

Pada NagariSDLC, *allowedTransitions* mendefinisikan bentuk perpindahan, *rolePermissions* membatasi pelaku, dan *validator* prasyarat menilai kondisi bisnis. Semua transisi proyek dipusatkan pada *ProjectWorkflowService*. Keputusan ini menghindari status yang diubah secara bebas oleh *controller* atau komponen UI dan memudahkan audit terhadap satu jalur perubahan yang konsisten.

3.6 Arsitektur Perangkat Lunak dan Pemisahan Tanggung Jawab

Arsitektur perangkat lunak menggambarkan struktur utama sistem, tanggung jawab elemen, serta hubungan di antaranya. Bass, Clements, dan Kazman menekankan bahwa keputusan arsitektur berkaitan erat dengan atribut kualitas seperti *modifiability*, *availability*, *security*, dan *performance* [30]. Arsitektur yang baik bukan sekadar diagram teknologi; keputusan harus menjelaskan *trade-off* yang ingin dikelola.

Pemisahan *presentation*, *application/domain logic*, dan *persistence* mengurangi ketergantungan langsung. Perubahan tampilan tidak seharusnya mengubah aturan *workflow*, sedangkan perubahan *database* tidak seharusnya memaksa komponen UI mengetahui detail penyimpanan. Batas yang jelas juga mempermudah pengujian karena perilaku bisnis dapat diuji melalui *service* atau API tanpa bergantung pada seluruh antarmuka.

NagariSDLC menggunakan React SPA sebagai lapis klien, Laravel REST API sebagai lapis aplikasi, dan MySQL/Eloquent sebagai lapis data. Di back end, Form Request menangani validasi input, controller mengorkestrasi request, service menampung aturan lintas endpoint, model merepresentasikan data, dan

Resource menormalisasi respons. Struktur tersebut meningkatkan maintainability walaupun tetap membutuhkan disiplin agar *service* tidak berkembang menjadi objek yang terlalu besar.

3.7 REST, HTTP, dan Desain API

REST merupakan gaya arsitektur untuk sistem jaringan yang menekankan *client-server*, *stateless interaction*, *cacheability*, *uniform interface*, *layered system*, dan *code-on-demand* opsional [3]. HTTP sendiri merupakan protokol *application-level* yang *stateless* dengan pola *request-response*, *resource* yang diidentifikasi URI, *method* yang menyatakan maksud, serta status code yang menyampaikan hasil [22]. REST bukan sinonim dari JSON, tetapi JSON umum digunakan sebagai representasi data.

Desain API perlu konsisten pada penamaan *resource*, penggunaan *method*, *status code*, format validasi, *pagination*, dan penanganan *error*. *Request* yang valid secara sintaksis masih dapat ditolak oleh aturan bisnis; dalam kasus tersebut respons harus membedakan kegagalan autentikasi, otorisasi, data tidak ditemukan, dan prasyarat *workflow*. Konsistensi kontrak mengurangi logika khusus pada klien dan mempercepat diagnosis integrasi.

NagariSDLC memakai REST API berbasis JSON dengan *envelope* { *status*, *message*, *data* }. Kode 401 digunakan untuk sesi tidak sah, 403 untuk larangan akses, 404 untuk resource yang tidak ditemukan, dan 422 untuk validasi atau aturan workflow. Front end memusatkan request pada satu service API sehingga *cookie*, *header*, serialisasi, *parsing error*, dan perilaku *logout* dapat dikelola secara seragam.

3.8 Laravel sebagai Kerangka Back End

Laravel menyediakan *routing*, *middleware*, *Form Request*, *Eloquent ORM*, *API Resource*, *event*, *notification*, *queue*, *broadcasting*, *cache*, dan fasilitas pengujian [4]. Kerangka ini mempercepat pengembangan, tetapi kualitas hasil tetap ditentukan oleh penempatan tanggung jawab. *Controller* yang terlalu banyak memuat aturan bisnis akan sulit digunakan ulang dan diuji, sedangkan model yang memuat seluruh orkestrasi dapat menimbulkan ketergantungan tersembunyi.

Validasi write pada Form Request memisahkan pemeriksaan bentuk input dari orkestrasi *endpoint*. Service digunakan ketika aturan melibatkan beberapa model, transaksi, notifikasi, dan perubahan status. Resource membentuk kontrak output sehingga perubahan internal model tidak langsung bocor ke klien. Pola ini sejalan dengan kebutuhan NagariSDLC yang memiliki alur *approval* dan pengujian lintas tabel.

ProjectWorkflowService menjadi pusat transisi status proyek. *UatExecutionService* menangani draft/final eksekusi dan revisi; *UatApprovalService* mengelola putaran persetujuan; *TestingTrackService* mengelola jalur QA dan keamanan siber; *ProjectReturnRoundService* menjaga

putaran pengembalian. Pembagian ini membuat aturan audit kritis memiliki lokasi yang dapat ditelusuri.

3.9 React, Vite, Tailwind CSS, dan Usability

React membangun antarmuka melalui komponen deklaratif dan state yang mendorong pembentukan UI berdasarkan data [5]. Vite menyediakan development server serta proses build [6], sedangkan Tailwind CSS menyediakan *utility class* untuk menyusun tampilan [8]. Kombinasi tersebut mendukung SPA yang responsif, tetapi perlu aturan struktur agar komponen tidak terlalu terikat pada pemanggilan API dan logika bisnis.

Nielsen menjelaskan *usability* melalui kemudahan dipelajari, efisiensi, kemudahan diingat, tingkat kesalahan, dan kepuasan [34]. Pada sistem *workflow*, *usability* juga dipengaruhi kejelasan status, aksi berikutnya, alasan sebuah aksi ditolak, serta konsistensi istilah. *Wizard* membantu memecah proses panjang, sedangkan pencarian dan filter membantu pengguna menemukan pekerjaan sesuai perannya.

Front end NagariSDLC menggunakan *router*, *protected route*, *context* untuk state lintas komponen, dan service API terpusat. Antarmuka berbahasa Indonesia, menampilkan action sesuai *role*, serta menerima indikator *can_resubmit* dan *resubmit_blocker* dari server. UI mencegah aksi yang pasti tidak sah, tetapi otorisasi akhir tetap berada di back end agar penyembunyian tombol tidak disalahartikan sebagai kontrol keamanan.

3.10 Role-Based Access Control, Least Privilege, dan Separation of Duties

RBAC menghubungkan permission dengan role dan menugaskan pengguna pada role tersebut. Model Sandhu dan rekan-rekan membedakan role dari group serta menunjukkan bagaimana hierarki dan constraint dapat membentuk kebijakan yang lebih kaya [21]. RBAC mengurangi kebutuhan mengelola izin satu per satu, tetapi role yang terlalu luas dapat menciptakan hak akses berlebihan.

Prinsip *least privilege* memberikan akses minimum yang diperlukan untuk menjalankan tugas. *Separation of duties* memisahkan aktivitas yang berpotensi menimbulkan konflik, misalnya pelaksana pengujian dan pemberi *sign-off*. Pada aplikasi proyek, role saja belum cukup karena dua pengguna dengan role sama belum tentu terlibat pada proyek yang sama. Otorisasi perlu menggabungkan *role*, *assignment*, *membership*, kepemilikan, dan keadaan *workflow*.

NagariSDLC memiliki 12 role tetap dan lima group bawaan. Group hanya mengelompokkan *role* untuk tampilan; group tidak memberi wewenang fase. *ProjectWorkflowService* mengatur hak transisi, *ProjectAccessService* membatasi cakupan proyek, dan *service* pengujian membatasi *assignee*. Model ini memperkecil risiko bahwa perubahan menu atau group tanpa sengaja mengubah otorisasi inti.

3.11 Autentikasi, Sesi, Cookie, CSRF, dan CORS

Autentikasi membuktikan identitas, sedangkan manajemen sesi mempertahankan konteks pengguna pada request berikutnya. RFC 6265 menjelaskan mekanisme server mengirim *Set-Cookie* dan *user agent* mengembalikan *Cookie* pada request selanjutnya [23]. Atribut *HttpOnly* mencegah JavaScript membaca *cookie*, *Secure* membatasi pengiriman melalui koneksi aman, dan *SameSite* membantu mengendalikan pengiriman lintas situs. Tidak satu pun atribut tersebut menggantikan validasi *server*.

Pada aplikasi SPA, risiko utama meliputi pencurian token melalui XSS, *cross-site request forgery*, *origin* yang terlalu longgar, *session fixation*, serta sesi yang tidak dicabut. OWASP ASVS menyediakan persyaratan verifikasi untuk autentikasi, session management, *access control*, validasi, dan kontrol keamanan web [20]. NIST SSDF menempatkan keamanan sebagai praktik yang perlu diintegrasikan ke dalam seluruh *lifecycle*, bukan pemeriksaan akhir [18].

NagariSDLC menggunakan token *Sanctum* di *cookie HttpOnly* sebagai jalur utama SPA, *credentials include* pada request, dan *X-Requested-With* untuk jalur *cookie* [11]. Token tidak disimpan di *localStorage*. *Header Authorization Bearer* dipertahankan untuk kompatibilitas klien serta pengujian. CORS dibatasi melalui konfigurasi environment. Rancangan ini harus dilengkapi konfigurasi HTTPS, *domain cookie*, *SameSite*, rotasi token, *rate limiting*, dan *monitoring* pada produksi.

3.12 Basis Data Relasional, ORM, dan Integritas Transaksi

Basis data relasional merepresentasikan entitas dalam tabel dan hubungan melalui *primary key* serta *foreign key*. Normalisasi membantu mengurangi redundansi dan anomali pembaruan, sedangkan indeks mempercepat pola pencarian tertentu. *Constraint* menjaga kondisi yang dapat ditegakkan pada tingkat database. MySQL menyediakan transaksi dan integritas referensial yang dibutuhkan untuk operasi multi-tabel [12].

ORM memetakan tabel ke objek dan relasi sehingga kode aplikasi lebih ekspresif. Namun ORM tidak menghilangkan kebutuhan memahami *query*, *eager loading*, *indeks*, transaksi, dan perilaku *delete*. Pilihan *CASCADE*, *SET NULL*, dan *RESTRICT* memiliki makna domain: *CASCADE* sesuai untuk anak yang tidak bermakna tanpa induk; *SET NULL* sesuai untuk referensi penugasan yang boleh hilang; *RESTRICT* sesuai untuk bukti audit yang tidak boleh terhapus diam-diam.

NagariSDLC menggunakan transaksi ketika satu aksi mengubah proyek, status jalur, histori, *approval*, return round, atau notifikasi. *Soft delete* diterapkan pada entitas yang perlu dipulihkan atau ditelusuri. *Data wizard* SIT/UAT berada pada *projects.sit_uat_data* untuk kompatibilitas evolusi, sedangkan *approval* aktif UAT dinormalisasi ke *uat_approval_rounds* dan *uat_approvers* agar *snapshot* serta keputusan per orang dapat diaudit.

3.13 Konsep dan Tingkatan Pengujian Perangkat Lunak

Pengujian adalah kegiatan mengevaluasi artefak perangkat lunak untuk menemukan perbedaan antara hasil aktual dan hasil yang diharapkan, serta memberikan informasi tentang kualitas dan risiko. Pengujian tidak dapat membuktikan ketiadaan seluruh cacat, tetapi dapat meningkatkan keyakinan melalui cakupan yang dirancang sesuai risiko. Verifikasi menilai apakah produk dibangun dengan benar terhadap spesifikasi; validasi menilai apakah produk yang dibangun sesuai kebutuhan pengguna [1], [2].

ISO/IEC/IEEE 29119-2:2021 mendefinisikan proses pengujian generik yang dapat digunakan untuk mengatur, mengelola, dan melaksanakan testing pada berbagai model *lifecycle* [17]. Proses tersebut menghubungkan perencanaan, monitoring, desain, implementasi, eksekusi, pelaporan insiden, dan penyelesaian. Test case perlu memiliki tujuan, prasyarat, data, langkah, hasil yang diharapkan, hasil aktual, bukti, dan status.

Unit test memeriksa bagian kecil; *integration test* memeriksa interaksi komponen; *system test* memeriksa sistem lengkap; *acceptance test* menilai penerimaan pemangku kepentingan. *Black-box* berfokus pada *input* dan *output*, *white-box* menggunakan struktur internal, sedangkan *risk-based testing* memprioritaskan area berdasarkan kemungkinan dan dampak kegagalan. NagariSDLC membutuhkan kombinasi tersebut karena risiko utamanya terdapat pada *workflow*, otorisasi, konsistensi data, dan *audit trail*.

3.14 System Integration Test dan User Acceptance Test

SIT memverifikasi bahwa komponen yang telah dikembangkan dapat bekerja sebagai kesatuan, termasuk interaksi API, *database*, layanan, dan antarmuka. UAT dilakukan dari sudut pandang kebutuhan bisnis untuk menentukan apakah sistem dapat diterima. Keduanya berbeda dari unit test: SIT dan UAT membutuhkan skenario *end-to-end*, data uji, lingkungan, bukti, serta pihak yang bertanggung jawab terhadap hasil [17].

UAT bukan hanya satu tombol persetujuan. Proses penerimaan perlu menjelaskan siapa peserta dan penanda tangan, skenario apa yang diuji, bukti apa yang dihasilkan, bagaimana revisi dicatat, serta apakah keputusan berlaku pada versi hasil yang sama. Ketika hasil berubah setelah revisi, approval lama tidak boleh dianggap mewakili artefak terbaru; diperlukan putaran baru atau mekanisme supersession.

NagariSDLC membagi SIT dan UAT menjadi tiga tahap. SIT memerlukan bukti per task dan sign-off developer, PM, serta Development Lead. UAT mengelola peserta, undangan, skenario, *Change Request*, dan tujuh peran *approval*. Revisi *minor* menahan approval sampai *task* perbaikan selesai; revisi mayor mengulang SIT penuh dan UAT dari tahap pertama. Putaran lama disimpan sebagai histori.

3.15 Quality Assurance dan Model Kualitas Produk

Quality Assurance (QA) berorientasi pada keyakinan bahwa proses dan produk memenuhi kebutuhan mutu, sedangkan *quality control* lebih berfokus pada pemeriksaan hasil. QA tidak sama dengan *testing*; *testing* adalah salah satu teknik yang memberikan bukti. Praktik QA juga mencakup *review* kebutuhan, standar pengembangan, definisi selesai, pengelolaan cacat, keterlacakan, dan evaluasi proses.

ISO/IEC 25010:2023 mendefinisikan model kualitas produk yang terdiri atas sembilan karakteristik: *functional suitability*, *performance efficiency*, *compatibility*, *interaction capability*, *reliability*, *security*, *maintainability*, *flexibility*, dan *safety* [16]. Karakteristik tersebut membantu mengubah istilah 'berkualitas' menjadi sasaran yang dapat dijelaskan dan diuji. Tidak semua karakteristik memiliki bobot sama; prioritas mengikuti konteks dan risiko produk.

Pada NagariSDLC, *functional suitability* berkaitan dengan kelengkapan *workflow*; *security* dengan autentikasi, otorisasi, serta audit; *reliability* dengan transaksi dan konsistensi; *maintainability* dengan pemisahan *service*; *interaction capability* dengan *wizard* dan pesan status; *compatibility* dengan dukungan data *legacy* serta *Bearer token*. Pengukuran kuantitatif penuh belum dilakukan, sehingga laporan membahas alignment desain dan bukti yang tersedia, bukan mengklaim sertifikasi ISO.

3.16 Secure SDLC dan Pengujian Keamanan Siber

NIST *Secure Software Development Framework* versi 1.1 mengelompokkan praktik ke dalam *Prepare the Organization*, *Protect the Software*, *Produce Well-Secured Software*, dan *Respond to Vulnerabilities* [18]. Kerangka tersebut dirancang agar dapat ditambahkan pada berbagai model SDLC. Tujuannya mengurangi jumlah kerentanan yang dirilis, mengurangi dampak eksploitasi, serta menangani akar penyebab agar masalah serupa tidak berulang.

NIST SP 800-115 membahas perencanaan assessment, pelaksanaan pengujian teknis, analisis temuan, dan pengembangan strategi mitigasi [19]. OWASP WSTG menyediakan kategori pengujian web seperti *identity*, *authentication*, *authorization*, *session*, *input validation*, *error handling*, *business logic*, *client-side*, dan API [7]. ASVS melengkapi WSTG sebagai basis persyaratan verifikasi kontrol [20].

NagariSDLC memisahkan QA dan keamanan siber menjadi dua jalur independen. Pemisahan ini menghindari asumsi bahwa lulus pengujian fungsional berarti aman. Setiap jalur memiliki pengajuan, disposisi, laporan pelaksana, *sign-off Lead*, kegagalan, *return round*, *task* perbaikan, dan *resubmission gate*. *Tested scenarios* disimpan sebagai teks karena ruang lingkup perlu disesuaikan dengan risiko proyek.

3.17 Audit Trail, Traceability, dan Tata Kelola Dokumen

Traceability menghubungkan kebutuhan dengan desain, implementasi, pengujian, temuan, perbaikan, dan keputusan rilis. Keterlacakan dua arah membantu menjawab dua pertanyaan: apakah setiap kebutuhan telah diwujudkan dan diuji, serta apakah setiap artefak implementasi memiliki alasan kebutuhan. Pada proyek yang melibatkan banyak approval, hubungan tersebut mengurangi risiko keputusan berdasarkan versi artefak yang keliru.

Audit trail merekam siapa melakukan apa, kapan, terhadap objek mana, dari kondisi apa menjadi kondisi apa, dan dengan hasil apa. *Audit trail* harus dilindungi dari perubahan atau penghapusan yang tidak sah. *Soft delete*, *immutable event*, *status superseded*, *foreign key RESTRICT*, dan *snapshot approval* adalah beberapa mekanisme untuk menjaga konteks historis.

NagariSDLC menyediakan *project_status_histories*, *activity_logs*, *test_reports*, *uat_approval_rounds*, *uat_approvers*, *project_return_rounds*, *release_requests*, dan *Document Vault*. Putaran approval yang tidak berlaku ditandai *superseded*, bukan dihapus. Baris approved tetap disimpan. Dokumen yang masih menjadi *evidence* tidak dapat dihapus sembarangan. Rancangan ini memungkinkan kronologi keputusan direkonstruksi sekaligus mempertahankan kompatibilitas data lama.

3.18 Pemodelan UML dan ERD

UML merupakan bahasa pemodelan standar untuk merepresentasikan struktur dan perilaku sistem. Spesifikasi UML 2.5.1 mencakup elemen untuk *use case*, *activity*, *state machine*, *sequence*, *class*, dan bentuk model lain [24]. Fowler menekankan bahwa diagram seharusnya dipilih sesuai tujuan komunikasi, bukan dibuat sebanyak mungkin [31]. Model yang baik menyederhanakan aspek penting tanpa mengklaim menampilkan seluruh detail implementasi.

ERD menggambarkan entitas, atribut kunci, dan kardinalitas hubungan data. *State diagram* menyoroti status serta transisi; *sequence diagram* menjelaskan urutan pesan; *use case* menjelaskan interaksi aktor dengan tujuan sistem; *class diagram* menjelaskan struktur tipe serta relasi. Satu diagram tidak dapat menggantikan diagram lain karena masing-masing menjawab pertanyaan berbeda.

3.19 Notifikasi, Realtime, Queue, dan Operasional

Notifikasi membantu pengguna mengetahui perubahan yang memerlukan perhatian, tetapi volume yang berlebihan dapat menimbulkan *alert fatigue*. Notifikasi perlu memiliki penerima yang tepat, konteks proyek, jenis peristiwa, status baca, dan tautan menuju tindakan. *Event* bisnis sebaiknya dipisahkan dari cara penyampaian agar satu kejadian dapat dikirim melalui database notification, email, atau kanal *realtime* tanpa menduplikasi aturan domain.

Queue memindahkan pekerjaan yang tidak harus selesai dalam response request, misalnya email, pemrosesan file, dan integrasi eksternal. *Broadcasting*

mengirim pembaruan ke klien secara *realtime*. Pola ini meningkatkan responsivitas, tetapi menambah kebutuhan *worker*, *retry*, *idempotency*, *observability*, dan pengelolaan kegagalan [4], [32].

NagariSDLC telah memiliki notification database dan rancangan Reverb, tetapi konfigurasi environment aktif masih menggunakan *broadcast log*. Karena itu laporan tidak mengklaim realtime produksi telah berjalan. Chat menggunakan polling. Aktivasi *Reverb* dan *queue worker* perlu didahului keputusan arsitektur produksi, kapasitas, keamanan koneksi, monitoring, serta prosedur pemulihan.

3.20 Sintesis Literatur dan Posisi NagariSDLC

Literatur menunjukkan tiga kebutuhan yang sering tarik-menarik: fleksibilitas menghadapi perubahan, disiplin *lifecycle* untuk menghasilkan bukti, dan integrasi keamanan sejak awal. *Agile* serta *continuous software engineering* mendorong umpan balik dan aliran kerja yang tidak terputus [25]–[28]. ISO 12207 dan 29119 menekankan proses lifecycle serta testing yang dapat ditata [15], [17]. SSDF, ASVS, dan WSTG menempatkan keamanan sebagai tanggung jawab sepanjang pengembangan [7], [18]–[20].

NagariSDLC mengambil posisi sebagai *governance-enabling system*. Sistem tidak menggantikan keahlian analis, developer, *tester*, *pentester*, PM, atau pengambil keputusan. Sistem membuat batas wewenang, urutan, prasyarat, *evidence*, dan histori dapat dilihat serta ditegakkan. Nilainya tidak hanya berupa otomatisasi administrasi, tetapi pengurangan ruang bagi transisi tanpa bukti dan keputusan yang tidak memiliki konteks.

3.21 Perbandingan Model Pengembangan

Tabel 3.1 Perbandingan model pengembangan

Model	Konteks	Kekuatan	Risiko/Keterbatasan
Sekuensial	Kebutuhan stabil dan gate formal	Dokumen dan tahap jelas	Perubahan terlambat relatif mahal
<i>Incremental</i>	Produk dapat dibagi menjadi bagian	Nilai dapat diberikan bertahap	Integrasi dan prioritas perlu dijaga
<i>Spiral</i>	Risiko tinggi dan ketidakpastian	Risiko menjadi penggerak iterasi	Memerlukan kemampuan analisis risiko
<i>Agile</i>	Perubahan dan umpan balik sering	Adaptif dan kolaboratif	Butuh disiplin teknis dan keterlibatan aktif
<i>Hibrida governance</i>	Kebutuhan iteratif dengan audit formal	Fleksibel tetapi tetap terlacak	Gate dapat menjadi bottleneck bila kriterianya kabur

3.22 Pemetaan Karakteristik Kualitas

Tabel 3.2 Pemetaan model kualitas ISO/IEC 25010:2023

Karakteristik ISO 25010	Makna pada Sistem	Bukti/Implikasi NagariSDLC
<i>Functional suitability</i>	Cakupan fungsi dan ketepatan workflow	Kebutuhan fungsional, status, prasyarat, dan test case
<i>Performance efficiency</i>	Waktu respons dan penggunaan sumber daya	Belum diukur formal; perlu load/performance test
<i>Compatibility</i>	Koeksistensi dan interoperabilitas	REST/JSON, Bearer compatibility, pembacaan data legacy
<i>Interaction capability</i>	Kemudahan interaksi pengguna	Wizard, filter, pesan blocker, Bahasa Indonesia
<i>Reliability</i>	Konsistensi dan pemulihan	Transaksi, constraint, retry operasional yang direncanakan
<i>Security</i>	Kerahasiaan, integritas, akuntabilitas	HttpOnly, RBAC, project scope, audit trail, jalur Siber
<i>Maintainability</i>	Kemudahan analisis dan perubahan	Form Request, service, Resource, API terpusat
<i>Flexibility</i>	Kemampuan beradaptasi	Status paralel, return round, JSON wizard kompatibel
<i>Safety</i>	Pencegahan dampak yang tidak dapat diterima	Tidak dinilai sebagai sistem safety-critical; release gate mengurangi risiko perubahan

3.23 Ringkasan Landasan Teori

Tabel 3.3 Sintesis landasan teori

Konsep	Pokok Literatur	Penerapan pada NagariSDLC
SDLC dan standar <i>lifecycle</i>	Proses dari kebutuhan hingga operasi dapat iteratif dan paralel	<i>Workflow end-to-end</i> dan <i>quality gate</i>
<i>Agile/continuous engineering</i>	Umpan balik, adaptasi, dan kontinuitas aktivitas	Task iteratif serta revisi tanpa kehilangan histori
<i>Arsitektur dan REST</i>	Pemisahan tanggung jawab dan antarmuka seragam	React–Laravel–MySQL dan envelope API

Konsep	Pokok Literatur	Penerapan pada NagariSDLC
<i>RBAC dan least privilege</i>	Akses berdasarkan peran, assignment, serta constraint	12 role, project scope, dan separation of duties
Testing dan kualitas	Bukti berbasis risiko pada berbagai tingkat	SIT/UAT bertahap dan QA independen
<i>Secure SDLC</i>	Keamanan terintegrasi sepanjang lifecycle	Jalur Siber, HttpOnly, audit, dan return round
<i>Traceability</i>	Kebutuhan–implementasi–test–keputusan terhubung	Histori, approval round, dokumen, dan release request
UML	Model dipilih sesuai pertanyaan yang dijawab	Flow, state, sequence, class, use case, dan ERD

3.24 Matriks Literatur dan Relevansi Penelitian

Tabel 3.4 Matriks literatur dan relevansi NagariSDLC

Sumber	Fokus	Temuan/Gagasan	Relevansi
Boehm (1988) [29]	Spiral berbasis risiko	Risiko perlu mendorong iterasi	Return round dan revisi mayor memicu siklus kerja baru
Dybå & Dingsøyr (2008) [26]	Systematic review agile	Manfaat agile bergantung konteks	Governance tidak mengunci satu metode pengembangan
Fitzgerald & Stol (2017) [27]	Continuous software engineering	Kurangi keterputusan antarfasa	Data analisis–task–test–rilis berada pada satu rantai
Lwakatare et al. (2019) [28]	Multiple case study DevOps	Kolaborasi development–operations mendukung update andal	Menjadi dasar roadmap integrasi CI/CD dan operasi
Sandhu et al. (1996) [21]	Model RBAC	Role, permission, hierarchy, dan constraint	12 role ditambah cakupan proyek serta assignment
NIST SSDF (2022) [18]	Secure development practices	Security terintegrasi pada SDLC	Jalur Siber, kontrol sesi, audit, dan roadmap secure pipeline

Sumber	Fokus	Temuan/Gagasan	Relevansi
ISO 29119-2 (2021) [17]	Proses pengujian	Testing perlu direncanakan, dilaksanakan, dan dilaporkan	Wizard SIT/UAT serta laporan QA/Siber
ISO 25010 (2023) [16]	Model kualitas produk	Kualitas dinilai melalui karakteristik	Evaluasi kualitatif dan identifikasi metrik yang belum ada

Matriks menunjukkan bahwa NagariSDLC tidak dibangun dari satu teori tunggal. Model risiko menjelaskan perlunya siklus revisi; agile dan continuous engineering menjelaskan kebutuhan adaptasi serta aliran informasi; RBAC menjelaskan pembagian akses; standar testing dan secure development menjelaskan evidence serta keamanan; model kualitas menyediakan kerangka evaluasi. Sintesis lintas sumber tersebut menjadi dasar pembahasan Bab IV.

BAB IV

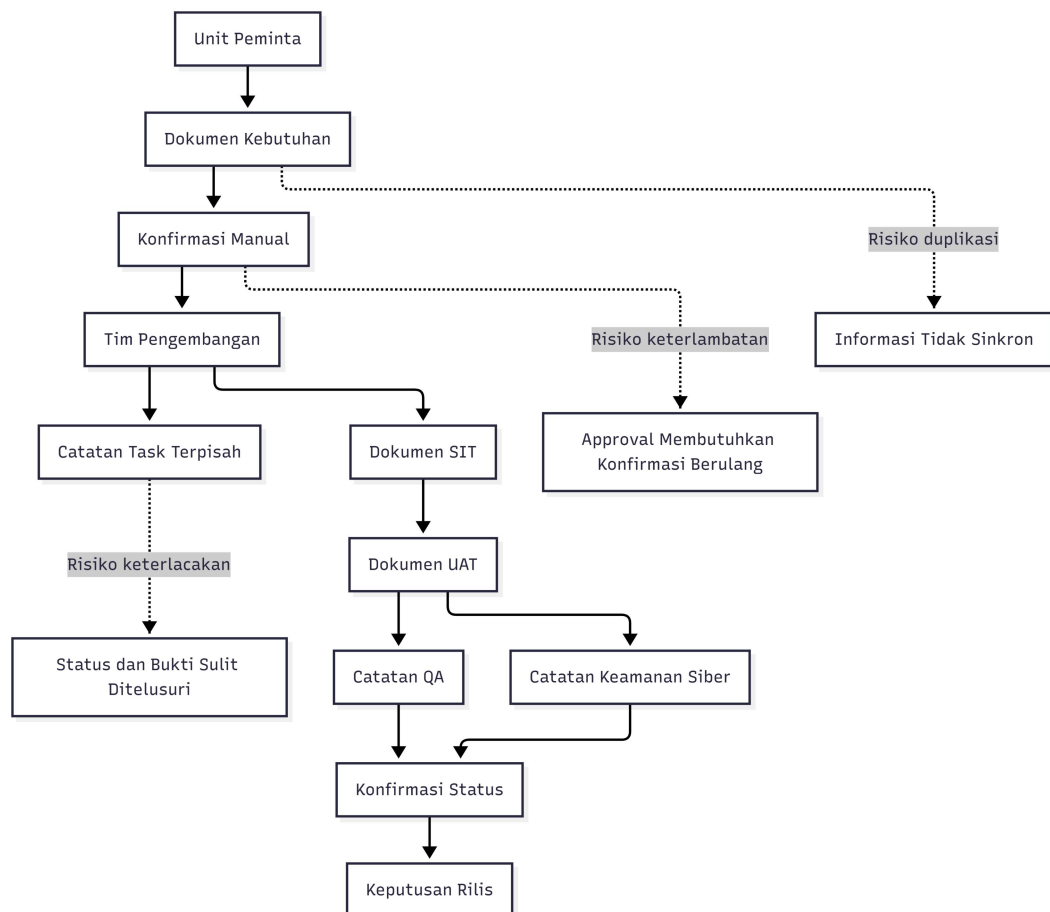
HASIL DAN PEMBAHASAN

4.1 Hasil

Hasil Kerja Praktek berupa sistem informasi NagariSDLC beserta dokumentasi alur, model data, endpoint, dan mekanisme pengujian. Bagian ini mengikuti urutan analisis proses bisnis, perancangan sistem, implementasi, dan verifikasi agar hubungan antara kebutuhan dan hasil dapat ditelusuri.

4.1.1 Analisis Proses Bisnis

Sebelum tersedia satu sistem governance terpusat, informasi kebutuhan, status, task, dokumen, pengujian, dan approval berisiko tersebar pada media yang berbeda. Kondisi tersebut membuat pemangku kepentingan harus melakukan konfirmasi manual dan menyulitkan pelacakan bukti keputusan. Gambar 4.1 menggambarkan kondisi konseptual tersebut dan bukan klaim mengenai prosedur resmi instansi yang belum diberikan.



Gambar 4.1 Proses bisnis sebelum sistem terpusat

Proses yang diusulkan menempatkan NagariSDLC sebagai pusat koordinasi. Business User mengajukan kebutuhan, pihak perencanaan melakukan review dan analisis, pengembangan mengalokasikan tim serta task, pengujian

4.1.2 Aktor dan Lingkup Sistem

Aktor	Tanggung Jawab
Business User	Mengajukan kebutuhan, memantau proyek, dan menjadi pihak peminta pada UAT.
Lead Group / Analyst	Review dan analisis kebutuhan pada fase perencanaan.
Development Lead / PM	Disposisi pengembangan, alokasi tim, task, SIT/UAT, dan pengajuan rilis.
Developer	Mengerjakan task, revisi, bukti SIT, dan approval yang ditugaskan.
QA Lead / QA Tester	Disposisi, pelaksanaan, laporan, dan sign-off QA.

Aktor	Tanggung Jawab
Cyber Lead / Pentester	Disposisi, pelaksanaan, laporan, dan sign-off keamanan siber.
Head of IT	Mengambil keputusan quality gate sebelum live production.
Super Admin	Mengelola pengguna, divisi, grup, role, dan akses administrasi.

4.1.3 Kebutuhan Fungsional

Tabel 4.2 Kebutuhan fungsional

ID	Kebutuhan	Deskripsi
F-01	Autentikasi dan sesi	Login, logout, refresh, profil, ganti/lupa/reset kata sandi.
F-02	Pengajuan proyek	Mencatat kebutuhan, pemohon, divisi, prioritas, RBB/Non-RBB, dan target.
F-03	Workflow proyek	Memvalidasi transisi status, peran, penugasan, prasyarat, histori, dan notifikasi.
F-04	Manajemen task	CRUD task, assignee, status, prioritas, revisi, dan task perbaikan.
F-05	SIT	Persiapan, eksekusi per task, bukti, revisi, review, dan sign-off.
F-06	UAT	Skenario, peserta, undangan, draft/final eksekusi, Change Request, dan approval.
F-07	QA	Pengajuan, disposisi, laporan, bukti, sign-off, dan pengembalian.
F-08	Keamanan siber	Pentest/secure code, disposisi, laporan, bukti, sign-off, dan pengembalian.
F-09	Rilis	Pengajuan target rilis, downtime, rollback, quality gate, dan produksi.
F-10	Dokumen	Upload, masking nama, daftar, unduh, dan penghapusan yang melindungi bukti.
F-11	Komunikasi	Chat proyek, notifikasi, dan informasi aktivitas.
F-12	Administrasi	Pengguna, divisi, grup, role, dan pembatasan menu.

4.1.4 Kebutuhan Nonfungsional

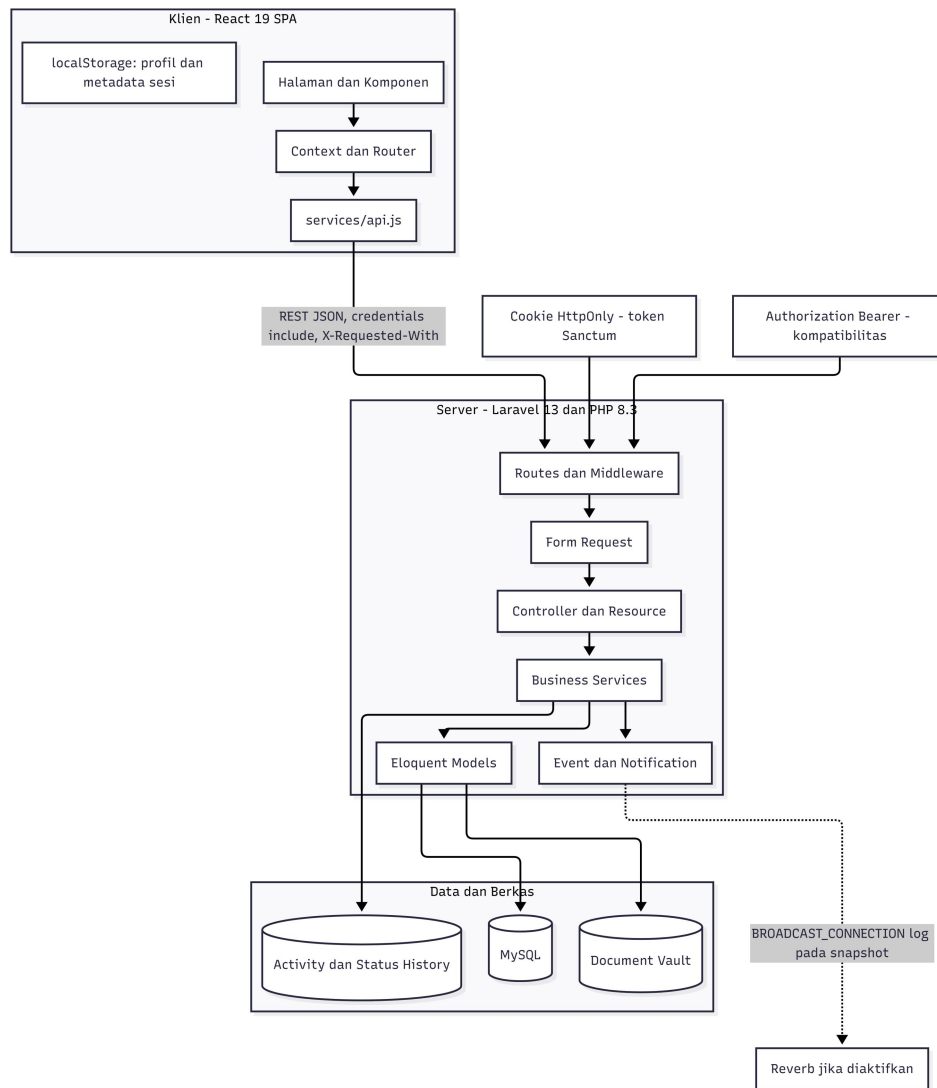
Tabel 4.3 Kebutuhan nonfungsional

Aspek	Kebutuhan
Keamanan	<i>Cookie HttpOnly</i> untuk sesi SPA, RBAC, <i>project scope</i> , validasi input, CORS berbasis environment, serta proteksi audit trail.
Keterlacakan	Histori status, activity log, test report, approval round, return round, dan document evidence.
Konsistensi	Envelope API seragam dan satu service API pada front end.
Reliabilitas	Transaksi database pada operasi workflow dan pengujian yang mengubah beberapa tabel.
Kompatibilitas	Dukungan Bearer token dan pembacaan data legacy tanpa menghapus histori.
<i>Usability</i>	Antarmuka Bahasa Indonesia, responsif, pencarian, filter, wizard, dan status yang mudah dipahami.
<i>Maintainability</i>	Pemisahan route, Form Request, controller, Resource, model, dan service.
<i>Deployability</i>	Template environment produksi tersedia; keputusan infrastruktur final masih perlu ditetapkan.

4.1.5 Arsitektur Sistem

Lapis klien berupa React 19 SPA yang dibangun dengan Vite 8 dan Tailwind CSS 4. Lapis aplikasi berupa Laravel 13 pada PHP 8.3 dengan REST API, Sanctum, *Form Request*, *Resource*, *service*, *event*, dan *notification*. Lapis data menggunakan MySQL melalui Eloquent ORM serta penyimpanan dokumen pada disk yang dikonfigurasi. Front end memanggil API melalui frontend/src/services/api.js. Respons mengikuti format { *status*, *message*, *data* }.

Autentikasi aktif memakai cookie HttpOnly nagari_sdmc_token sebagai jalur utama SPA dengan credentials *include*. Header *X-Requested-With* diwajibkan pada jalur cookie. Header *Authorization Bearer* tetap dipertahankan untuk klien lama dan pengujian. *localStorage* hanya memuat metadata sesi dan profil yang diperlukan antarmuka, bukan token. *Broadcasting Reverb* tersedia dalam rancangan, tetapi template environment saat ini memakai *BROADCAST_CONNECTION=log* sehingga pengaktifan *realtime* produksi memerlukan keputusan dan konfigurasi operasional.



Gambar 4.3 Arsitektur sistem

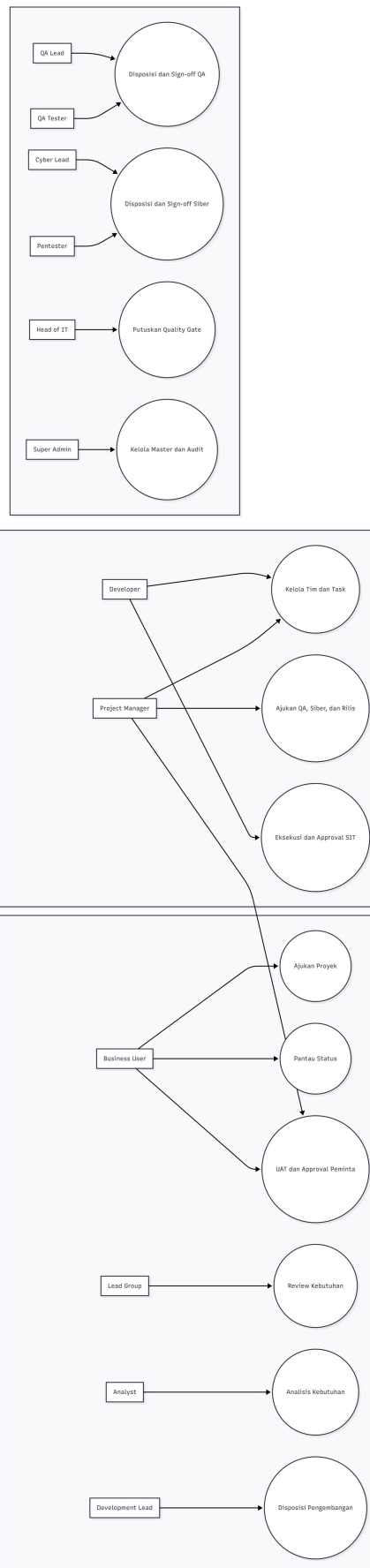
4.1.6 Model Peran dan Use Case

Tabel 4.4 Daftar 12 peran pengguna

Role	Nama Tampilan	Grup Bawaan	Tanggung Jawab
super_admin	Super Admin	Manajemen TI	Administrasi dan akses global.
head_of_it	Head of IT	Manajemen TI	Quality gate dan pengawasan.
lead_group	Lead Group	Perencanaan-QA	Review dan pengawasan grup.
analyst	Analyst	Perencanaan-QA	Analisis kebutuhan Fase 1.
development_	Development	Pengembangan	Disposisi dan pengawasan

Role	Nama Tampilan	Grup Bawaan	Tanggung Jawab
lead	Lead		pengembangan.
project_manager	Project Manager/Dev Analyst	Pengembangan	Pengelolaan proyek Fase 2.
developer	Developer	Pengembangan	Implementasi dan perbaikan.
qa_lead	QA Lead	Perencanaan-QA	Disposisi dan sign-off QA.
qa_tester	QA Tester	Perencanaan-QA	Pelaksanaan pengujian QA.
cyber_lead	Cyber Lead	Keamanan Siber	Disposisi dan sign-off Siber.
pentester	Pentester	Keamanan Siber	Pentest atau secure code review.
business_user	Business User	Pemohon	Pengajuan dan penerimaan UAT.

Lima grup bawaan—Perencanaan-QA, Pengembangan, Keamanan Siber, Manajemen TI, dan Pemohon—mengelompokkan role untuk tampilan. Grup tidak memberi wewenang. Hak transisi tetap berada di ProjectWorkflowService, cakupan proyek di ProjectAccessService, dan hak pelaksana pengujian di TestingTrack. Role baru yang dibuat melalui administrasi belum otomatis berfungsi karena daftar otorisasi tetap berada di kode.



Gambar 4.4 Use case aktor utama

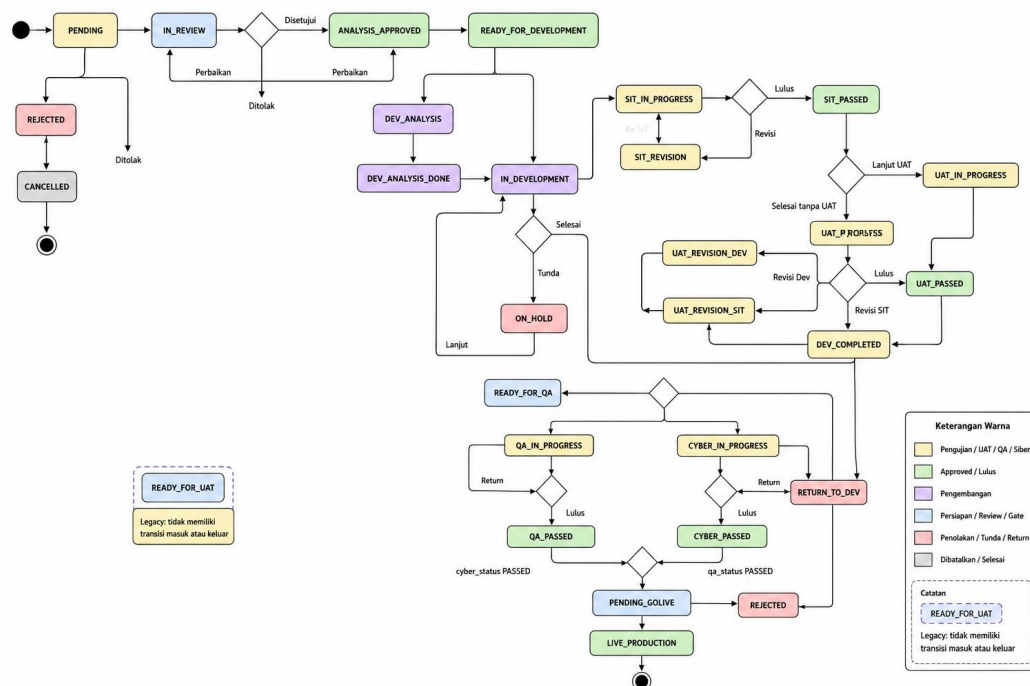
4.1.7 Mesin Status Proyek

Tabel 4.5 Daftar 27 status proyek

No.	Status	Fase/Makna
1	PENDING	Inisiasi
2	IN_REVIEW	Review
3	ANALYSIS_APPROVED	Analisis
4	REJECTED	Nonlinear
5	READY_FOR_DEVELOPMENT	Pengembangan
6	DEV_ANALYSIS	Pengembangan
7	DEV_ANALYSIS_DONE	Pengembangan
8	IN_DEVELOPMENT	Pengembangan
9	SIT_IN_PROGRESS	SIT
10	SIT_PASSED	SIT
11	SIT_REVISION	SIT
12	UAT_IN_PROGRESS	UAT
13	UAT_REVISION_SIT	Legacy/revisi
14	UAT_REVISION_DEV	UAT/revisi
15	DEV_COMPLETED	Pasca-UAT
16	READY_FOR_QA	QA/Siber
17	QA_IN_PROGRESS	QA
18	RETURN_TO_DEV	Pengembalian
19	QA_PASSED	QA
20	CYBER_IN_PROGRESS	Siber
21	CYBER_PASSED	Siber
22	READY_FOR_UAT	Legacy
23	UAT_PASSED	Legacy/opsional
24	PENDING_GOLIVE	Rilis
25	LIVE_PRODUCTION	Produksi

No.	Status	Fase/Makna
26	ON_HOLD	Nonlinear
27	CANCELLED	Terminal

READY_FOR_UAT dipertahankan sebagai status legacy agar histori lama tetap terbaca, tetapi tidak memiliki transisi masuk maupun keluar. LIVE_PRODUCTION dan CANCELLED bersifat terminal pada matriks aktif. Status revisi dapat membawa proyek kembali ke pengembangan; revisi tidak direpresentasikan sebagai pembatalan.

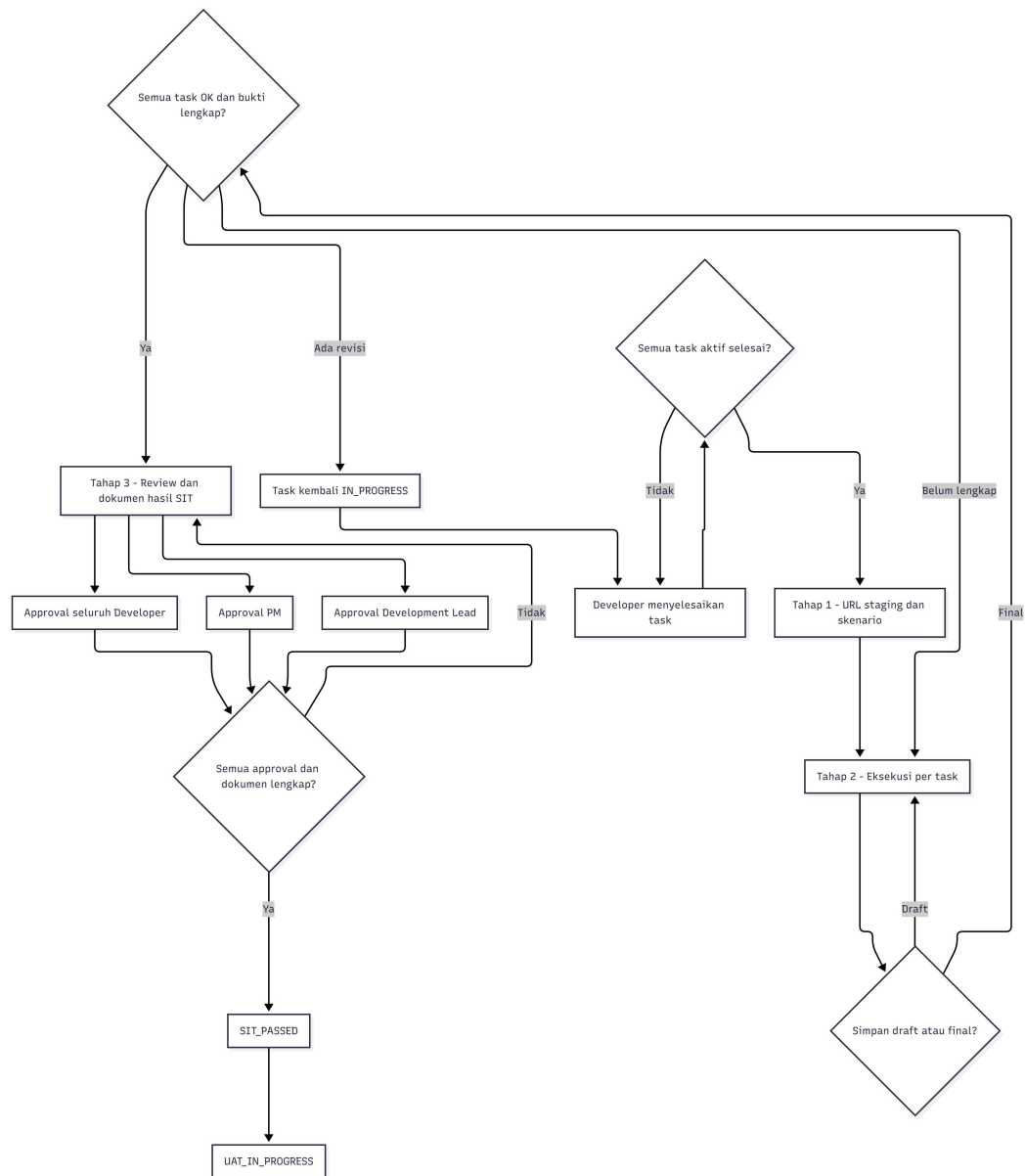


Gambar 4.5 Mesin status proyek

4.1.8 Alur System Integration Test

SIT dimulai setelah seluruh task aktif selesai; task TAKE DOWN tidak dihitung. Tahap 1 menyimpan URL staging dan menyiapkan skenario. Tahap 2 menampilkan setiap task, status OK, komentar, bukti, dan revisi. Draft dapat disimpan tanpa berpindah tahap. Tahap 3 memuat ringkasan, catatan review, dokumen hasil review/berita acara, serta approval seluruh developer tim, PM, dan Development Lead.

Jika UAT menemukan revisi mayor, SIT diulang secara menyeluruh pada seluruh task aktif kecuali TAKE DOWN. Perbedaannya bukan pada daftar task, melainkan pada ketatnya bukti: task harus selesai, di-OK, memiliki lampiran yang sah, dan approval tim harus lengkap. Bukti per task dibekukan setelah SIT lulus agar berita acara tidak berubah setelah keputusan.



Gambar 4.6 Alur SIT

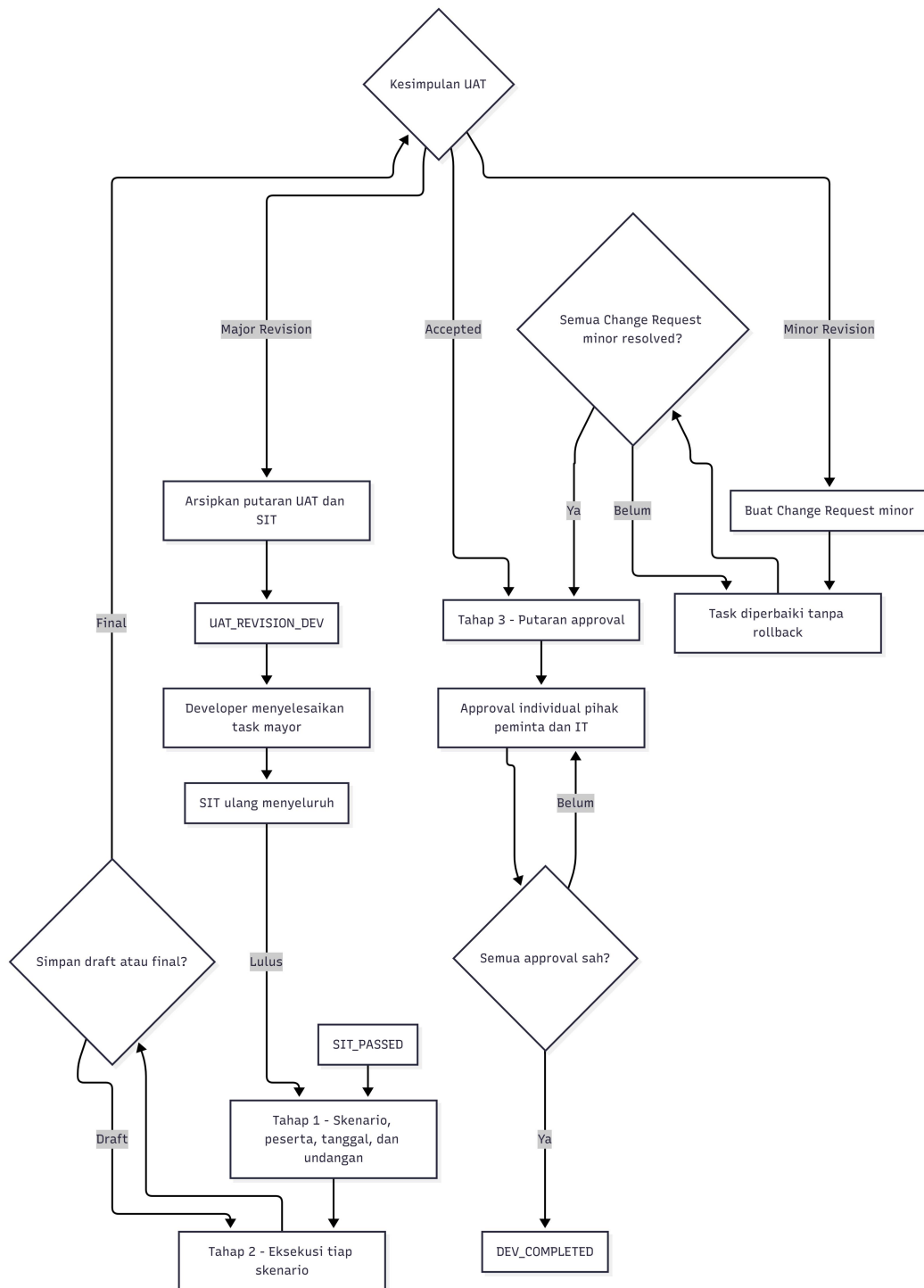
4.1.9 Alur User Acceptance Test

Tahap 1 UAT memuat skenario dari task, unit peminta, tanggal pelaksanaan, penyiap, peserta, dan undangan. Peserta menjadi sumber kandidat approver. Roster uat1_participants tidak pernah dikosongkan oleh jalur kode; PM boleh menambah atau memperbaiki, tetapi pengosongan ditolak karena penanda tangan harus dibawa ke setiap putaran revisi.

Tahap 2 mencatat hasil setiap skenario sebagai accepted atau revision. Revisi minor membuat Change Request dan membuka kembali task tanpa rollback atau SIT ulang, tetapi menahan approval sampai seluruh perbaikan resolved. Revisi mayor mengarsipkan putaran UAT, memindahkan proyek ke UAT_REVISION_DEV, membuka task perbaikan, menjalankan SIT ulang

menyeluruh, lalu mengulang UAT dari Tahap 1. Tahap 3 membuka putaran approval baru atas hasil terbaru.

Kompatibilitas penanda pengulangan UAT pada data legacy dibaca hanya melalui `Project::isUatRestartPending()`, bukan dengan mengakses key JSON yang telah dipensiunkan secara langsung.



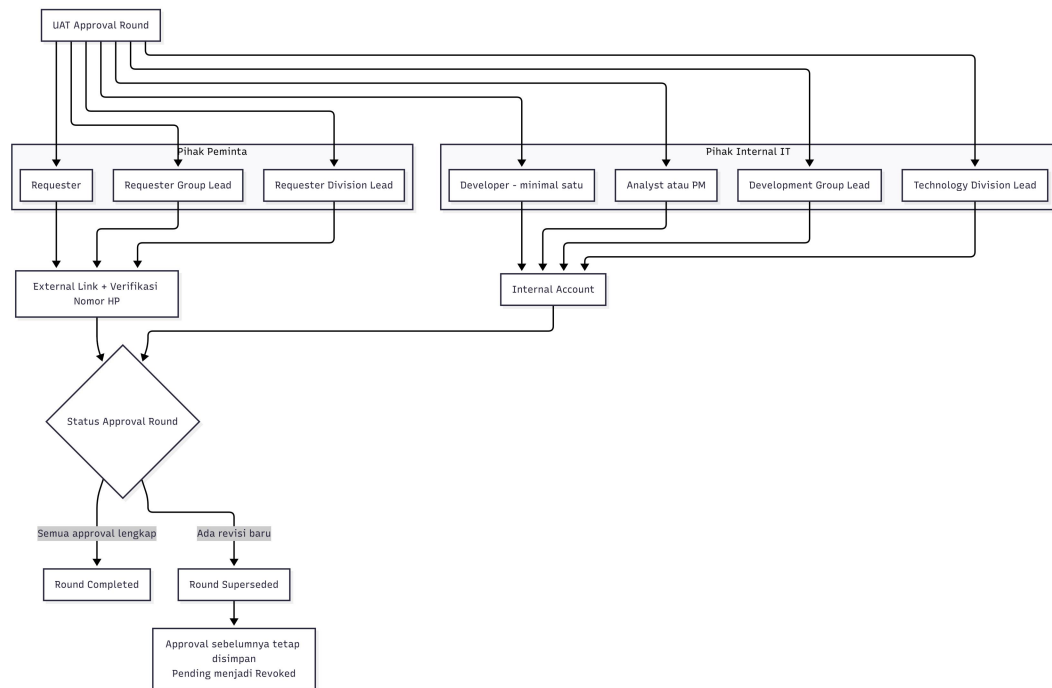
Gambar 4.7 Alur UAT

4.1.10 Matriks Persetujuan UAT

Tabel 4.6 Matriks approver UAT

Approval Role	Sisi	Mode	Kewajiban
requester	Pihak peminta	External link	Ya, satu
requester_group_lead	Pihak peminta	External link	Ya, satu
requester_division_lead	Pihak peminta	External link	Ya, satu
developer	Pihak IT	Internal account	Minimal satu
analyst_pm	Pihak IT	Internal account	Ya, satu
development_group_lead	Pihak IT	Internal account	Ya, satu
technology_division_lead	Pihak IT	Internal account	Ya, satu

Approval aktif berada pada `uat_approval_rounds` dan `uat_approvers`. Pihak peminta memakai link pribadi dan verifikasi nomor HP, sedangkan pihak IT memakai akun. Setiap orang memberikan keputusan secara individual dan dapat paralel. Putaran yang tidak berlaku lagi ditandai `superseded`; token dicabut, baris `pending` menjadi `revoked`, dan baris `approved` tetap disimpan sebagai audit trail.

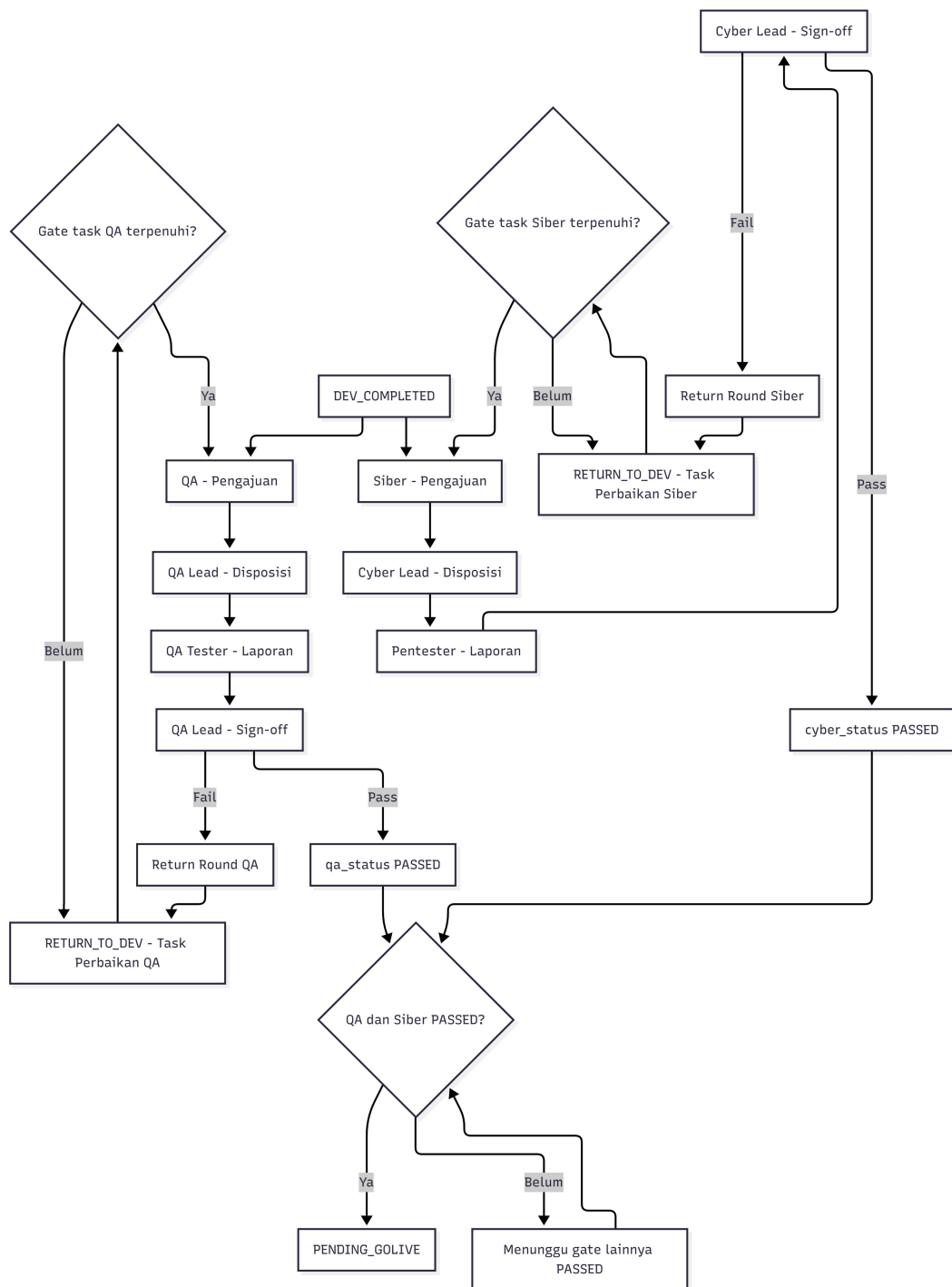


Gambar 4.8 Relasi approver UAT

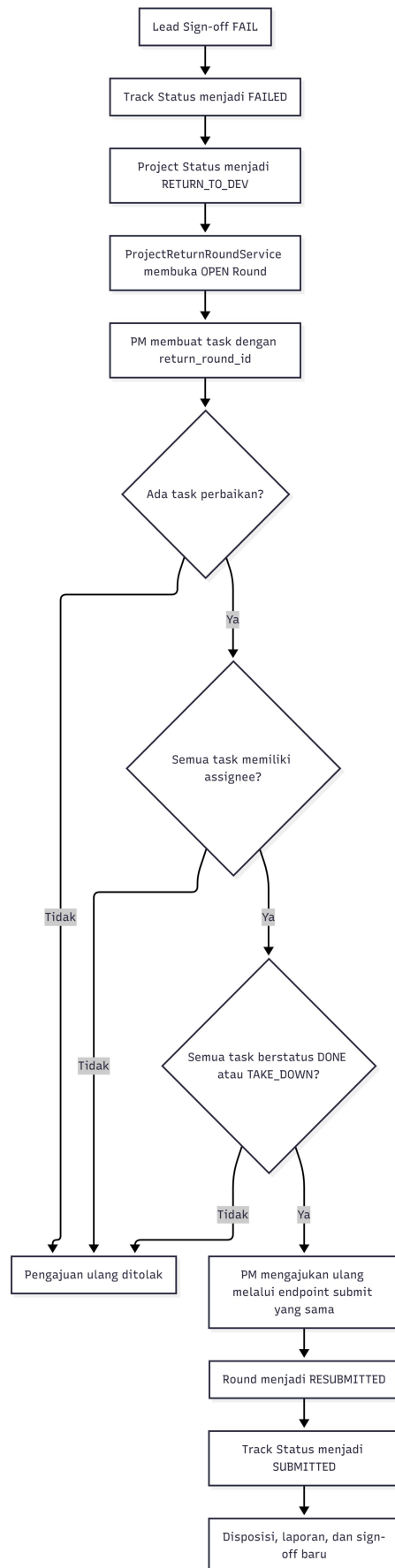
4.1.11 Jalur QA dan Keamanan Siber

Setelah DEV_COMPLETED, PM mengajukan proyek ke QA dan keamanan siber. Masing-masing jalur memiliki kolom status sendiri: qa_status dan cyber_status. Empat langkah pada setiap jalur adalah pengajuan, disposisi oleh Lead, laporan pelaksana, dan sign-off Lead. Cakupan pengujian ditulis pada test_reports.tested_scenarios sebagai teks bebas; checklist tetap hanya dipertahankan untuk membaca laporan lama.

Sign-off fail menandai jalur FAILED, memindahkan proyek ke RETURN_TO_DEV, dan membuka project_return_rounds. PM harus membuat task perbaikan yang menunjuk return round. Pengajuan ulang ditolak bila belum ada task, masih ada task tanpa assignee, atau ada task yang belum done/take_down. Penilaian dilakukan per jalur sehingga pengembalian Siber tidak otomatis menahan QA yang belum berjalan, dan sebaliknya.



Gambar 4.9 Jalur QA dan keamanan siber



Gambar 4.10 Putaran pengembalian pengujian

4.1.12 Gerbang Rilis

PENDING_GOLIVE hanya dapat dicapai bila qa_status dan cyber_status sama-sama PASSED. PM mengajukan release request yang memuat target tanggal, estimasi downtime, rollback plan, dan catatan. Head of IT melihat antrean quality gate, lalu menyetujui menuju LIVE_PRODUCTION atau menolak dengan alasan yang tercatat. Tidak terdapat UAT final setelah kedua jalur pengujian lulus.

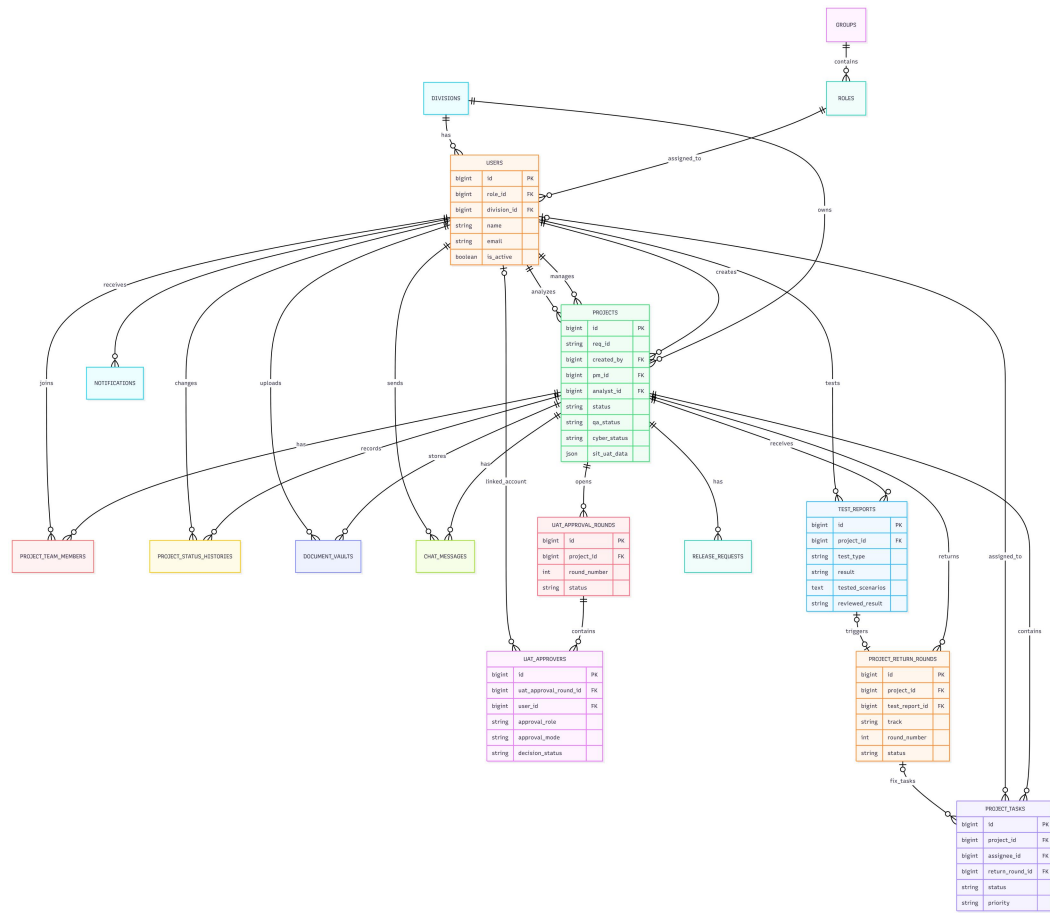
4.1.13 Model Data dan Relationship

Tabel 4.7 Domain utama

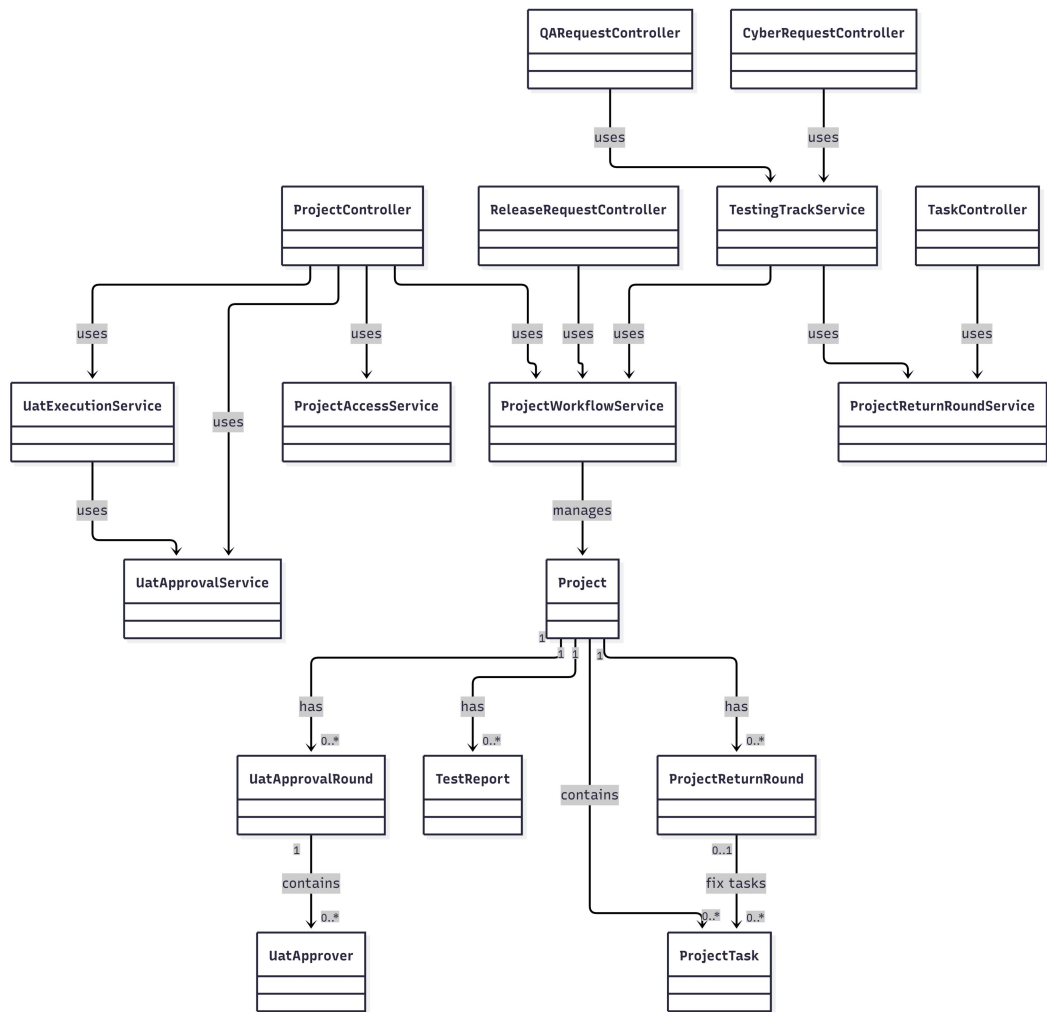
Tabel	Fungsi
users	Akun, role, divisi, nomor HP, status aktif, soft delete
roles	Nama role, nama tampilan, grup, dan pembatasan menu
groups	Pengelompokan role untuk tampilan
divisions	Master unit/divisi pengguna dan proyek
projects	Entitas pusat proyek, status, penugasan, jalur uji, dan sit_uat_data
project_tasks	Task, assignee, status, revisi, dan return_round_id
project_team_members	Keanggotaan tim proyek
project_status_histories	Riwayat transisi status
project_return_rounds	Putaran pengembalian QA/Siber
test_reports	Laporan pengujian dan sign-off Lead
uat_approval_rounds	Putaran persetujuan final UAT
uat_approvers	Approver individual, mode akses, dan keputusan
release_requests	Pengajuan, rencana, dan keputusan rilis
document_vaults	Metadata dan lokasi dokumen
chat_messages	Pesan diskusi proyek
activity_logs	Aktivitas pengguna dan metadata audit
notifications	Kotak masuk pemberitahuan pengguna

Selain 17 tabel domain, terdapat delapan tabel framework: sessions, cache, cache_locks, jobs, job_batches, failed_jobs, password_reset_tokens, dan personal_access_tokens. Dengan demikian model data mencakup 25 tabel. projects.sit_uat_data menyimpan data wizard yang berstruktur JSON, sedangkan

approval aktif UAT dipindahkan ke tabel terstruktur agar snapshot per putaran dan keputusan per orang dapat diaudit.



Gambar 4.11 ERD NagariSDLC



Gambar 4.12 Class relationship inti

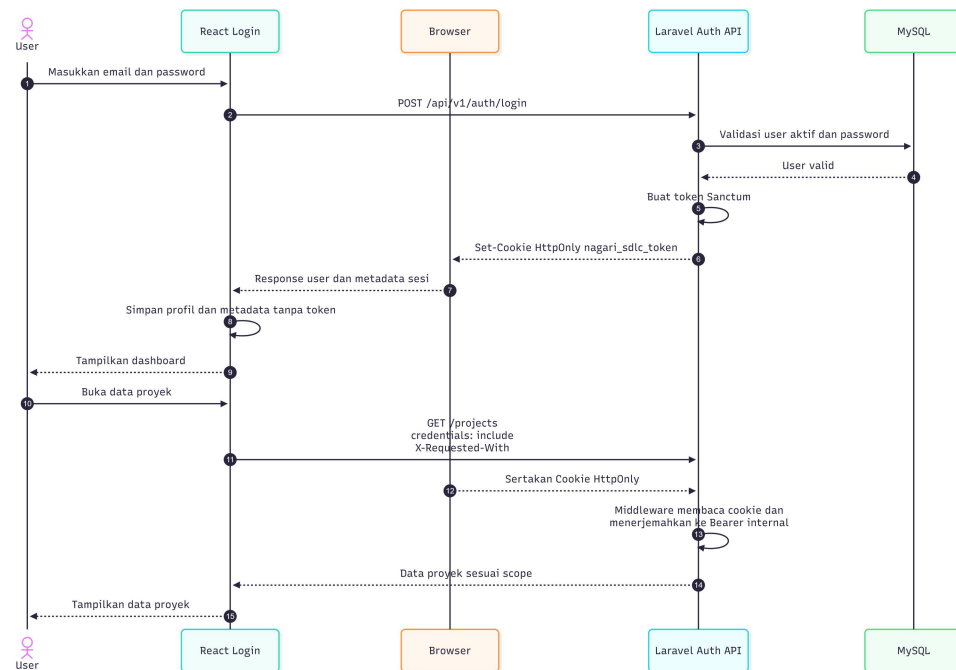
4.1.14 Desain API dan Sequence

Tabel 4.8 Endpoint API representatif

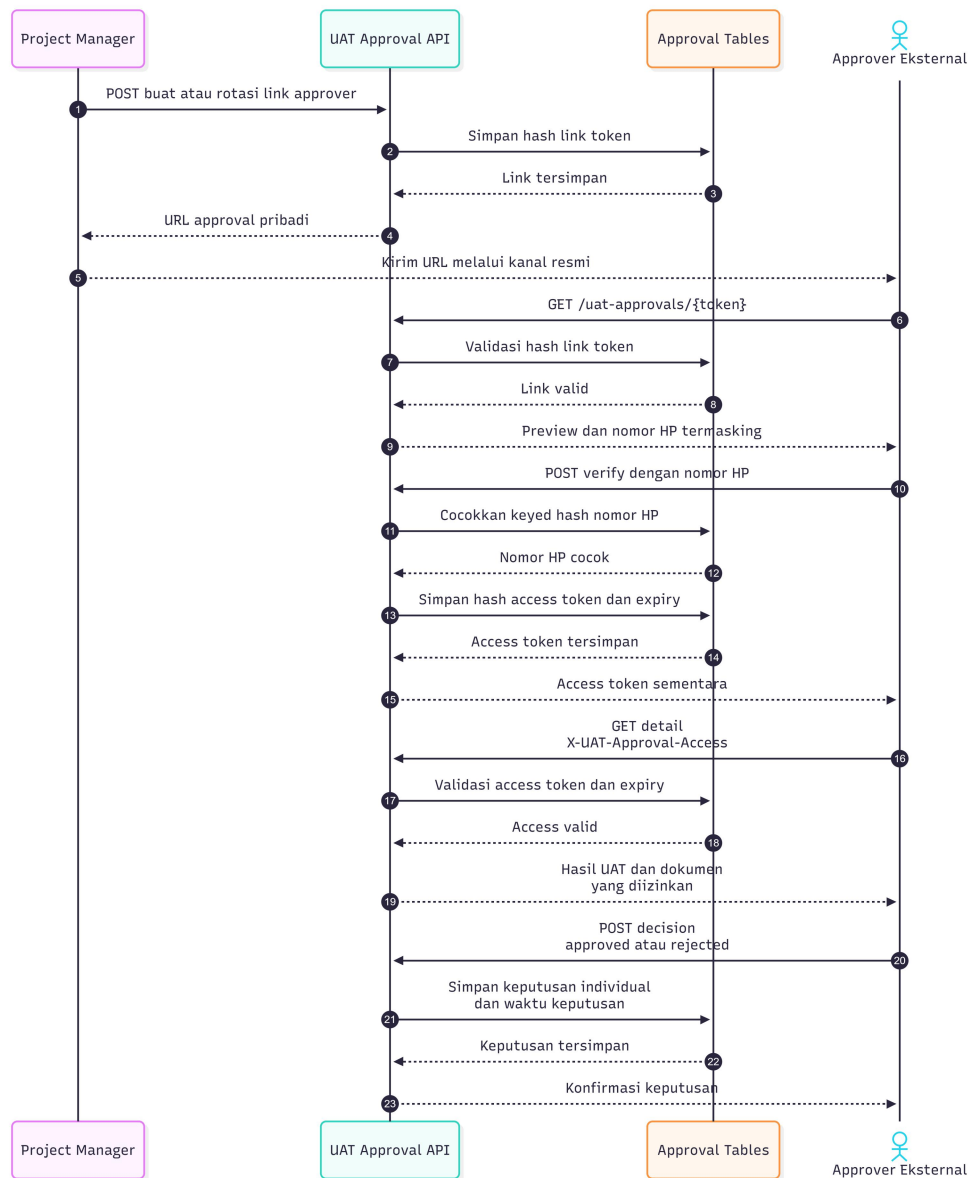
Method	Endpoint	Fungsi
POST	/auth/login	Login dan penerbitan sesi Sanctum
GET/POST	/projects	Daftar dan pengajuan proyek
PATCH	/projects/{id}/status	Transisi melalui workflow
GET/POST	/projects/{id}/tasks	Daftar dan pembuatan task
POST	/projects/{id}/sit-approval	Approval SIT
PUT/POST	/projects/{id}/uat-execution[/draft]	Draft/final eksekusi UAT

Method	Endpoint	Fungsi
POST	/qa-requests/*	Empat langkah jalur QA
POST	/cyber-requests/*	Empat langkah jalur Siber
POST	/release-requests	Pengajuan migrasi dan rilis
POST	/quality-gate/approve reject	Keputusan Head of IT

Seluruh error API menggunakan envelope seragam. Kegagalan validasi menambahkan errors per field; 401 menandai sesi tidak sah, 403 menandai larangan otorisasi, 404 untuk data tidak ditemukan, 422 untuk validasi atau aturan workflow, dan 500 untuk kegagalan internal. Detail implementasi endpoint ditempatkan pada Lampiran C.



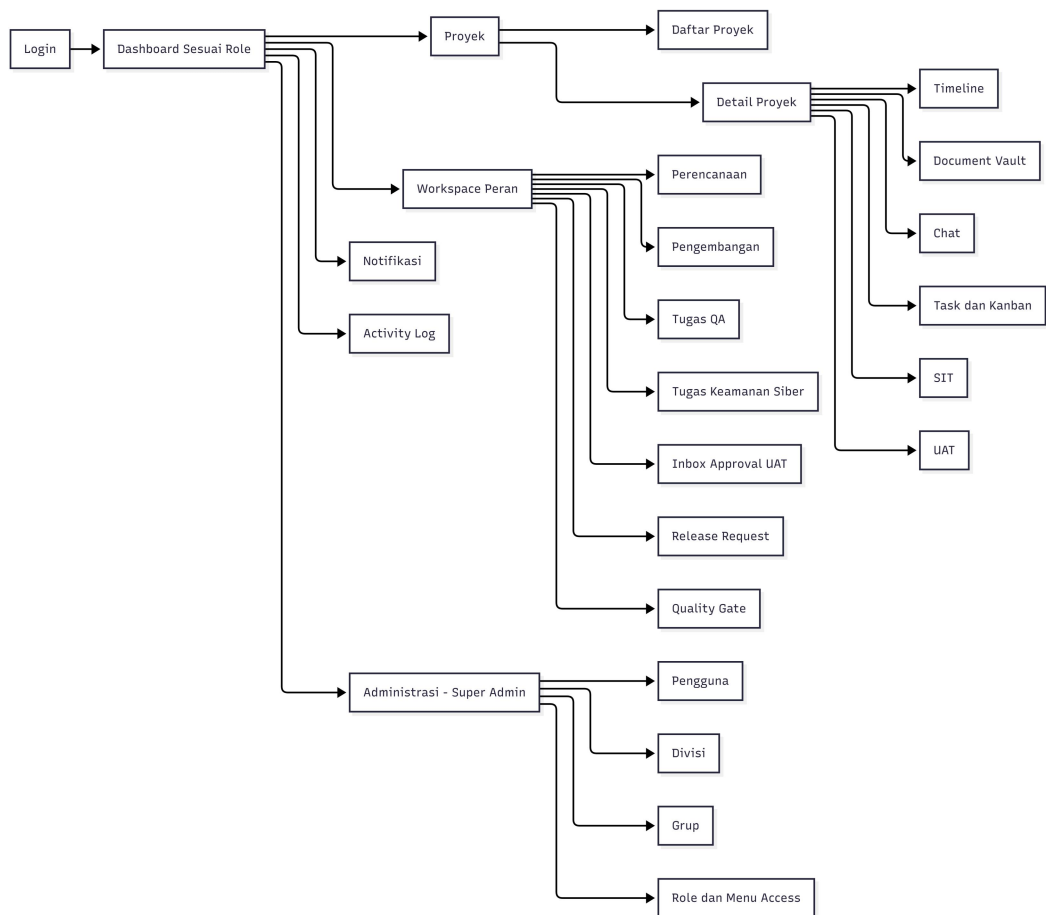
Gambar 4.13 Sequence login



Gambar 4.14 Sequence persetujuan UAT eksternal

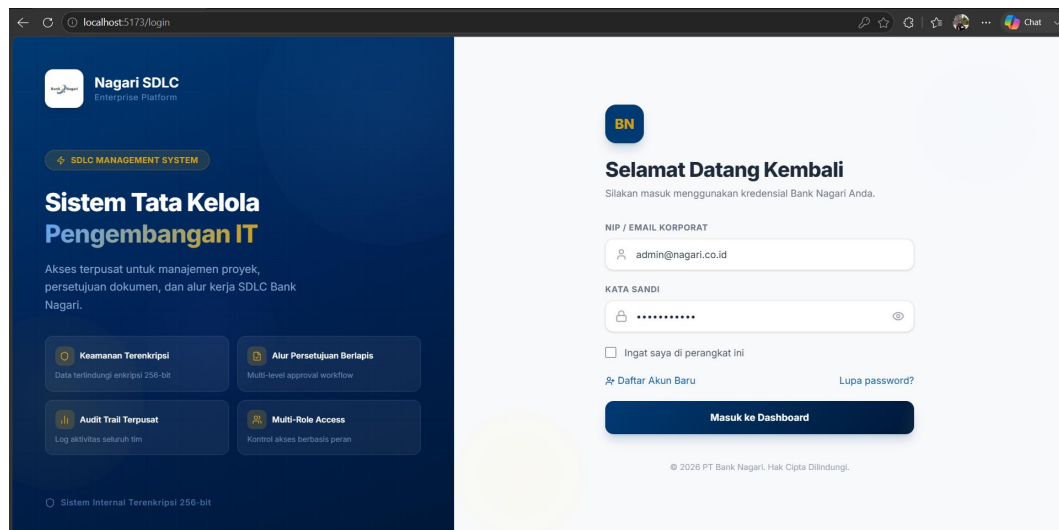
4.1.15 Perancangan Navigasi dan Antarmuka

Navigasi ditentukan oleh role guard dan menuConfig. Pembatasan roles.menu_access hanya dapat mengurangi menu yang telah tersedia; menyembunyikan menu tidak menggantikan otorisasi route maupun backend. Halaman dikelompokkan menjadi dashboard, proyek, workspace per peran, task, SIT/UAT, QA/Siber, approval, release, quality gate, notifikasi, activity log, dan administrasi.



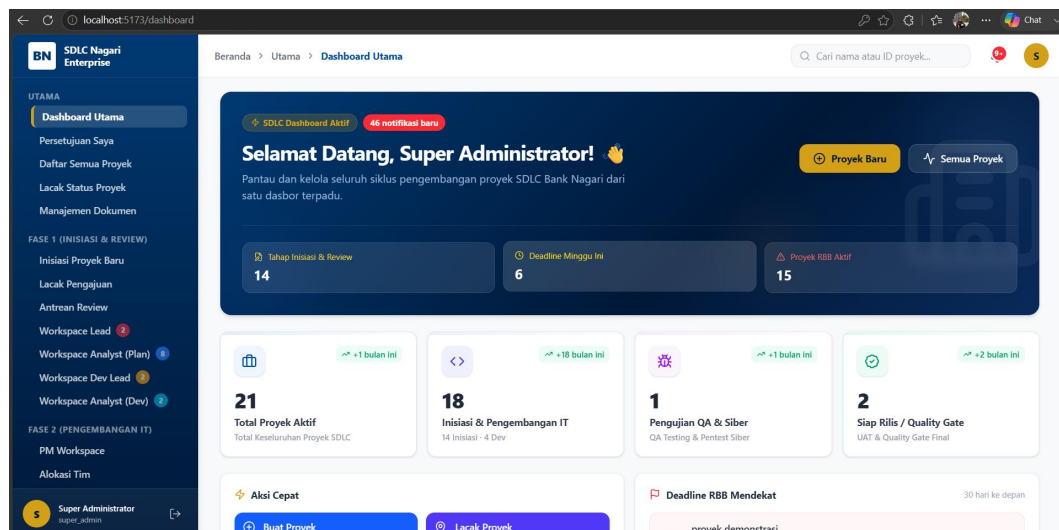
Gambar 4.15 Peta navigasi halaman

4.1.15.1 Antarmuka login



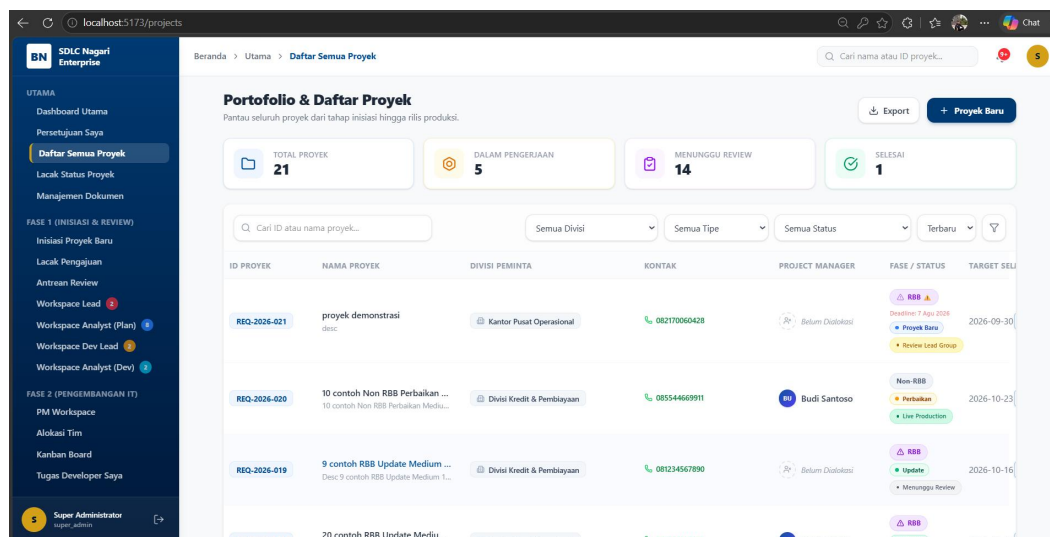
Gambar 4.16 Antarmuka login

4.1.15.2 Antarmuka dashboard



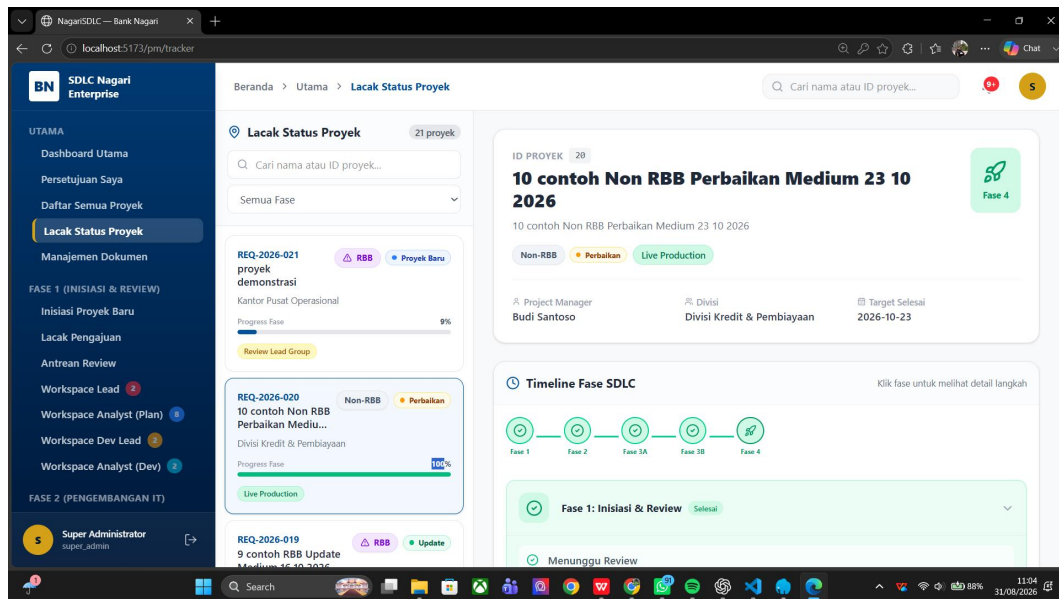
Gambar 4.17 Antarmuka dashboard Admin

4.1.15.3 Daftar proyek



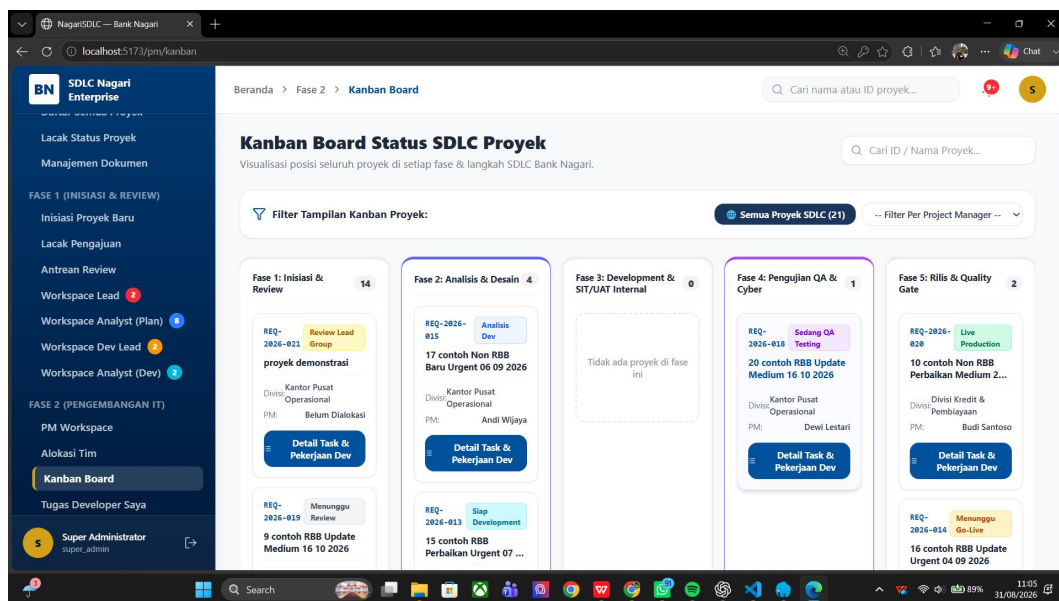
Gambar 4.18 Daftar proyek

4.1.15.4 Detail proyek



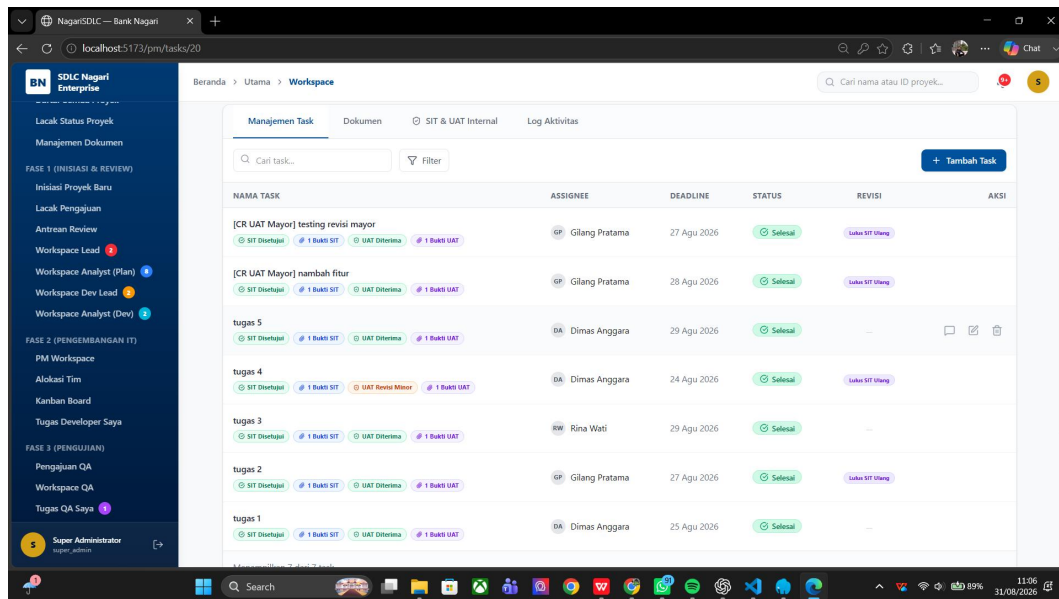
Gambar 4.19 Detail proyek

4.1.15.5 Papan Kanban



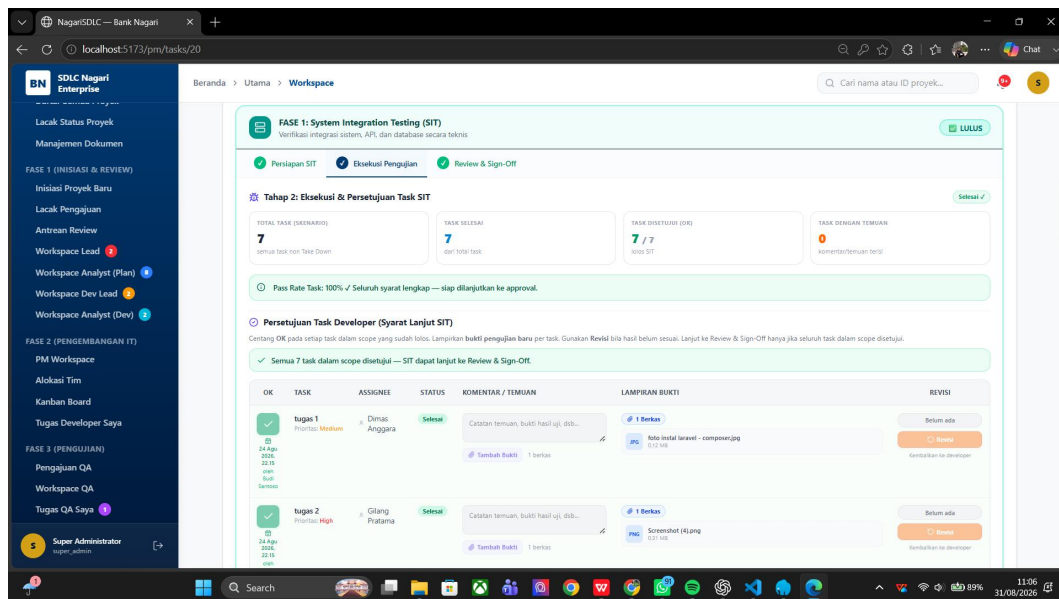
Gambar 4.20 Papan Kanban

4.1.15.6 Detail task



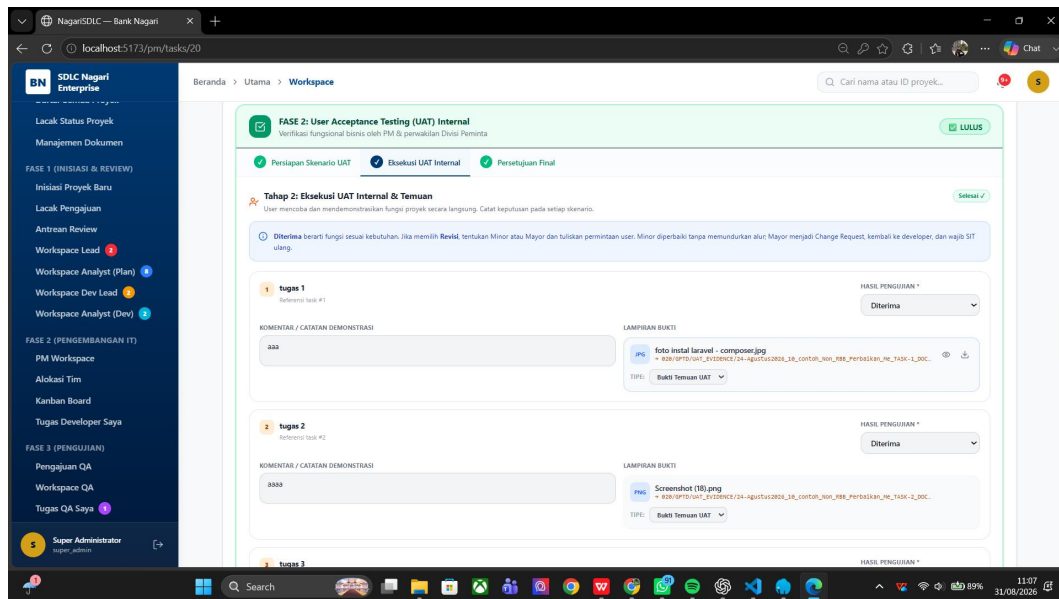
Gambar 4.21 Detail task

4.1.15.7 Wizard SIT



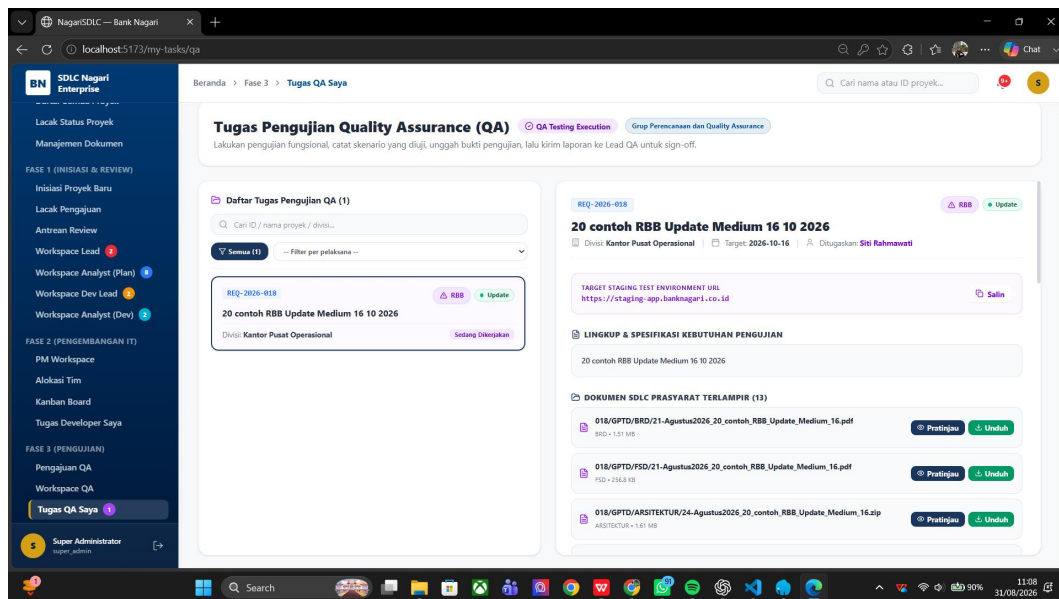
Gambar 4.22 Wizard SIT

4.1.15.8 Wizard UAT



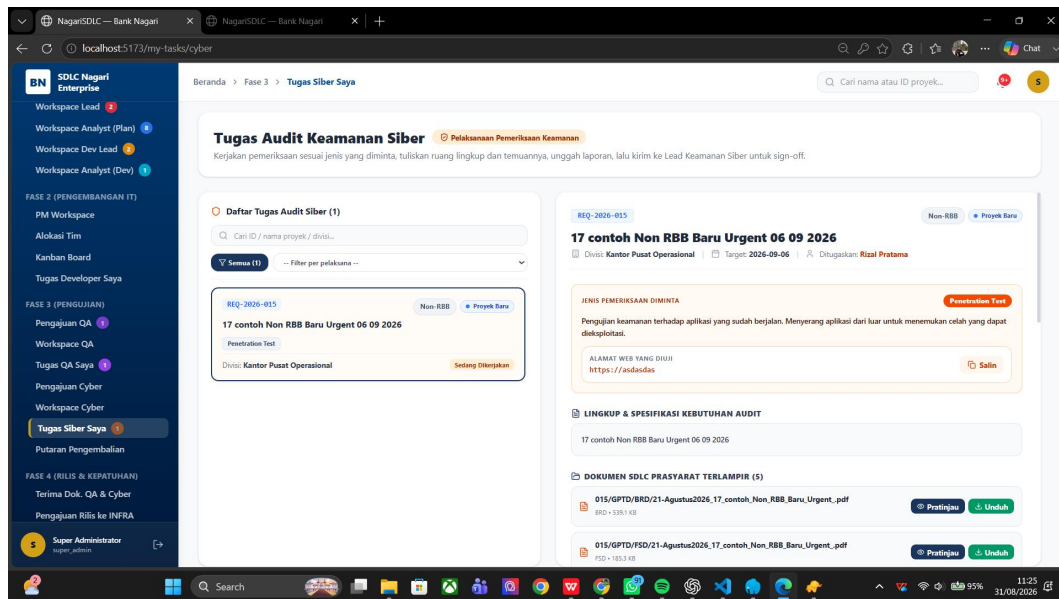
Gambar 4.23 Wizard UAT

4.1.15.9 Tugas QA



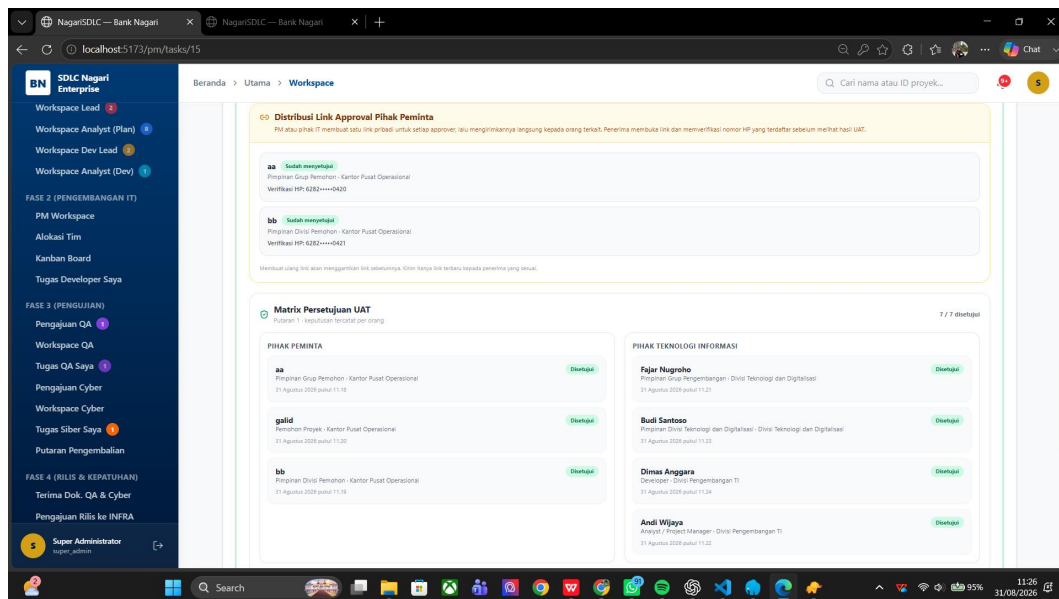
Gambar 4.24 Tugas QA

4.1.15.10 Tugas keamanan siber



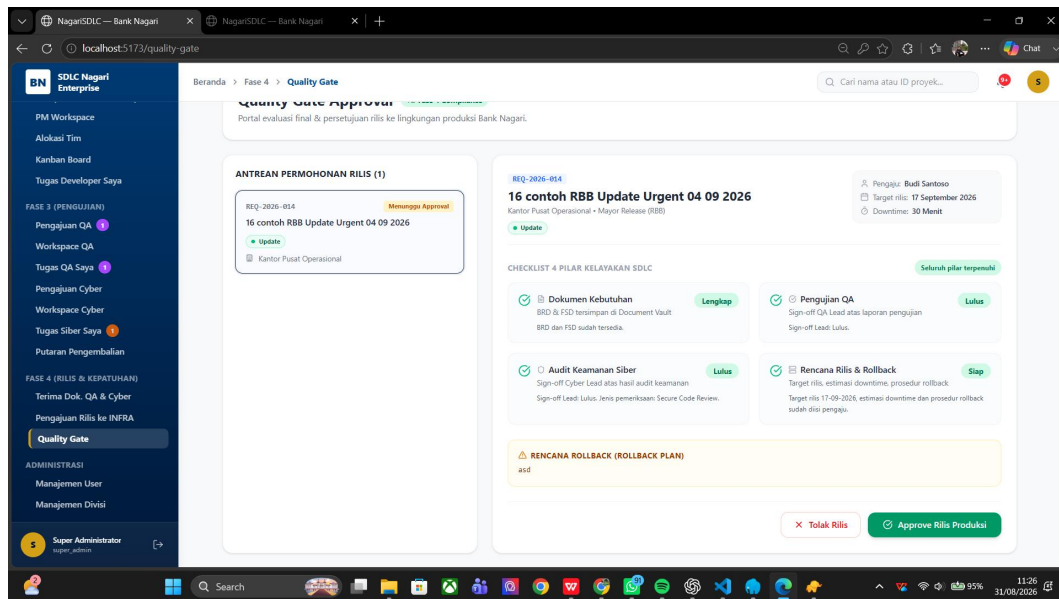
Gambar 4.25 Tugas keamanan siber

4.1.15.11 Matriks persetujuan UAT



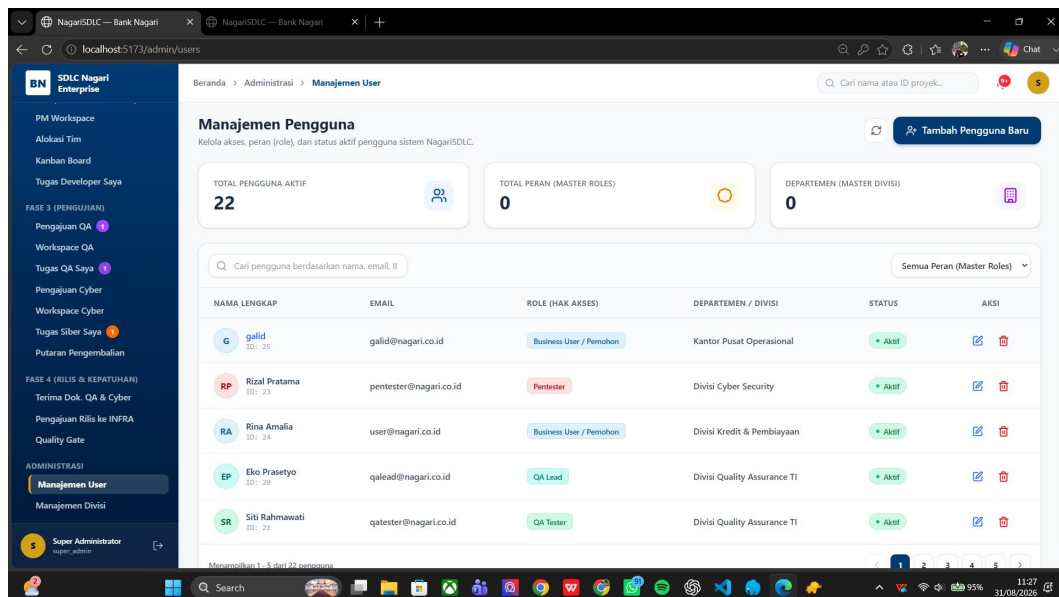
Gambar 4.26 Matriks persetujuan UAT

4.1.15.12 Quality Gate



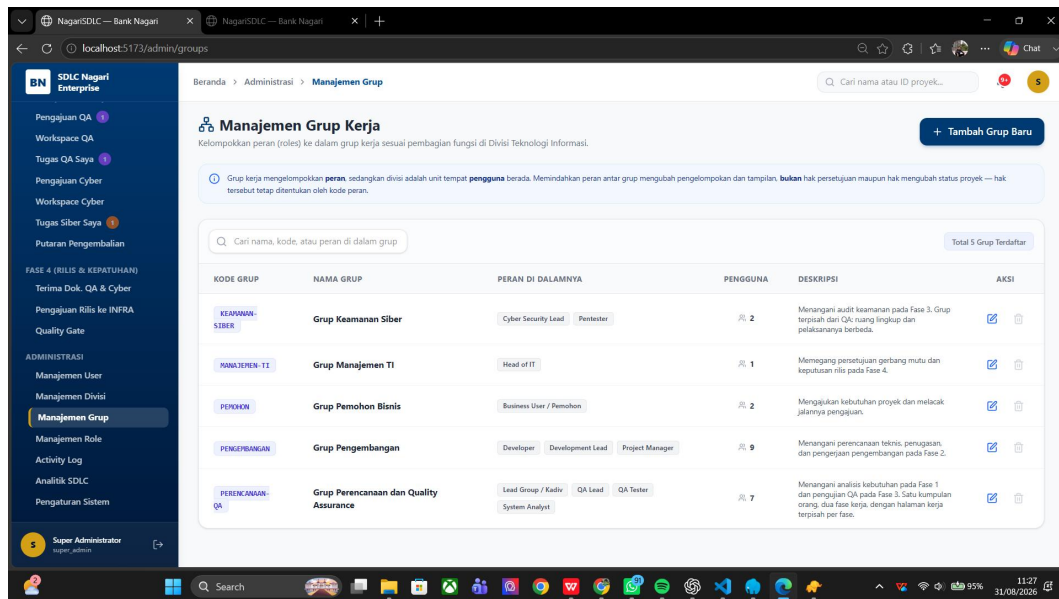
Gambar 4.27 Quality Gate

4.1.15.13 Manajemen pengguna

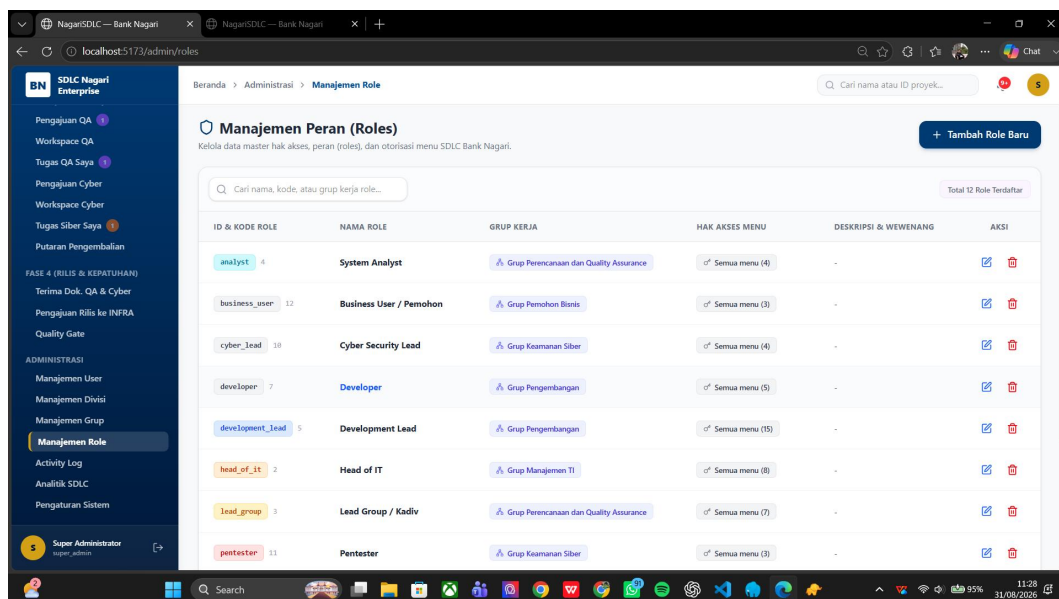


Gambar 4.28 Manajemen pengguna

4.1.15.14 Manajemen grup dan role

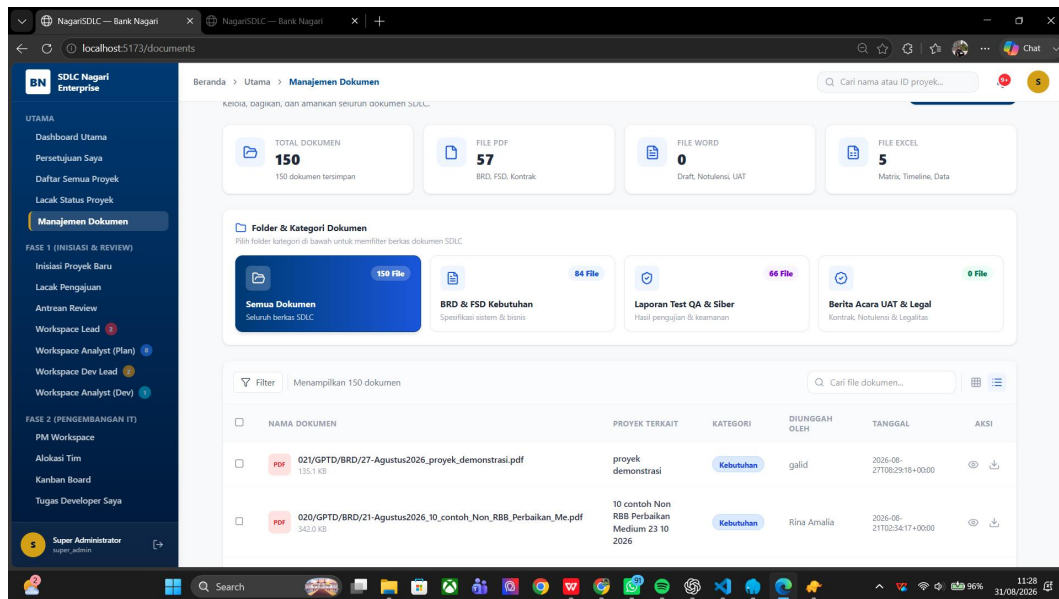


Gambar 4.29 Manajemen grup



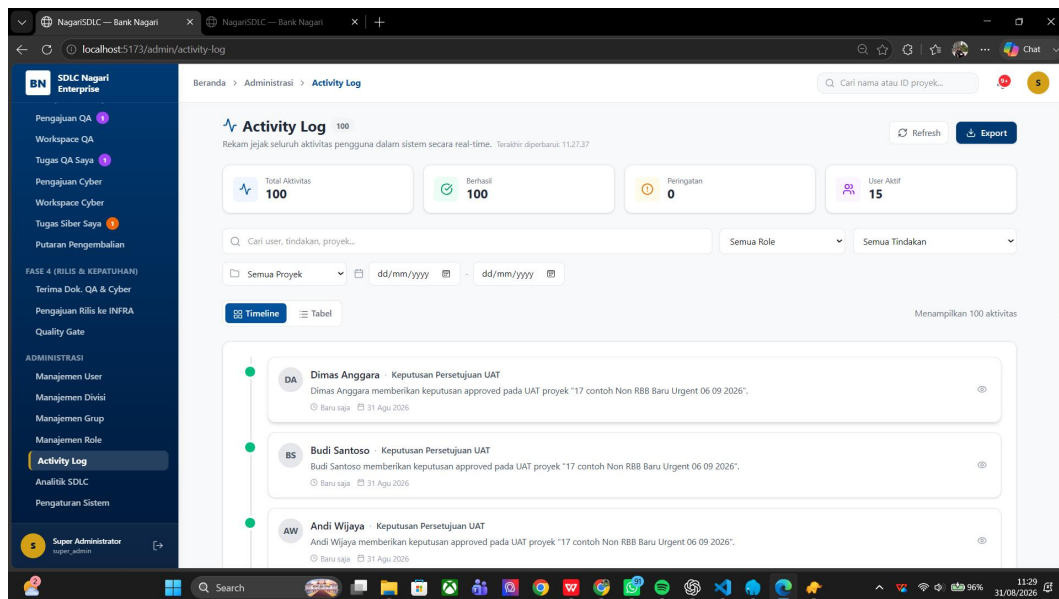
Gambar 4.30 Manajemen role

4.1.15.15 Document Vault



Gambar 4.31 Document Vault

4.1.15.16 Activity Log



Gambar 4.32 Activity Log

4.1.16 Implementasi Back End

Back end memakai pola route–Form Request–controller–service–model–Resource. Validasi write dilakukan pada Form Request. Logika bisnis lintas endpoint ditempatkan pada service. ProjectWorkflowService::transition() menjadi satu-satunya jalur transisi status proyek. TestingTrackService mengelola empat langkah QA/Siber, UatExecutionService mengelola draft/final UAT dan hold

revisi, UatApprovalService mengelola approval round, sedangkan ProjectReturnRoundService mengelola putaran pengembalian.

Operasi yang mengubah beberapa data dibungkus transaksi. Status proyek, kolom jalur, histori, notifikasi, approval, dan return round tidak boleh berada pada keadaan setengah jadi. Resource menormalisasi data untuk front end, termasuk key task_ pada sit2_task_approvals agar JSON tetap berbentuk object. Endpoint penghapusan melindungi dokumen dan data audit yang masih dirujuk.

4.1.17 Implementasi Front End

Front end menggunakan React Router dengan ProtectedRoute, context untuk autentikasi, proyek, chat, notifikasi, dan master data, serta service API terpusat. Komponen wizard SIT/UAT mengatur tiga tahap, mode read-only, bukti per task/skenario, dan approval. Halaman QA/Siber menyediakan pencarian dan filter per assignee bagi role pengawas yang sesuai. UI menggunakan Bahasa Indonesia dan warna identitas #00529C.

Keputusan bisnis tidak dihitung ulang secara bebas pada front end. Contohnya, can_resubmit dan resubmit_blocker berasal dari server; angka ringkasan UAT dihitung backend; dan gate status tetap ditegakkan service. Front end bertugas menyajikan kondisi dan mencegah aksi yang pasti ditolak, sedangkan back end tetap menjadi sumber kebenaran otorisasi.

4.1.18 Keamanan dan Audit Trail

1. Token sesi SPA disimpan pada cookie HttpOnly; Bearer tetap didukung sebagai jalur kompatibilitas.
2. CORS dibatasi oleh origin environment dan mendukung credentials untuk cookie.
3. Form Request dan middleware role memvalidasi input serta akses endpoint.
4. ProjectAccessService mengisolasi proyek berdasarkan peran, penugasan, dan keterlibatan personal.
5. Soft delete, RESTRICT, SET NULL, dan CASCADE dipilih sesuai makna audit setiap relasi.
6. Approval dan histori tidak di-hard-delete; putaran lama ditandai completed, superseded, revoked, atau resubmitted.
7. Document Vault mencegah penghapusan bukti yang masih dirujuk laporan atau berita acara.

4.1.19 Pengujian dan Verifikasi

Tabel 4.9 Ringkasan hasil verifikasi

Jenis	Cakupan	Hasil/Keterangan
Backend test	236 test; 1.467 assertions	Lulus pada snapshot 26 Agustus 2026

Jenis	Cakupan	Hasil/Keterangan
ESLint	npx eslint src	Tidak ada error pada snapshot
Build front end	npx vite build	Berhasil; terdapat peringatan lama ukuran chunk
Database test	SQLite :memory:	Tidak menyentuh database pengembangan
Verifikasi laporan ini	Audit source dan dokumentasi	Tidak menjalankan ulang test/build aplikasi

Snapshot kualitas bersifat historis dan dicantumkan untuk menggambarkan kondisi yang telah dilaporkan dokumentasi proyek. Karena penyusunan laporan tidak mengubah source code dan pengguna memilih menjalankan pengujian sendiri, test suite, ESLint, dan build tidak dijalankan ulang pada pekerjaan ini.

4.1.20 Matriks Keterlacakan Kebutuhan

Matriks keterlacakan menghubungkan kebutuhan dengan modul, data, kontrol, dan bukti verifikasi. Matriks ini bukan daftar seluruh endpoint, tetapi menunjukkan bahwa fungsi utama tidak berdiri sendiri. Setiap kebutuhan memiliki lokasi implementasi, objek data, dan bukti yang dapat digunakan untuk menilai hasil.

Tabel 4.10 Matriks keterlacakan kebutuhan

ID	Modul	Data Utama	Kontrol	Bukti
F-01	Auth dan profil	users, personal_access_tokens, sessions	Cookie/Bearer, middleware	Test autentikasi dan respons sesi
F-02	Registrasi proyek	projects, divisions	Form Request, project scope	Data pengajuan dan histori awal
F-03	Workflow	projects, project_status_histories	ProjectWorkflowService	Transisi, from/to, actor, timestamp
F-04	Task	project_tasks, team_members	Assignee dan status gate	Task selesai/ta

ID	Modul	Data Utama	Kontrol	Bukti
				ke down dan histori revisi
F-05	SIT	sit_uat_data, documents	Tiga tahap dan sign-off	Bukti task, review, berita acara
F-06	UAT	sit_uat_data, approval_rounds, approvers	Putaran, token, individual approval	Skenario, CR, keputusan tiap orang
F-07/F-08	QA dan Siber	test_reports, return_rounds	Jalur independen dan resubmit gate	Laporan, sign-off, task perbaikan
F-09	Rilis	release_requests, projects	Kedua jalur harus passed	Rencana rilis dan keputusan Head of IT
F-10/F-11	Dokumen/komunikasi	document_vaults, notifications, chat_messages	Access scope dan proteksi evidence	File, notifikasi, serta pesan proyek
F-12	Administrasi	users, roles, groups, divisions	Super admin dan menu restriction	Perubahan master dan activity log

4.1.21 Pemetaan Kontrol Keamanan

Kontrol keamanan dipetakan terhadap risiko yang relevan, bukan digunakan untuk mengklaim kepatuhan penuh terhadap ASVS atau SSDF. Pemetaan menunjukkan titik kontrol yang sudah ada dan pekerjaan lanjutan yang masih diperlukan sebelum produksi.

Tabel 4.11 Pemetaan kontrol dan risiko keamanan

Area	Risiko	Kontrol Saat Ini	Tindak Lanjut
Identitas dan sesi	Pencurian/penyalahgunaan sesi	Sanctum, cookie HttpOnly, logout, Bearer compatibility	HTTPS, konfigurasi cookie, expiry, dan monitoring produksi
Otorisasi	Akses proyek atau aksi tanpa hak	RBAC, middleware, ProjectAccessService, assignee gate	Uji negatif per role dan review matriks akses
Validasi	Input tidak sah dan mass assignment	Form Request, validation errors, model fillable	Baseline ASVS dan fuzz/abuse case
Integritas workflow	Status dilompati atau bukti tidak lengkap	ProjectWorkflowService dan prasyarat	Property/transition coverage lebih luas
Audit	Histori diubah atau dihapus	Soft delete, RESTRICT, superseded, activity log	Retensi, immutability, dan ekspor audit
Dokumen	Akses atau penghapusan evidence	Masking nama, project scope, referential protection	Malware scan, object storage, encryption, retention
Transport dan origin	Penyadapan/permintaan lintas origin	CORS environment dan credentials	TLS, CSP, SameSite, CSRF review
Operasional	Kegagalan tak terdeteksi	Log dan failed job framework	Centralized logging, alerting, backup, DR drill

4.1.22 Evaluasi Kualitas Produk

Tabel 4.12 Evaluasi kualitatif berdasarkan ISO/IEC 25010:2023

Karakteristik	Penilaian Kualitatif	Dasar	Kesenjangan
Functional suitability	Kuat	12 role, 27 status, task, SIT/UAT, QA/Siber, dan rilis tercakup	Validasi dengan pengguna resmi tetap diperlukan
Performance efficiency	Belum dinilai	Belum ada angka response time, throughput, atau beban	Lakukan performance test berbasis skenario

Karakteristik	Penilaian Kualitatif	Dasar	Kesenjangan
Compatibility	Cukup	REST/JSON, Bearer, normalisasi key legacy	Uji browser dan integrasi environment produksi
Interaction capability	Cukup	Wizard, filter, pesan blocker, UI Bahasa Indonesia	Usability test dan accessibility audit formal
Reliability	Cukup	Transaksi dan constraint menjaga operasi multi-tabel	Uji kegagalan, retry, backup, dan recovery
Security	Cukup	HttpOnly, RBAC, project scope, audit, pengujian Siber	Threat model, ASVS baseline, pentest independen
Maintainability	Kuat	Service domain, Form Request, Resource, API service terpusat	Pantau kompleksitas service dan coverage
Flexibility	Cukup	Parallel track, return round, kompatibilitas legacy	Konfigurasi role dinamis masih terbatas
Safety	Di luar fokus	Bukan sistem kontrol keselamatan	Tetap evaluasi risiko perubahan terhadap layanan bank

Penilaian di atas bersifat analisis desain dan bukti proyek, bukan hasil sertifikasi. Istilah kuat atau cukup menunjukkan tingkat dukungan relatif berdasarkan artefak yang tersedia. Karakteristik yang belum memiliki metrik dinyatakan belum dinilai agar laporan tidak mengubah asumsi menjadi fakta.

4.1.23 Keputusan Arsitektur dan Trade-off

Tabel 4.13 Ringkasan keputusan arsitektur

Keputusan	Manfaat	Trade-off	Mitigasi/Roadmap
State transition terpusat	Konsistensi dan audit	Service dapat menjadi kompleks	Pisahkan validator dan tambahkan transition tests
Approval UAT	Snapshot/keputusa	Migrasi dari JSON	Helper normalisasi

Keputusan	Manfaat	Trade-off	Mitigasi/Roadmap
ternormalisasi	n per orang jelas	legacy perlu kompatibilitas	dan supersession
SIT/UAT sebagian dalam JSON	Fleksibel terhadap evolusi wizard	Query dan constraint lebih terbatas	Normalisasi bertahap bila pola stabil
Cookie HttpOnly + Bearer	Keamanan SPA dan kompatibilitas	Dua jalur autentikasi perlu diuji	Kontrak middleware dan dokumentasi eksplisit
QA/Siber independen	Status tidak ambigu dan dapat paralel	Koordinasi menuju gate lebih kompleks	Prasyarat kedua jalur passed
Soft delete dan supersession	Audit trail terlindungi	Volume data bertambah	Kebijakan retensi dan arsip resmi
Reverb belum aktif	Menghindari klaim operasional palsu	Polling menambah request	Aktifkan setelah kapasitas dan SOP diputuskan

4.2 Pembahasan

4.2.1 Kesesuaian terhadap Tujuan

Tujuan pertama adalah memusatkan tata kelola proyek teknologi dari pengajuan sampai produksi. Hasil implementasi menunjukkan bahwa kebutuhan, review, analisis, pengembangan, task, SIT, UAT, QA, keamanan siber, release request, dan quality gate berada pada satu agregat proyek. Pemusatan ini mengurangi kebutuhan mencocokkan status dari beberapa media dan memungkinkan pengguna melihat konteks sebelum mengambil tindakan.

Tujuan kedua adalah menegakkan alur dan pembagian wewenang. Keberadaan 27 status saja tidak cukup; pencapaian tujuan ditentukan oleh transisi terpusat, role permission, cakupan proyek, assignment, serta validasi prasyarat. Karena ProjectWorkflowService menjadi satu-satunya jalur transisi, aturan tidak bergantung pada tombol yang terlihat di front end. Hal ini membuat workflow memiliki sifat enforceable, bukan hanya informatif.

Tujuan ketiga berkaitan dengan pengujian, approval, dan audit. Wizard SIT/UAT, putaran persetujuan, jalur QA/Siber, return round, Document Vault, histori status, dan activity log membentuk bukti berlapis. Hasil tersebut memenuhi sasaran desain. Walaupun demikian, keberhasilan operasional masih memerlukan data pengguna nyata, SOP, infrastruktur produksi, pengujian ulang, dan evaluasi setelah sistem digunakan.

4.2.2 Kesesuaian dengan SDLC dan Literatur

ISO/IEC/IEEE 12207 menempatkan lifecycle sebagai kerangka proses yang dapat disesuaikan dan dilaksanakan berulang atau paralel [15]. NagariSDLC sejalan dengan prinsip tersebut karena tidak memaksakan satu model delivery. Task serta perbaikan dapat iteratif, sementara keputusan formal tetap menggunakan gate. QA dan Siber berjalan paralel; SIT dan UAT berulang ketika revisi mayor terjadi. Struktur ini lebih tepat disebut workflow governance hibrida daripada waterfall murni.

Prinsip agile menekankan kolaborasi dan respons terhadap perubahan [25], sedangkan penelitian Dybå dan Dingsøyr mengingatkan bahwa manfaat metode perlu dipahami sesuai konteks [26]. Implementasi NagariSDLC menerima perubahan melalui Change Request, task revisi, return round, dan supersession. Pada saat yang sama, sistem menahan approval ketika artefak berubah. Dengan cara ini, adaptasi tidak menghapus kebutuhan akan bukti dan akuntabilitas.

Continuous software engineering berupaya mengurangi keterputusan antaraktivitas [27]. NagariSDLC belum mengotomatisasi pipeline deployment, tetapi telah mengurangi keterputusan informasi: task menjadi basis pengujian, hasil pengujian menjadi prasyarat rilis, dan rilis memiliki keputusan eksplisit. Integrasi CI/CD di masa depan akan bernilai apabila pipeline menyampaikan evidence kembali ke project record, bukan berjalan sebagai kanal yang terpisah.

4.2.3 Analisis Workflow dan State Machine

Kompleksitas workflow berasal dari kombinasi fase linear, revisi, hold, penolakan, dua jalur paralel, dan status legacy. Model satu kolom status proyek tidak cukup untuk menyatakan kemajuan QA dan Siber secara bersamaan. Solusinya adalah mempertahankan status proyek sebagai fase global serta qa_status dan cyber_status sebagai state lokal. PENDING_GOLIVE hanya terbuka ketika kedua state lokal PASSED.

Pemisahan state global dan lokal mengurangi ledakan kombinasi status. Jika setiap kombinasi QA–Siber dibuat menjadi status proyek, jumlah status akan bertambah dan maknanya sulit dipahami. Trade-off-nya adalah setiap endpoint harus menilai lebih dari satu kolom. TestingTrackService dan validator workflow berfungsi sebagai tempat konsolidasi aturan tersebut.

READY_FOR_UAT dan UAT_PASSED dipertahankan untuk kompatibilitas histori, bukan sebagai bukti bahwa alur aktif masih memiliki UAT final. Keputusan ini penting karena menghapus nilai lama dapat merusak audit, sedangkan mengaktifkannya kembali akan menimbulkan alur yang tidak sesuai. Dokumentasi membedakan status legacy, opsional, aktif, dan terminal agar pembaca tidak menyimpulkan alur hanya dari nama enum.

4.2.4 Analisis Kontrol Akses dan Separation of Duties

Model RBAC mempermudah pengelolaan permission melalui role [21]. NagariSDLC menambahkan project scope karena role tidak menyatakan

hubungan seseorang dengan proyek tertentu. Seorang developer hanya boleh melihat atau bertindak pada proyek yang ditugaskan; tester juga dibatasi oleh disposisi jalur. Kombinasi role dan relation-based scope lebih sempit daripada akses role global.

Separation of duties terlihat pada pemisahan pelaksana, reviewer, dan pengambil keputusan. QA Tester membuat laporan, QA Lead melakukan sign-off; Pentester menguji, Cyber Lead melakukan sign-off; PM mengajukan rilis, Head of IT memutuskan quality gate. Persetujuan UAT dibagi ke sisi peminta dan IT dengan keputusan individual. Pemisahan ini menurunkan risiko satu orang membuat, memeriksa, dan menyetujui hasilnya sendiri.

Keterbatasannya adalah daftar role otorisasi masih tetap di kode. Administrasi dapat membuat role baru, tetapi role tersebut tidak otomatis memperoleh semantics fase. Kondisi ini aman dari sudut default-deny, tetapi kurang fleksibel untuk organisasi yang sering mengubah struktur. Pengembangan permission engine dinamis harus dilakukan hati-hati karena kesalahan konfigurasi dapat memiliki dampak lebih besar daripada daftar eksplisit.

4.2.5 Analisis SIT, UAT, QA, dan Keamanan Siber

ISO/IEC/IEEE 29119 menekankan proses pengujian yang dapat ditata dari perencanaan sampai penyelesaian [17]. NagariSDLC menerjemahkannya ke artefak praktis: skenario, pelaksana, hasil, bukti, komentar, approval, laporan, dan status. Draft dipisahkan dari final submission agar pengguna dapat menyimpan pekerjaan tanpa memicu transisi. Pembekuan bukti setelah lulus mencegah hasil yang telah disetujui berubah tanpa putaran baru.

Perbedaan revisi minor dan mayor pada UAT memiliki implikasi proses. Revisi minor mempertahankan konteks UAT dan membuka task perbaikan; approval ditahan sampai task resolved. Revisi mayor menganggap perubahan cukup besar untuk membatalkan relevansi hasil sebelumnya, sehingga SIT diulang penuh dan UAT dimulai dari tahap pertama. Keputusan ini lebih mahal, tetapi menjaga validitas bukti integrasi dan penerimaan.

SSDF, WSTG, dan SP 800-115 menekankan pengujian keamanan yang terencana, analisis temuan, mitigasi, serta integrasi keamanan dalam lifecycle [7], [18], [19]. Jalur Siber memenuhi kerangka proses dasar tersebut melalui disposisi, tested scenarios, laporan, sign-off, return round, dan task perbaikan. Namun kontrol ini belum setara dengan program AppSec lengkap tanpa threat modeling, baseline ASVS, dependency scanning, SAST/DAST, secret scanning, dan vulnerability response operasional.

4.2.6 Analisis Keamanan Aplikasi dan Audit Trail

Penggunaan cookie HttpOnly mengurangi paparan token terhadap JavaScript, tetapi tidak menghilangkan risiko XSS maupun CSRF [23]. Karena itu kontrol perlu dipandang sebagai defense in depth: validasi input, output encoding, kebijakan origin, X-Requested-With, SameSite yang tepat, TLS, pembatasan sesi,

serta otorisasi server tetap dibutuhkan. Bearer compatibility memperluas skenario penggunaan sekaligus memperluas cakupan pengujian.

Audit trail memiliki dua dimensi: kelengkapan informasi dan perlindungan histori. NagariSDLC mencatat aktor, waktu, status, keputusan, laporan, serta bukti. Perlindungan dilakukan melalui soft delete, foreign key RESTRICT, status superseded/revoked, dan larangan penghapusan evidence yang masih dirujuk. Keputusan approved pada putaran lama tetap tersimpan sehingga kronologi tidak direvisi secara retrospektif.

Masih diperlukan kebijakan retensi formal: berapa lama dokumen disimpan, siapa yang boleh memulihkan soft delete, kapan data boleh dimusnahkan, dan bagaimana ekspor audit dilakukan. Tanpa kebijakan tersebut, perlindungan teknis dapat menyebabkan akumulasi data tanpa aturan. Kebijakan perlu mempertimbangkan kebutuhan audit internal, klasifikasi informasi, dan keputusan resmi instansi.

4.2.7 Analisis Data, Transaksi, dan Kompatibilitas

Model data 17 tabel domain dan delapan tabel framework menggambarkan pemisahan antara data bisnis dan fasilitas infrastruktur aplikasi. Project menjadi pusat relasi. Approval UAT dinormalisasi karena membutuhkan query per orang, snapshot putaran, token, dan status yang konsisten. Data wizard SIT/UAT tetap pada JSON karena strukturnya berkembang dan kompatibilitas baris lama masih dibutuhkan.

Keputusan mempertahankan JSON mempunyai trade-off. Penambahan field lebih mudah, tetapi constraint dan query lintas key lebih lemah. Oleh karena itu key yang memengaruhi workflow tidak boleh dibaca bebas dari berbagai tempat. Contohnya, penanda UAT restart hanya dibaca melalui `Project::isUatRestartPending()`, sedangkan task approval dinormalisasi agar key `task_` selalu konsisten.

Transaksi database menjaga atomicity pada operasi multi-tabel. Tanpa transaksi, proyek dapat berpindah status sementara histori atau notifikasi gagal tersimpan. Constraint foreign key melengkapi validasi aplikasi. Kombinasi service transaction dan referential integrity memberi dua lapis perlindungan, walaupun pengujian concurrency dan idempotency tetap diperlukan untuk skenario request ganda.

4.2.8 Analisis Antarmuka dan Pengalaman Pengguna

Antarmuka workflow harus membantu pengguna menjawab tiga hal: kondisi saat ini, alasan kondisi tersebut, dan aksi berikutnya. Dashboard, detail proyek, Kanban, wizard, matriks approval, quality gate, notifikasi, dan activity log membagi informasi sesuai konteks. Penggunaan Bahasa Indonesia mengurangi beban terminologi bagi pengguna internal, sedangkan istilah teknis dipertahankan ketika menjadi nama domain resmi seperti SIT dan UAT.

Wizard SIT/UAT mengurangi kepadatan satu formulir besar, tetapi berpotensi menyembunyikan ketergantungan antarbagian. Karena itu ringkasan, status penyimpanan draft, indikator read-only, dan blocker perlu terlihat. Pesan error server harus diterjemahkan menjadi arahan yang dapat dilakukan, bukan hanya kode status. `can_resubmit` dan `resubmit_blocker` dari server adalah contoh pengiriman alasan bisnis ke UI.

Evaluasi usability formal belum dilakukan. Berdasarkan kerangka Nielsen [34], pekerjaan lanjutan perlu menilai learnability, efficiency, error recovery, consistency, dan satisfaction melalui task-based usability test. Accessibility juga perlu diperiksa untuk navigasi keyboard, label form, kontras, fokus state, tabel kompleks, dan pesan validasi. Laporan hanya menyatakan dukungan desain yang tersedia, bukan hasil pengujian pengguna.

4.2.9 Kelebihan Sistem

1. Alur proyek terpusat dengan 27 status dan otorisasi yang terdokumentasi.
2. SIT/UAT bertahap dengan bukti dan approval individual.
3. QA dan keamanan siber paralel tanpa status gabungan yang ambigu.
4. Perlindungan audit trail melalui soft delete, supersession, dan larangan hard delete bukti.
5. Kontrak API konsisten dan satu lapisan layanan pada front end.
6. Kompatibilitas data lama dipertahankan tanpa menghidupkan kembali alur yang dipensiunkan.

Kelebihan utama bukan jumlah modul, melainkan hubungan antarmodul. Status terkait dengan task; task terkait dengan SIT/UAT; hasil UAT terkait dengan approver; kegagalan QA/Siber terkait dengan return round dan task perbaikan; rilis terkait dengan dua jalur lulus serta keputusan Head of IT. Hubungan tersebut membentuk traceability chain yang lebih bernilai daripada kumpulan fitur yang berdiri sendiri.

4.2.10 Kekurangan dan Batasan

1. Target database, realtime, queue, object storage, backup, monitoring, dan SLA produksi belum diputuskan.
2. Reverb belum aktif pada template environment yang memakai broadcast log.
3. Role baru yang dibuat melalui administrasi belum berfungsi tanpa perubahan daftar otorisasi di kode.
4. `menu_access` hanya menyaring tampilan dan tidak menutup route secara mandiri.
5. Chat masih memakai polling dan belum menggunakan WebSocket.
6. Beberapa data SIT/UAT tetap berbentuk JSON karena kebutuhan kompatibilitas dan evolusi fitur.

Batasan tersebut menunjukkan perbedaan antara software yang telah memiliki fungsi inti dan layanan yang siap dioperasikan pada skala organisasi.

Production readiness membutuhkan konfigurasi, kapasitas, keamanan jaringan, backup, observability, runbook, support model, dan acceptance operasional. Karena keputusan tersebut belum tersedia, laporan tidak mengklaim kesiapan produksi penuh walaupun status workflow memiliki LIVE_PRODUCTION.

4.2.11 Kendala dan Solusi

Tabel 4.14 Kendala dan solusi

Kendala	Solusi
Percabangan status kompleks	Memusatkan transisi dan prasyarat di ProjectWorkflowService.
QA/Siber paralel	Menyimpan qa_status dan cyber_status secara independen.
Revisi UAT mayor	Mengarsipkan siklus, SIT ulang penuh, dan UAT ulang dari Tahap 1.
Approval lintas akun	Menyediakan internal account dan external link dengan verifikasi nomor HP.
Jejak audit berisiko hilang	Soft delete, RESTRICT, superseded/revoked, dan larangan hapus bukti.
Data legacy	Membaca key lama melalui helper normalisasi tanpa menulis kembali alur pensiun.
Pengajuan ulang QA/Siber	Return round dan gerbang task perbaikan per jalur.

Solusi pada tabel dipilih pada lapisan yang paling bertanggung jawab. Status paralel tidak diselesaikan dengan label UI, melainkan model data dan service; histori tidak dilindungi hanya dengan menyembunyikan tombol hapus, tetapi melalui aturan delete dan status supersession. Pendekatan ini mengurangi kemungkinan solusi terlewat ketika request datang dari klien berbeda.

4.2.12 Potensi Pengembangan

1. Menetapkan arsitektur produksi, CI/CD, staging, backup, monitoring, logging, dan disaster recovery.
2. Mengaktifkan Reverb dan queue worker setelah kebutuhan realtime serta kapasitas server diputuskan.
3. Menambah pengujian end-to-end untuk alur kritis, terutama approval eksternal dan siklus revisi.
4. Menentukan kebijakan retensi, terminal CANCELLED, pemulihan soft delete, dan pemusnahan data yang sah.
5. Mengevaluasi bundle front end dan pemisahan chunk agar peringatan ukuran aset dapat dikurangi.

6. Menyusun panduan operasional, matriks RACI resmi, dan SOP penggunaan untuk setiap peran.

Tabel 4.15 Roadmap pengembangan

Urutan	Program	Ruang Lingkup	Hasil Diharapkan
Prioritas 1	Production readiness	Environment, TLS, database, storage, backup, queue, monitoring, runbook	Prasyarat penggunaan operasional
Prioritas 2	Security assurance	Threat model, ASVS baseline, SAST/DAST, dependency/secret scan, pentest	Risiko keamanan lebih terukur
Prioritas 3	Quality engineering	E2E, performance, accessibility, usability, mutation/transition coverage	Bukti kualitas lebih lengkap
Prioritas 4	Delivery integration	CI/CD, deployment evidence, release automation, rollback drill	Kontinuitas dari kode ke produksi
Prioritas 5	Governance analytics	Lead time, defect escape, approval aging, bottleneck, audit export	Perbaikan proses berbasis data

Urutan roadmap menempatkan kesiapan dan keamanan sebelum fitur tambahan. Realtime atau dashboard analitik akan memberikan nilai terbatas apabila backup, monitoring, dan prosedur pemulihan belum ada. Setelah fondasi operasional tersedia, metrik governance dapat digunakan untuk mengidentifikasi bottleneck tanpa menjadikan jumlah aktivitas sebagai satu-satunya ukuran produktivitas.

4.2.13 Kontribusi Mahasiswa

Kontribusi mahasiswa mencakup pengembangan fullstack, analisis aturan bisnis, integrasi front end–back end, workflow, pengujian, dokumentasi, dan perbaikan berbasis temuan. Kontribusi teknis tidak hanya berupa penulisan halaman dan endpoint, tetapi juga menjaga konsistensi kontrak, memusatkan aturan domain, melindungi histori, serta menerjemahkan proses bisnis menjadi state dan prasyarat yang dapat diuji. Uraian kontribusi dibatasi pada aktivitas yang dapat diverifikasi melalui implementasi dan dokumentasi proyek.

BAB V

PENUTUP

5.1 Kesimpulan

1. NagariSDLC berhasil membentuk sistem tata kelola SDLC terpusat untuk mengelola proyek teknologi dari pengajuan sampai produksi.
2. Mesin status, RBAC, cakupan proyek, validasi prasyarat, histori, dan notifikasi menegakkan alur serta pemisahan tanggung jawab.
3. SIT dan UAT dikelola melalui wizard tiga tahap dengan bukti, revisi, dan approval individual yang mempertahankan audit trail.
4. QA dan keamanan siber berjalan paralel dengan status independen, return round, task perbaikan, dan gerbang pengajuan ulang per jalur.
5. Rilis hanya dapat diajukan setelah kedua jalur lulus dan diputuskan Head of IT pada quality gate; tidak ada UAT final setelah QA/Siber.
6. Arsitektur Laravel–React–MySQL dan kontrak REST menyediakan fondasi pengembangan yang terstruktur, sementara keputusan infrastruktur produksi masih perlu dilengkapi.

5.2 Saran

1. Lakukan pengujian regresi backend, pemeriksaan lint, build frontend, dan pengujian end-to-end setelah setiap perubahan utama agar kestabilan sistem tetap terjaga.
2. Tetapkan konfigurasi dan prosedur operasional produksi untuk database, queue, object storage, CORS, Reverb, backup, monitoring, logging, serta retensi data.
3. Tambahkan pemantauan kesehatan layanan, metrik performa, pencatatan kesalahan terpusat, dan mekanisme pemulihan agar gangguan operasional dapat dideteksi lebih cepat.
4. Pertahankan seluruh transisi proyek melalui ProjectWorkflowService dan seluruh penulisan return round melalui ProjectReturnRoundService agar aturan bisnis tetap terpusat.
5. Pertahankan approval, histori status, laporan pengujian, activity log, dan bukti dokumen sebagai audit trail serta hindari hard delete tanpa kebijakan retensi resmi.
6. Lakukan evaluasi usability dan pengukuran karakteristik kualitas ISO/IEC 25010 secara berkala menggunakan data penggunaan nyata setelah sistem diterapkan.

DAFTAR PUSTAKA

- [1] I. Sommerville, Software Engineering, 10th ed. Pearson, 2016.
- [2] R. S. Pressman and B. R. Maxim, Software Engineering: A Practitioner's Approach, 9th ed. McGraw-Hill, 2020.
- [3] R. T. Fielding, Architectural Styles and the Design of Network-based Software Architectures. University of California, Irvine, 2000.
- [4] Laravel, Laravel 13.x Documentation. <https://laravel.com/docs/13.x> [Diakses: 28 Agustus 2026].
- [5] Meta Open Source, React Documentation. <https://react.dev> [Diakses: 28 Agustus 2026].
- [6] Vite, Vite Documentation. <https://vite.dev> [Diakses: 28 Agustus 2026].
- [7] OWASP Foundation, Web Security Testing Guide, Version 4.2. <https://owasp.org/www-project-web-security-testing-guide/v42/> [Diakses: 28 Agustus 2026].
- [8] Tailwind Labs, Tailwind CSS Documentation. <https://tailwindcss.com/docs> [Diakses: 28 Agustus 2026].
- [9] ISO/IEC/IEEE 12207:2017, Systems and software engineering - Software life cycle processes. ISO, 2017.
- [10] ISO/IEC/IEEE 29119-1:2022, Software and systems engineering - Software testing - Part 1: General concepts. ISO, 2022. <https://www.iso.org/standard/81291.html> [Diakses: 28 Agustus 2026].
- [11] Laravel, Laravel Sanctum Documentation. <https://laravel.com/docs/13.x/sanctum> [Diakses: 28 Agustus 2026].
- [12] Oracle, MySQL Reference Manual. <https://dev.mysql.com/doc/> [Diakses: 28 Agustus 2026].
- [13] Mermaid, Mermaid Documentation. <https://mermaid.js.org> [Diakses: 28 Agustus 2026].
- [14] Tim Proyek NagariSDLC, Dokumentasi internal: AI_HANDOFF, PROJECT_SUMMARY, WORKFLOW, ARCHITECTURE, DATA_MODEL, API_REFERENCE, dan PRD, snapshot 26 Agustus 2026.
- [15] ISO/IEC/IEEE 12207:2026, Systems and software engineering - Software life cycle processes. ISO, 2026. <https://www.iso.org/standard/90219.html> [Diakses: 28 Agustus 2026].
- [16] ISO/IEC 25010:2023, Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model. ISO, 2023. <https://www.iso.org/standard/78176.html> [Diakses: 28 Agustus 2026].

- [17] ISO/IEC/IEEE 29119-2:2021, Software and systems engineering - Software testing - Part 2: Test processes. ISO, 2021.
<https://www.iso.org/standard/79428.html> [Diakses: 28 Agustus 2026].
- [18] M. Souppaya, K. Scarfone, and D. Dodson, Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities, NIST SP 800-218. National Institute of Standards and Technology, 2022. doi: 10.6028/NIST.SP.800-218.
- [19] K. Scarfone, M. Souppaya, A. Cody, and A. Orebaugh, Technical Guide to Information Security Testing and Assessment, NIST SP 800-115. National Institute of Standards and Technology, 2008. doi: 10.6028/NIST.SP.800-115.
- [20] OWASP Foundation, Application Security Verification Standard, Version 5.0.0. <https://owasp.org/www-project-application-security-verification-standard/> [Diakses: 28 Agustus 2026].
- [21] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, 'Role-Based Access Control Models,' *Computer*, vol. 29, no. 2, pp. 38-47, 1996, doi: 10.1109/2.485845.
- [22] R. Fielding, M. Nottingham, and J. Reschke, HTTP Semantics, RFC 9110. IETF, 2022. doi: 10.17487/RFC9110.
- [23] A. Barth, HTTP State Management Mechanism, RFC 6265. IETF, 2011.
<https://www.rfc-editor.org/info/rfc6265/> [Diakses: 28 Agustus 2026].
- [24] Object Management Group, Unified Modeling Language, Version 2.5.1, 2017. <https://www.omg.org/spec/UML/2.5.1> [Diakses: 28 Agustus 2026].
- [25] K. Beck et al., Manifesto for Agile Software Development and Principles behind the Agile Manifesto, 2001. <https://agilemanifesto.org/> [Diakses: 28 Agustus 2026].
- [26] T. Dybå and T. Dingsøy, 'Empirical studies of agile software development: A systematic review,' *Information and Software Technology*, vol. 50, no. 9-10, pp. 833-859, 2008, doi: 10.1016/j.infsof.2008.01.006.
- [27] B. Fitzgerald and K.-J. Stol, 'Continuous software engineering: A roadmap and agenda,' *Journal of Systems and Software*, vol. 123, pp. 176-189, 2017, doi: 10.1016/j.jss.2015.06.063.
- [28] L. E. Lwakatare et al., 'DevOps in practice: A multiple case study of five companies,' *Information and Software Technology*, vol. 114, pp. 217-230, 2019, doi: 10.1016/j.infsof.2019.06.010.
- [29] B. W. Boehm, 'A Spiral Model of Software Development and Enhancement,' *Computer*, vol. 21, no. 5, pp. 61-72, 1988, doi: 10.1109/2.59.
- [30] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Addison-Wesley Professional, 2021.

- [31] M. Fowler, UML Distilled: A Brief Guide to the Standard Object Modeling Language, 3rd ed. Addison-Wesley Professional, 2004.
- [32] J. Humble and D. Farley, Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley Professional, 2010.
- [33] N. Forsgren, J. Humble, and G. Kim, Accelerate: The Science of Lean Software and DevOps. IT Revolution Press, 2018.
- [34] J. Nielsen, Usability Engineering. Academic Press, 1993.
- [35] Bank Nagari, 'Visi dan Misi' dan profil kantor pusat.
<https://www.banknagari.co.id/profile?page=hkcukYGoSeEHiTnFnlSWg%3D%3D> [Diakses: 28 Agustus 2026].
- [36] Bank Nagari, 'Sejarah.'
<https://banknagari.co.id/profile?page=G1lnugtldJSwW%2FaHA5UGAQ%3D%3D> [Diakses: 28 Agustus 2026].

LAMPIRAN

Lampiran A Endpoint API

Tabel A.1 Daftar endpoint API

Method	Endpoint	Fungsi
POST	/auth/login	Login dan pembuatan sesi
POST	/auth/logout	Mencabut sesi aktif
POST	/auth/refresh	Rotasi token sesi
POST	/auth/forgot-password	Permintaan reset tanpa enumerasi akun
POST	/auth/reset-password	Reset kata sandi dan cabut seluruh token
GET/POST	/projects	Daftar terscope / pengajuan proyek
GET/PATCH	/projects/{id}	Detail / perubahan proyek
PATCH	/projects/{id}/status	Transisi status resmi
POST	/projects/{id}/team	Alokasi tim
GET	/projects/{id}/timeline	Riwayat status
GET	/projects/{id}/sit-gate	Prasyarat masuk SIT
POST	/projects/{id}/sit-approval	Approval SIT
PUT	/projects/{id}/uat-execution/draft	Simpan draft eksekusi UAT
POST	/projects/{id}/uat-execution	Finalisasi eksekusi UAT
GET	/projects/{id}/uat-approval-matrix	Matrix approver terbaru
POST	/projects/{id}/uat-approval-rounds	Membuka putaran approval
POST	/projects/{id}/uat-approval-rounds/sync	Sinkronisasi peserta
POST	/projects/{id}/uat-approvers/{approver}/decision	Keputusan approver internal
GET/POST	/projects/{projectId}/tasks	Daftar / buat task
PATCH/DELETE	/tasks/{taskId}	Ubah / hapus task

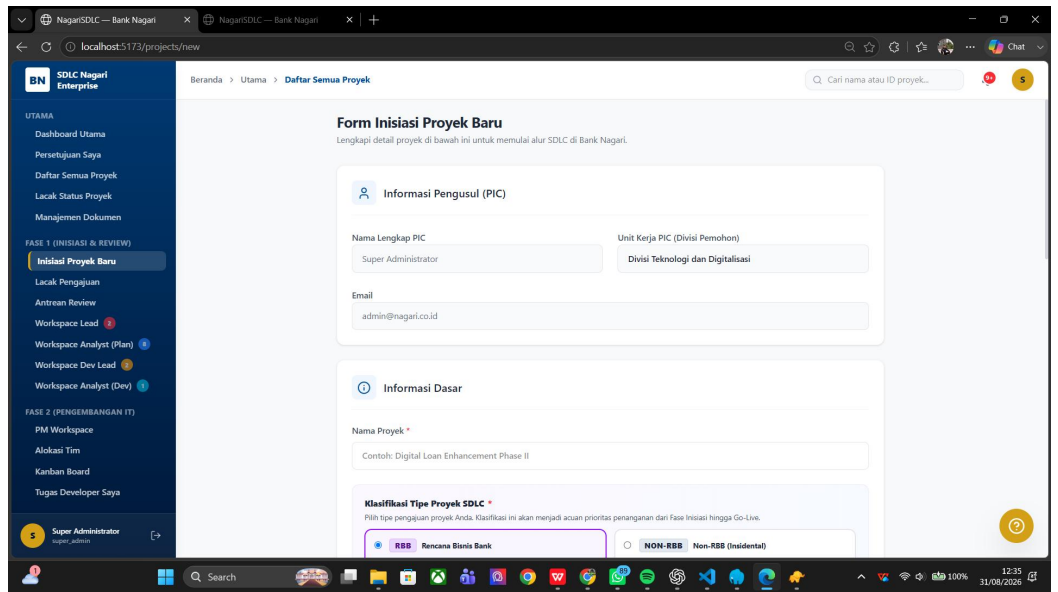
Method	Endpoint	Fungsi
ELETE		
POST	/tasks/{taskId}/request-revision	Permintaan revisi task
GET/POST	/projects/{projectId}/chat	Diskusi proyek
GET/POST	/documents	Daftar / unggah dokumen
GET	/documents/{id}/download	Unduh dokumen terotorisasi
DELETE	/documents/{id}	Hapus dengan proteksi audit trail
POST	/qa-requests/submit assign report sign-off	Empat langkah QA
POST	/cyber-requests/submit assign report sign-off	Empat langkah keamanan siber
GET/POST	/release-requests	Daftar / pengajuan rilis
GET	/quality-gate/queue	Antrean quality gate
POST	/quality-gate/approve reject	Keputusan rilis
GET	/dashboard/summary	Ringkasan sesuai scope
GET	/dashboard/analytics	Analitik global Super Admin
GET/PATCH	/notifications	Notifikasi dan status baca
GET	/activity-logs	Jejak aktivitas
GET/POST/PATCH/DELETE	/users roles groups divisions	Master dan administrasi
GET	/health	Pemeriksaan koneksi database

Lampiran B Kamus Data Ringkas

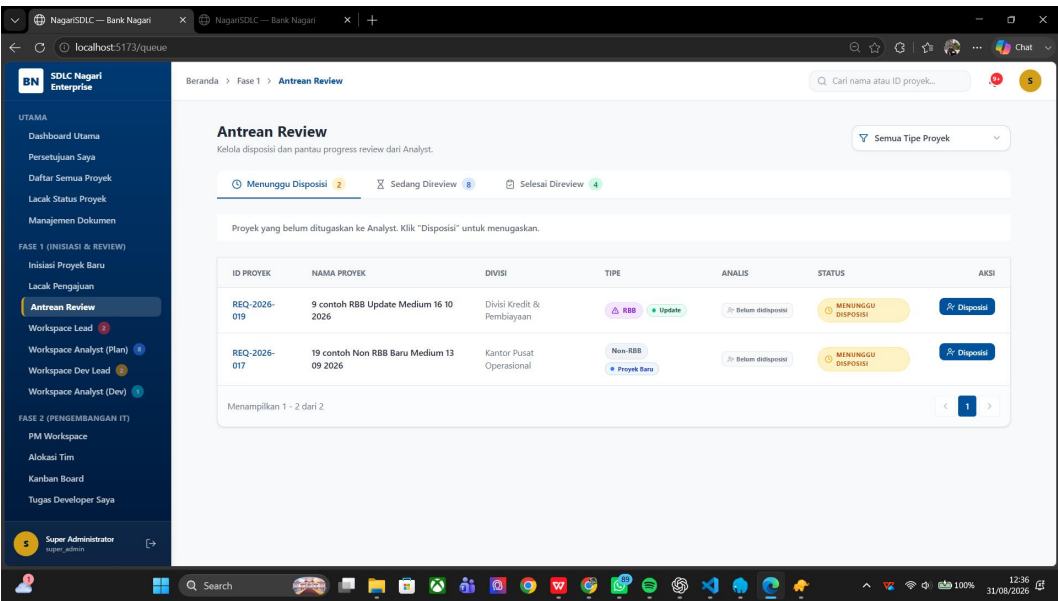
Tabel B.1 Kamus data ringkas

Field	Tipe	Keterangan
projects.req_id	string	Kode REQ-YYYY-NNN
projects.status	enum/string	27 status ProjectStatus
projects.qa_status	enum/string	Status jalur QA
projects.cyber_status	enum/string	Status jalur keamanan siber
projects.sit_uat_data	json	Data aktif dan histori wizard SIT/UAT
projects.priority	string	High, Medium, atau Low
project_tasks.status	enum/string	todo, in_progress, hold, done, take_down
project_tasks.return_round_id	FK nullable	Task perbaikan putaran QA/Siber
roles.menu_access	json nullable	Pembatasan menu yang bersifat mengurangi
test_reports.tested_scenarios	text nullable	Ruang lingkup pengujian bebas
test_reports.evidence_document_ids	json	ID Document Vault untuk bukti
uat_approval_rounds.status	string	active, completed, superseded
uat_approvers.decision_statuses	string	pending, approved, rejected, revoked
project_return_rounds.status	string	OPEN atau RESUBMITTED
release_requests.rollback_plan	text	Rencana pemulihan rilis

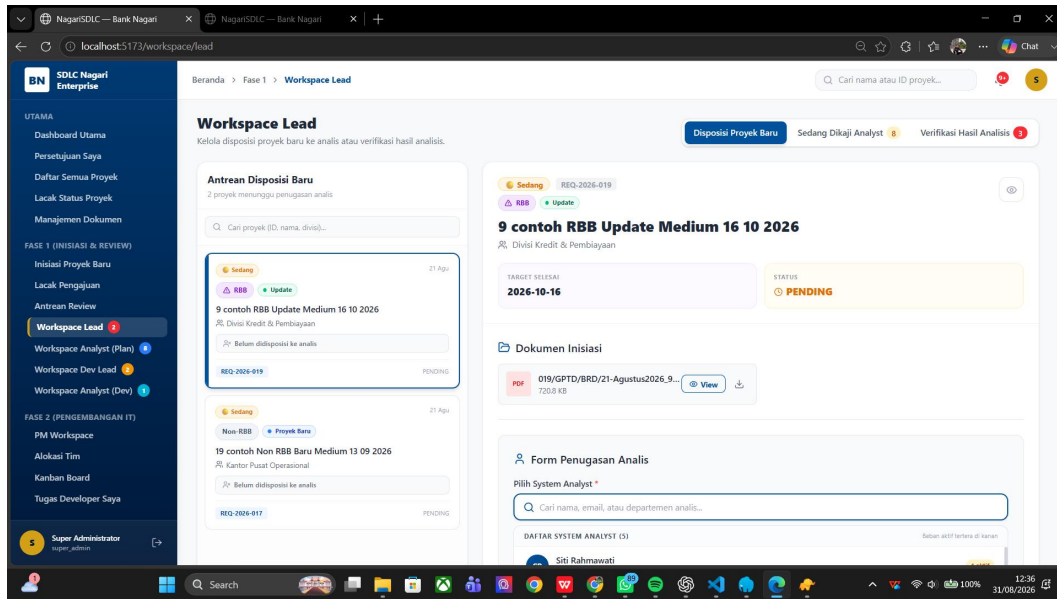
Lampiran C Tampilan Lengkap



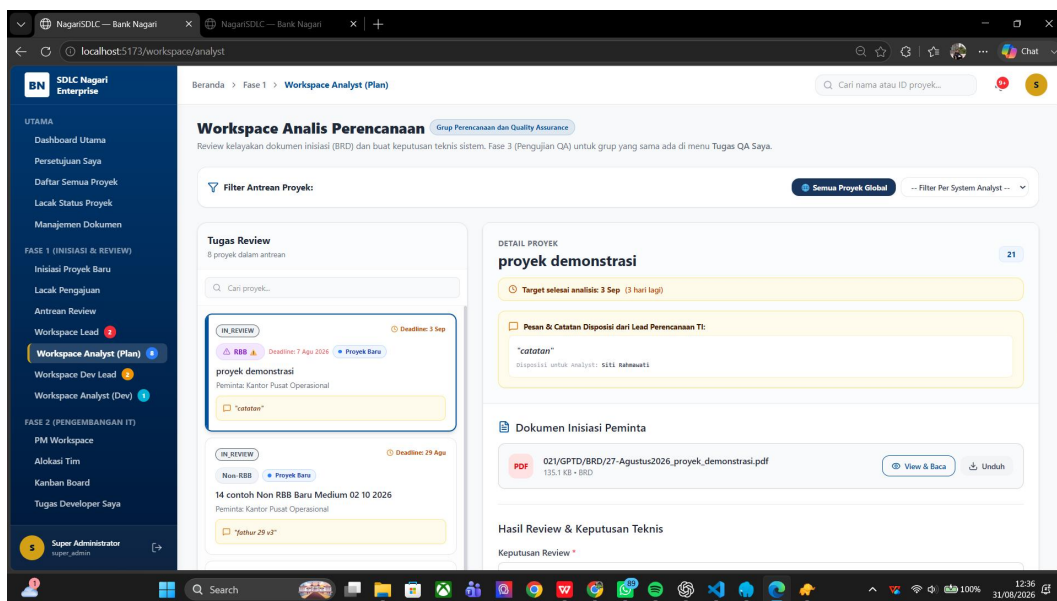
Gambar Lampiran 1 Form Inisiasi Proyek oleh User



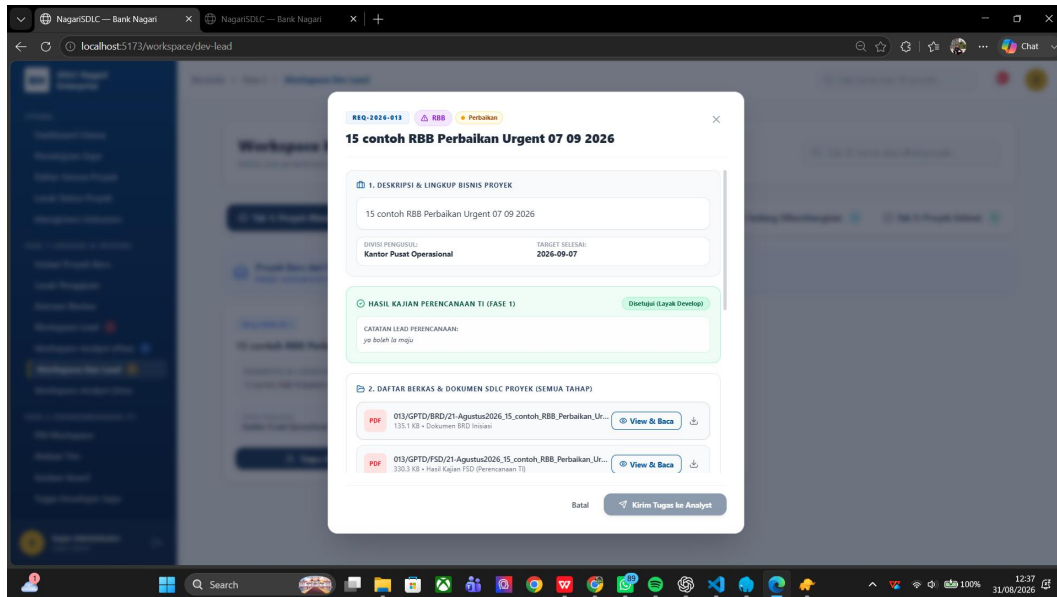
Gambar Lampiran 2 Antrian Review



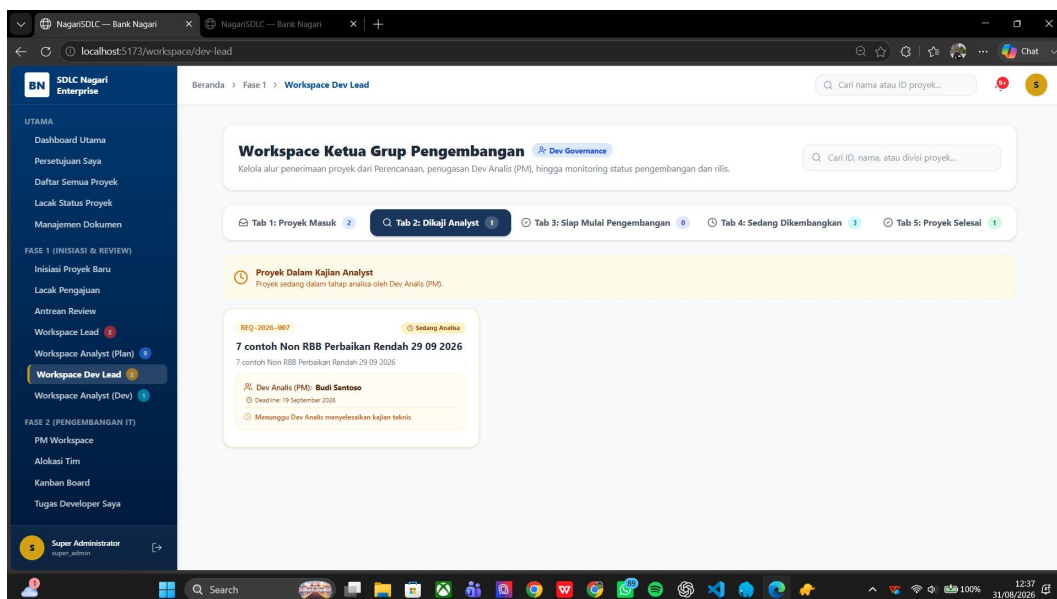
Gambar Lampiran 3 Workspace Lead



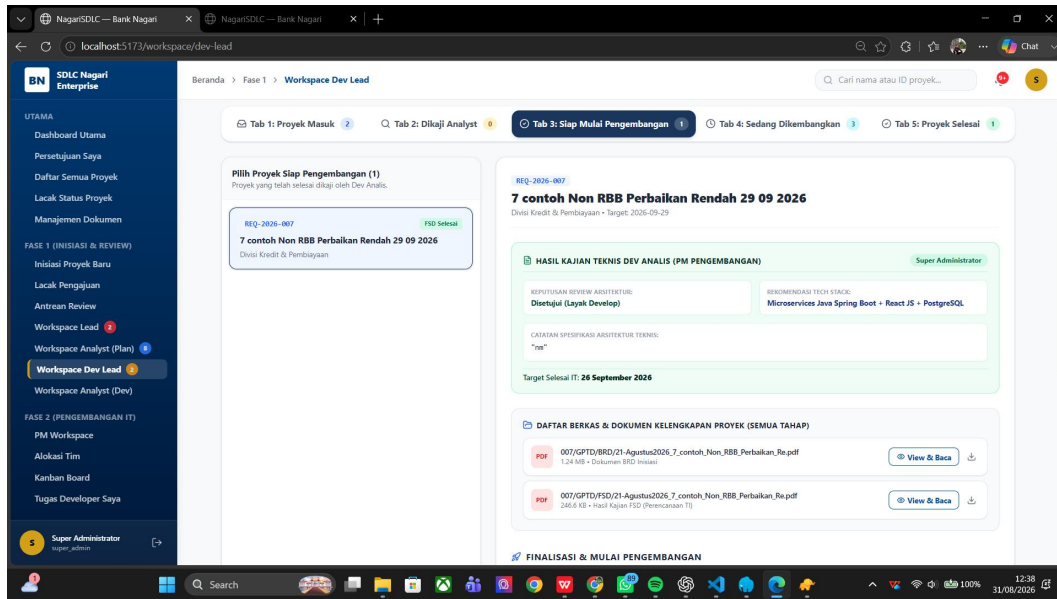
Gambar Lampiran 4 Workspace Analis Perencanaan



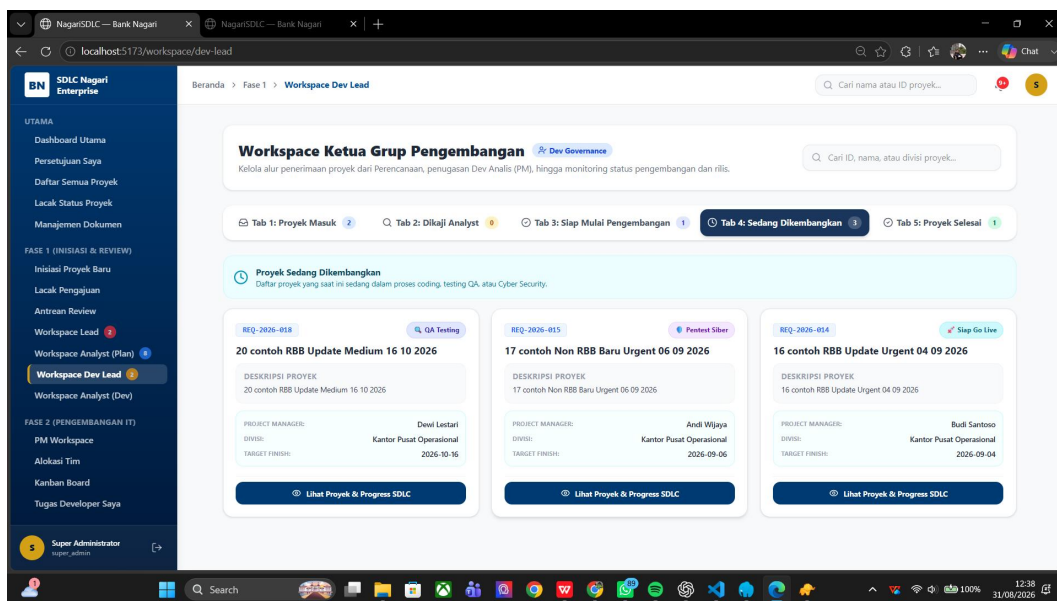
Gambar Lampiran 5 Penunjukan Analis oleh Lead Pengembangan



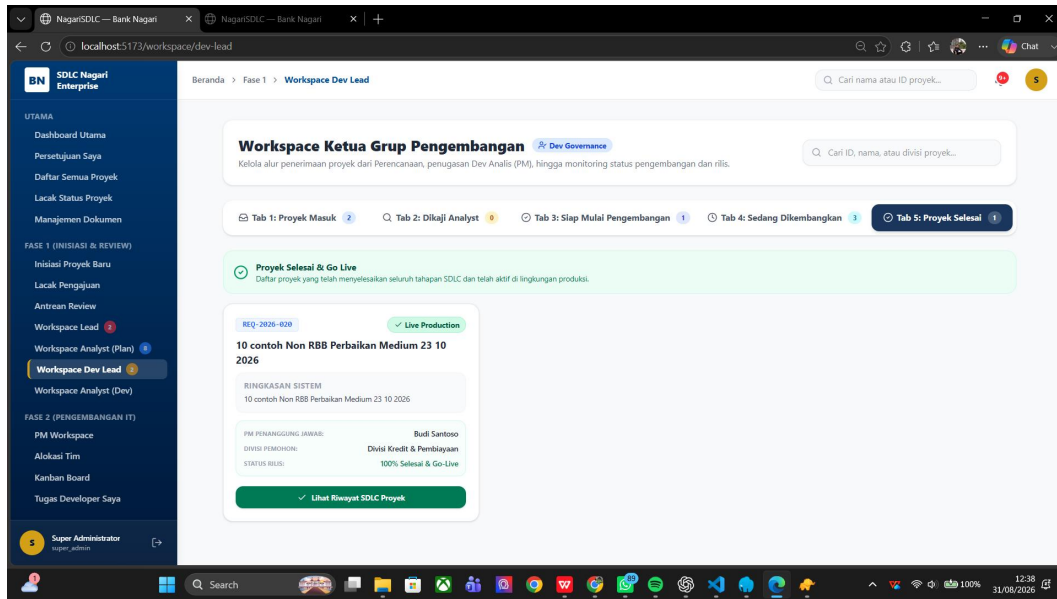
Gambar Lampiran 6 Monitoring Kerja Analis oleh Lead Pengembangan



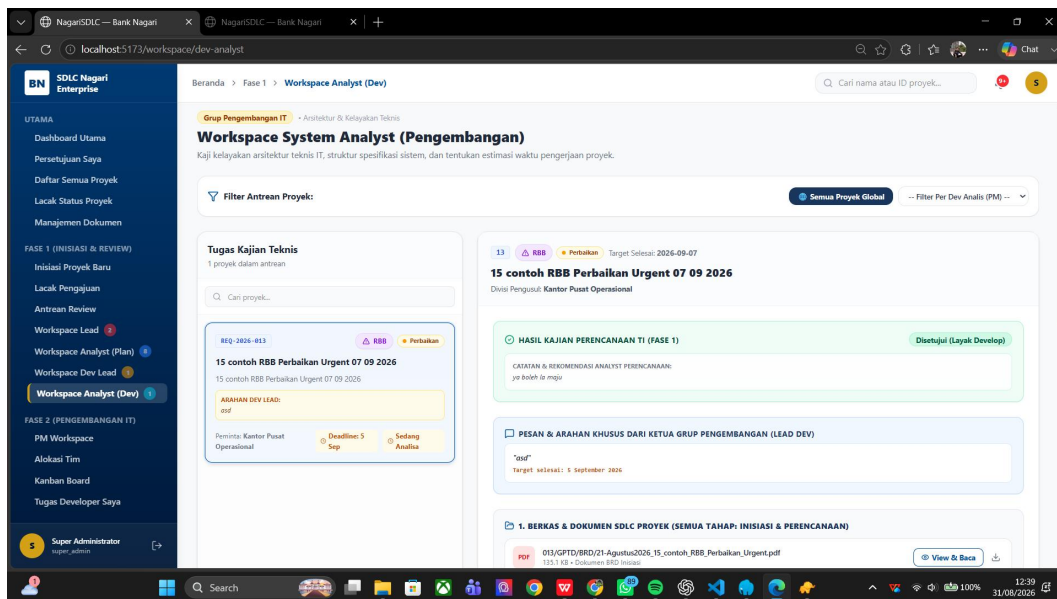
Gambar Lampiran 7 Penunjukan Developer oleh Lead Pengembangan



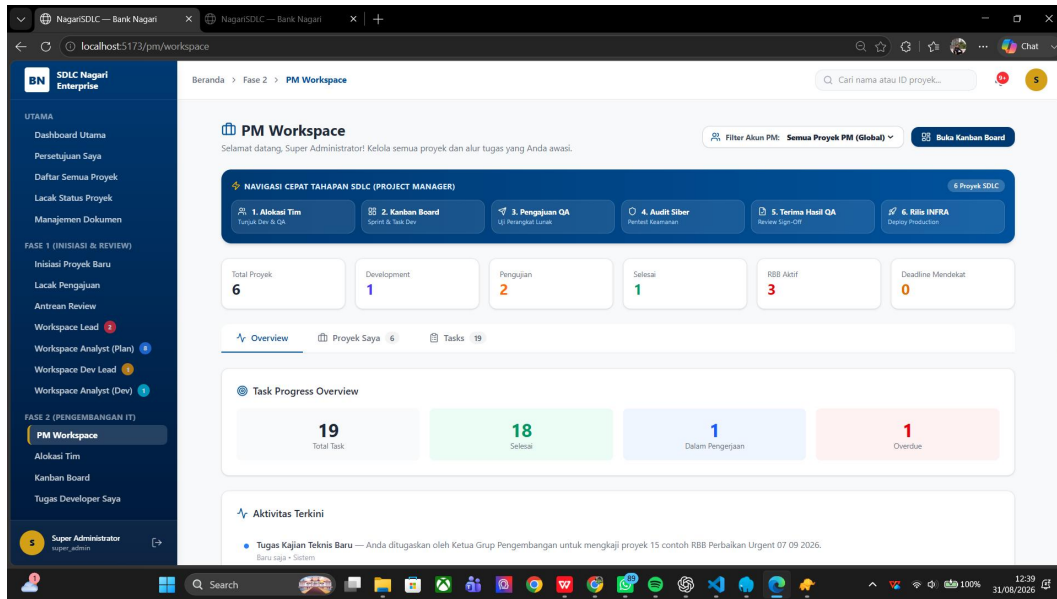
Gambar Lampiran 8 Monitoring Proyek Berjalan oleh Lead Pengembangan



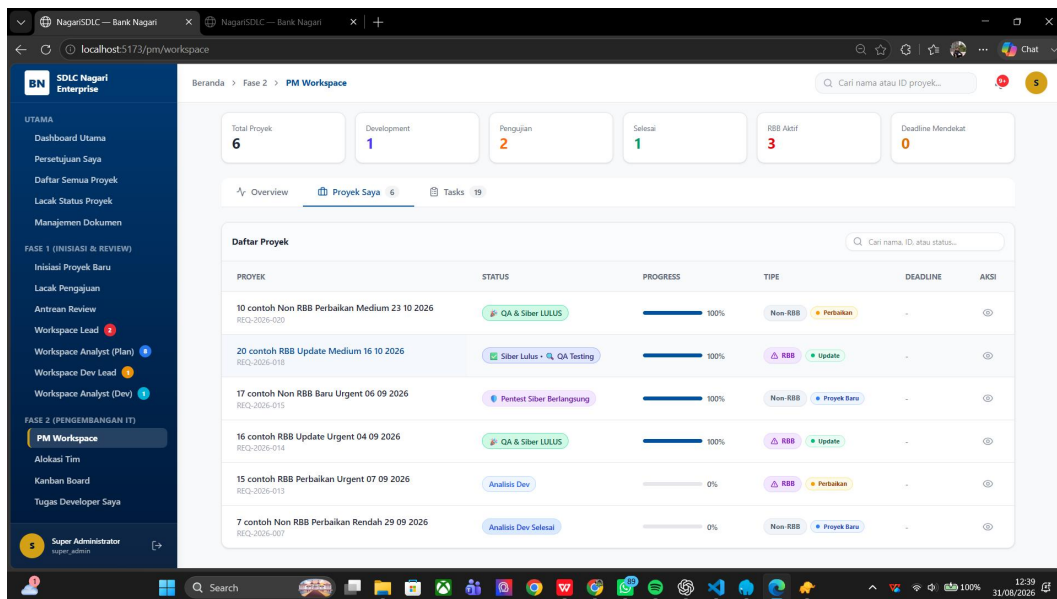
Gambar Lampiran 9 Proyek Selesai



Gambar Lampiran 10 Workspace Analis Pengembangan



Gambar Lampiran 11 Overview Workspace PM



Gambar Lampiran 12 Daftar Proyek PM

The screenshot displays the 'PM Workspace' interface. On the left is a sidebar with navigation options like 'Dashboard Utama', 'Persetujuan Saya', and 'Daftar Semua Proyek'. The main area shows a table of tasks with columns for TASK, PROYEK, ASSIGNEE, STATUS, PRIORITY, and AKSI.

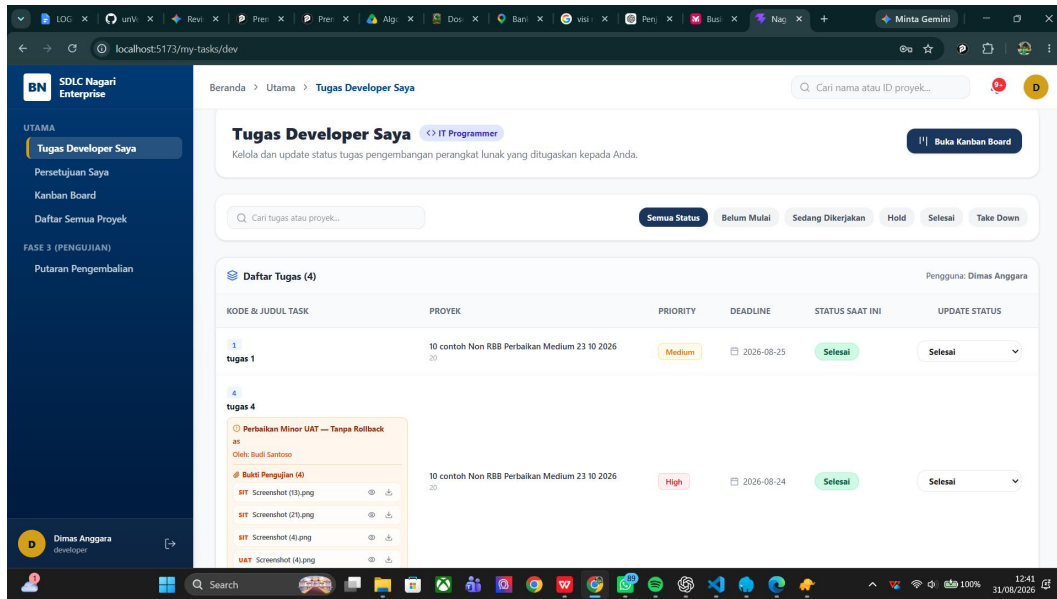
TASK	PROYEK	ASSIGNEE	STATUS	PRIORITY	AKSI
tugas 1	10 contoh Non RBB Perbaikan Medium 23 10 2026	Dimas Anggara	Selesai	Medium	🔗
tugas 2	10 contoh Non RBB Perbaikan Medium 23 10 2026	Gilang Pratama	Selesai	High	🔗
tugas 3	10 contoh Non RBB Perbaikan Medium 23 10 2026	Rina Wati	Selesai	Low	🔗
tugas 4	10 contoh Non RBB Perbaikan Medium 23 10 2026	Dimas Anggara	Selesai	High	🔗
tugas 5	10 contoh Non RBB Perbaikan Medium 23 10 2026	Dimas Anggara	Selesai	Low	🔗
[CR UAT Mayor] nambah fitur	10 contoh Non RBB Perbaikan Medium 23 10 2026	Gilang Pratama	Selesai	High	🔗
[CR UAT Mayor] testing revisi mayor	10 contoh Non RBB Perbaikan Medium 23 10 2026	Gilang Pratama	Selesai	High	🔗
tugas 1	20 contoh RBB Update Medium 16 10 2026	Fani Wijaya	Selesai	Medium	🔗
tugas 2	20 contoh RBB Update Medium 16 10 2026	Fani Wijaya	Take Down	Low	🔗
tugas 3	20 contoh RBB Update Medium 16 10 2026	Eka Putri	Selesai	High	🔗

Gambar Lampiran 13 Daftar Task Proyek PM

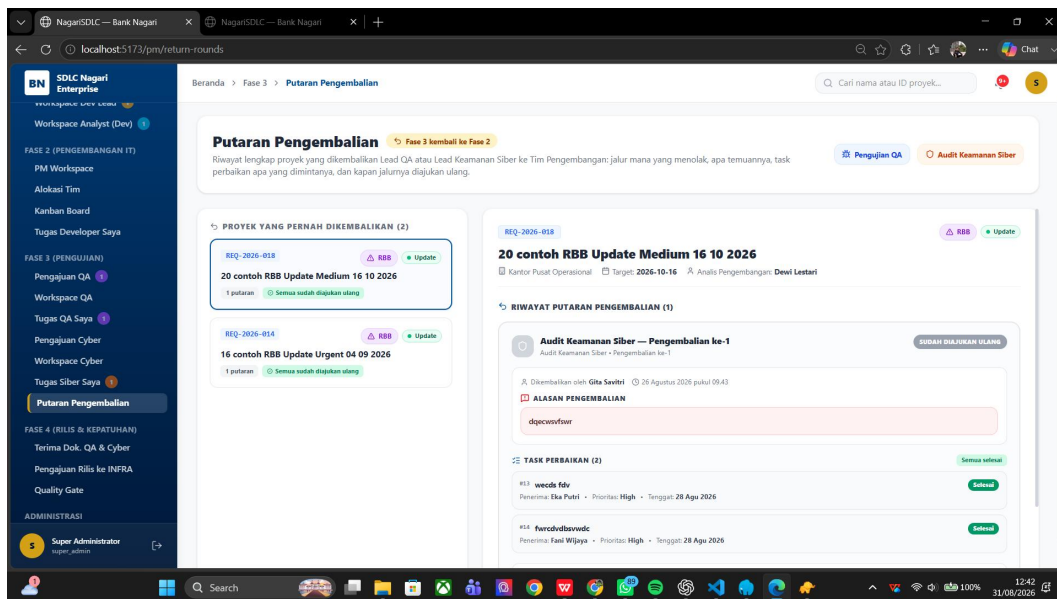
The screenshot shows the 'Alokasi Tim Developer' interface. It includes a sidebar with navigation options and a main area with sections for 'PROYEK DIKELOLA (3)', '17 contoh Non RBB Baru Urgent 06 09 2026', and 'Daftar Kandidat Developer'. The 'Daftar Kandidat Developer' section contains a table of potential developers.

PILIH	NAMA DEVELOPER	BEBAN KERJA SAAT INI	KETERSEDIAAN
☐	Gilang Pratama	1 Proyek Aktif	Tersedia
☐	Rina Wati	1 Proyek Aktif	Tersedia
☒	Dimas Anggara	1 Proyek Aktif	Ditugaskan Lead

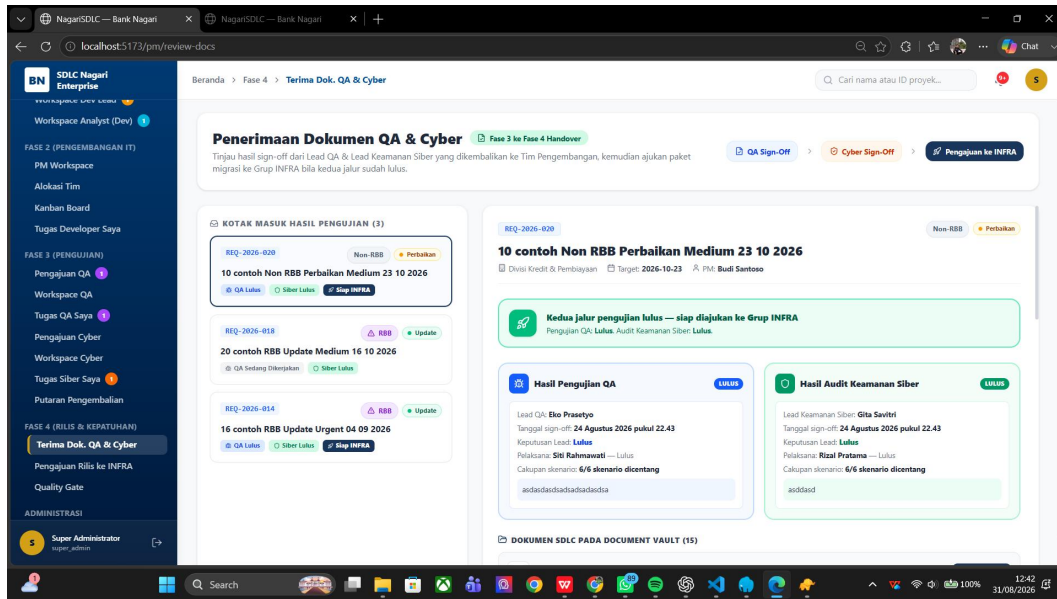
Gambar Lampiran 14 Alokasi Developer oleh PM



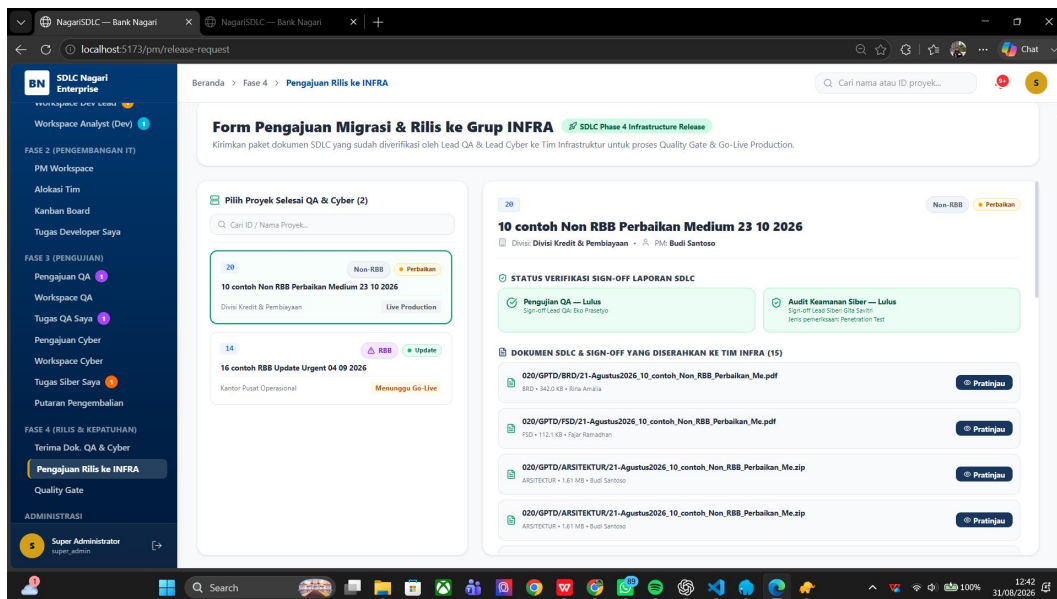
Gambar Lampiran 15 Tampilan Task oleh Developer



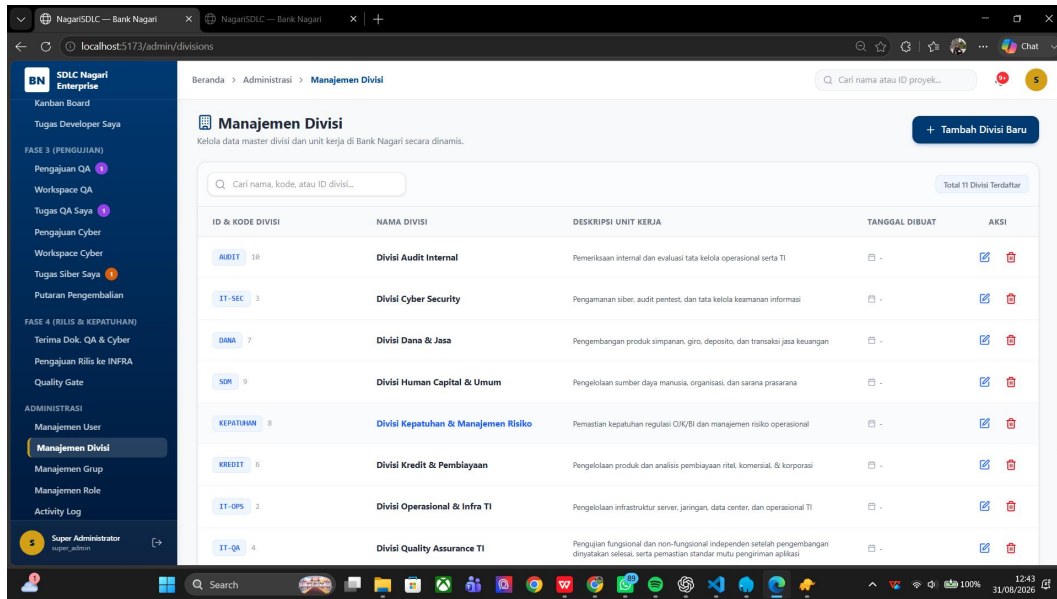
Gambar Lampiran 16 Putaran Pengembalian oleh Siber dan QA



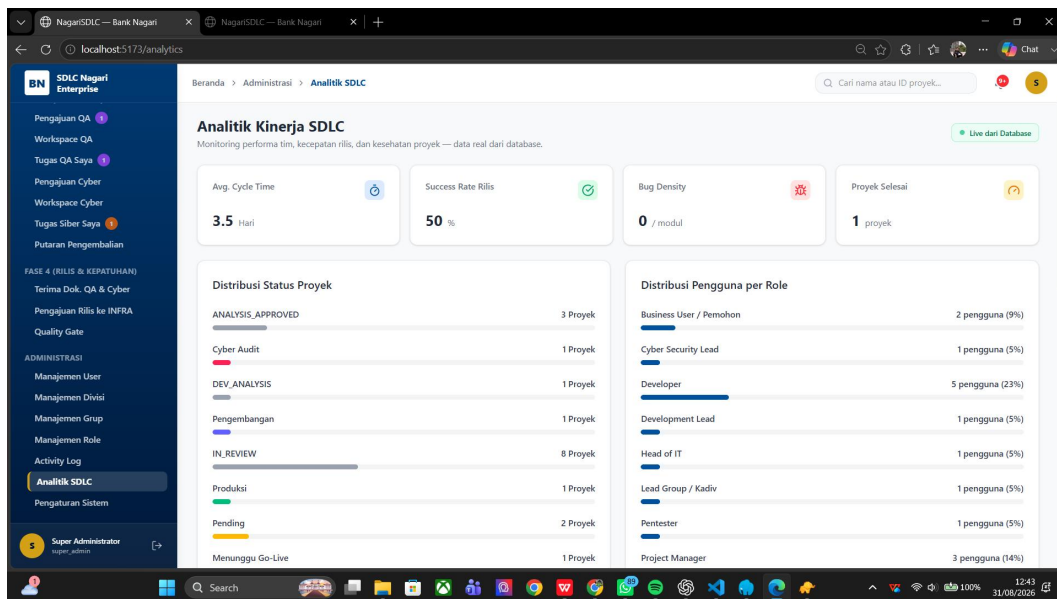
Gambar Lampiran 17 Laman Penerimaan dari QA Siber



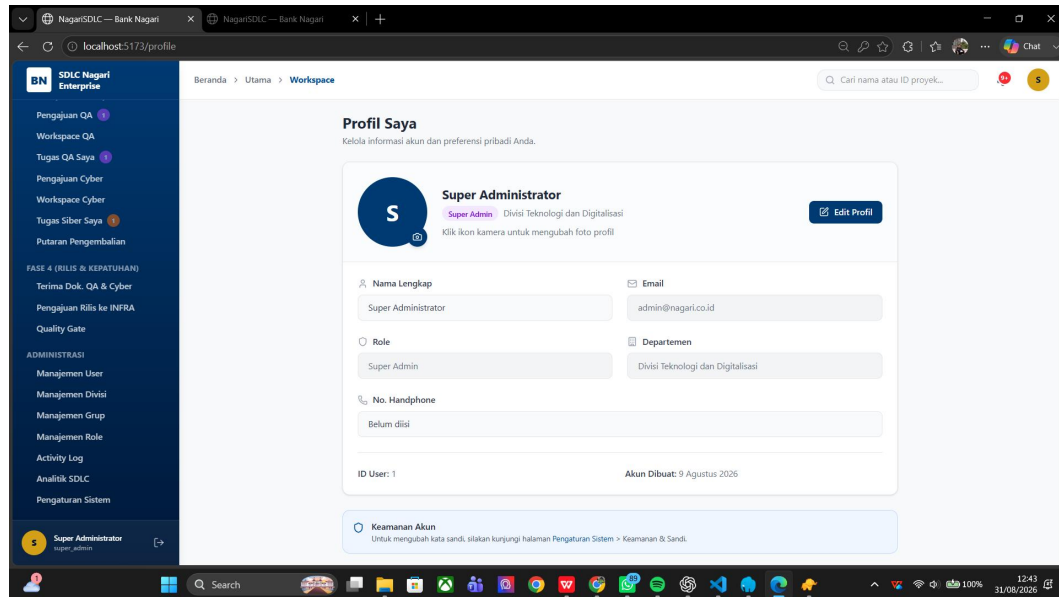
Gambar Lampiran 18 Pengkajuan Ke Grup Infrastruktur



Gambar Lampiran 19 Manajemen Divisi



Gambar Lampiran 20 Analitik SDLC



Gambar Lampiran 21 Profil Super Admin